

# INTERNET OF THINGS

## cho người mới bắt đầu



## Hướng dẫn chi tiết từng bước

# Internet Of Things

## Cho Người Mới Bắt Đầu

Bạn có tò mò về những ngôi nhà thông minh tự động bật đèn khi bạn bước vào, hay những chiếc xe ô tô tự lái có thể tự di chuyển mà không cần người lái? Hay bạn muốn tìm hiểu cách xây dựng hệ thống tưới nước tự động cho khu vườn của mình? Tất cả những điều này đều có thể thực hiện được với công nghệ Internet vạn vật (IoT)!

Internet vạn vật (IoT) đang phát triển nhanh chóng và trở thành một phần thiết yếu trong cuộc sống hiện đại. IoT kết nối các thiết bị thông thường với Internet, cho phép chúng giao tiếp và chia sẻ dữ liệu với nhau. Nhờ vậy, chúng ta có thể điều khiển các thiết bị từ xa, tự động hóa các tác vụ và thu thập dữ liệu thông tin để đưa ra quyết định sáng suốt hơn.

### Tài liệu này dành cho ai?

Đối tượng hướng đến của tài liệu này là:

- Học sinh, sinh viên đam mê công nghệ
- Người mới bắt đầu muốn tìm hiểu về IoT
- Các nhà phát triển phần mềm muốn mở rộng kiến thức sang lĩnh vực IoT
- Cộng đồng maker, những người đam mê các dự án DIY và muốn tự tay xây dựng các dự án IoT

Nói chung tài liệu này là dành cho những người mới bắt đầu muốn khám phá thế giới IoT đầy thú vị. Nó sẽ cung cấp cho bạn một cái nhìn tổng quan về hệ thống IoT, giới thiệu các khái niệm và thành phần cơ bản, và hướng dẫn bạn thực hiện các bài thực hành để áp dụng kiến thức vào thực tế.

Chúng tôi hy vọng tài liệu này là một nguồn tham khảo hữu ích cho những ai muốn bắt đầu hành trình khám phá thế giới IoT đầy thú vị. Với nội dung dễ hiểu, không quá hàn lâm, các bài thực hành thực tế và hướng dẫn chi tiết, bạn sẽ có thể tự tin xây dựng các dự án IoT của riêng mình và góp phần tạo ra một thế giới thông minh hơn.

**Hãy bắt đầu hành trình IoT của bạn ngay hôm nay!**

**OhStem Education**

# Mục lục

|                                                        |           |
|--------------------------------------------------------|-----------|
| <b>Mục lục.....</b>                                    | <b>2</b>  |
| <b>Chương 1: Giới thiệu về Internet of Things.....</b> | <b>4</b>  |
| Giới thiệu.....                                        | 5         |
| Internet of Things (IoT) là gì?.....                   | 5         |
| Ứng dụng của IoT.....                                  | 6         |
| Các thành phần trong hệ thống IoT.....                 | 7         |
| 1. Thiết bị (Things):.....                             | 7         |
| 2. IoT Gateway:.....                                   | 8         |
| 3. IoT Platform:.....                                  | 8         |
| 4. Giao diện người dùng (User Interface):.....         | 8         |
| Thiết bị cho nhà phát triển (IoT Dev Kit).....         | 9         |
| Lựa chọn phần cứng cho giáo trình này.....             | 10        |
| Bộ công cụ phát triển Yolo UNO Starter Kit.....        | 10        |
| Thực hành: Thiết lập môi trường phát triển cho.....    | 13        |
| Yolo UNO.....                                          | 13        |
| Lập trình bằng ngôn ngữ khối lệnh.....                 | 13        |
| Lập trình bằng Arduino.....                            | 13        |
| <b>Chương 2: Thiết bị IoT.....</b>                     | <b>14</b> |
| Giới thiệu.....                                        | 14        |
| Things.....                                            | 14        |
| Gateway - Edge devices.....                            | 15        |
| Vi điều khiển.....                                     | 16        |
| CPU.....                                               | 17        |
| Bộ nhớ RAM.....                                        | 18        |
| Input - Output (IO).....                               | 19        |
| Framework và hệ điều hành.....                         | 19        |
| Arduino.....                                           | 20        |
| Single Board Computer - SBC.....                       | 22        |
| Raspberry Pi.....                                      | 23        |
| Lập trình máy tính SBC.....                            | 24        |
| Máy tính SBC trong các dự án IoT chuyên nghiệp.....    | 25        |
| Thực hành: Viết chương trình cho vi điều khiển.....    | 25        |
| Lập trình bằng khối lệnh và OhStem App.....            | 25        |
| Lập trình bằng Arduino.....                            | 25        |
| <b>Chương 3: Sensor &amp; Actuator.....</b>            | <b>26</b> |
| Giới thiệu.....                                        | 26        |
| Cảm biến là gì?.....                                   | 26        |
| Các loại cảm biến.....                                 | 27        |
| Cảm biến analog.....                                   | 27        |
| Bộ chuyển đổi Analog to Digital Converter (ADC).....   | 28        |
| Cảm biến digital.....                                  | 28        |
| Thực hành: Lập trình đọc cảm biến.....                 | 30        |
| Thiết bị truyền động là gì?.....                       | 30        |

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| Các loại thiết bị truyền động.....                               | 30        |
| Thiết bị truyền động Analog.....                                 | 30        |
| PWM - Điều chế độ rộng xung.....                                 | 31        |
| Thiết bị truyền động digital.....                                | 32        |
| Thực hành: Lập trình bộ truyền động.....                         | 33        |
| Sự kết hợp giữa cảm biến và bộ truyền động.....                  | 34        |
| <b>Chương 4: Kết nối Internet.....</b>                           | <b>35</b> |
| Giới thiệu.....                                                  | 35        |
| Giao thức truyền thông (Communication Protocols).....            | 35        |
| Giao thức MQTT.....                                              | 36        |
| Publish - Subscribe.....                                         | 36        |
| Telemetry (dữ liệu đo từ xa).....                                | 37        |
| Command (lệnh).....                                              | 38        |
| QoS.....                                                         | 39        |
| Retain.....                                                      | 39        |
| LWT.....                                                         | 40        |
| Tần suất gửi dữ liệu.....                                        | 40        |
| Mất kết nối.....                                                 | 41        |
| Thực hành thử nghiệm giao thức MQTT.....                         | 42        |
| Kết nối với server MQTT.....                                     | 42        |
| Publish và Subscribe.....                                        | 43        |
| Publish.....                                                     | 43        |
| Subscribe.....                                                   | 44        |
| Dự án: Giám sát nhiệt độ phòng và bật tắt quạt qua Internet..... | 45        |
| <b>Chương 5: IoT Platform.....</b>                               | <b>47</b> |
| IoT Platform là gì?.....                                         | 47        |
| Giới thiệu nền tảng Core IoT.....                                | 48        |
| Thực hành: Tìm hiểu và thao tác cơ bản với.....                  | 49        |
| Core IoT Platform.....                                           | 49        |
| Tạo tài khoản trên Core IoT.....                                 | 49        |
| Tạo Device trên Core IoT.....                                    | 50        |
| Màn hình điều hành (Dashboard).....                              | 53        |
| <b>Phụ lục 1: Hướng dẫn lập trình Yolo UNO bằng Arduino.....</b> | <b>65</b> |
| Giới thiệu về Arduino.....                                       | 65        |
| Tải và cài đặt phần mềm Arduino IDE.....                         | 65        |
| Làm quen với phần mềm Arduino IDE.....                           | 69        |
| Các nút chức năng trên giao diện.....                            | 69        |
| Cấu trúc một chương trình của Arduino.....                       | 70        |
| Comment.....                                                     | 71        |
| Hàm setup().....                                                 | 72        |
| Hàm loop().....                                                  | 73        |
| Upload code trong Arduino IDE.....                               | 73        |
| Cách thêm thư viện cho Arduino IDE.....                          | 74        |
| Tại sao chúng ta cần thêm thư viện cho Arduino?.....             | 74        |

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| Cấu trúc của thư viện Arduino.....                                     | 75        |
| Cách 1: Cách thêm thư viện cho Arduino IDE dùng Libraries Manager..... | 75        |
| Cách 2: Cách thêm thư viện cho Arduino IDE từ file .zip.....           | 76        |
| Giao tiếp dữ liệu với máy tính.....                                    | 76        |
| Gửi data từ board Arduino đến máy tính.....                            | 76        |
| Gửi data từ máy tính đến Arduino.....                                  | 78        |
| Làm việc với cảm biến ánh sáng.....                                    | 79        |
| Cảm biến ánh sáng.....                                                 | 79        |
| Kết nối.....                                                           | 80        |
| Chương trình Arduino.....                                              | 80        |
| Đèn LED tự động.....                                                   | 81        |
| Kết nối.....                                                           | 81        |
| Chương trình Arduino.....                                              | 82        |
| Lập trình PWM với module quạt mini.....                                | 82        |
| Kết nối.....                                                           | 83        |
| Chương trình Arduino.....                                              | 83        |
| Đổi màu đèn led RGB.....                                               | 84        |
| Kết nối.....                                                           | 84        |
| Chương trình Arduino.....                                              | 84        |
| Đèn thông minh dùng LED RGB.....                                       | 85        |
| Kết nối.....                                                           | 85        |
| Chương trình Arduino.....                                              | 85        |
| Hiển thị lên màn hình LCD1602.....                                     | 86        |
| Kết nối.....                                                           | 86        |
| Chương trình Arduino.....                                              | 87        |
| Giám sát và hiển thị nhiệt độ.....                                     | 87        |
| Kết nối.....                                                           | 87        |
| Chương trình Arduino.....                                              | 88        |
| Quạt tự động.....                                                      | 89        |
| Kết nối.....                                                           | 89        |
| Chương trình Arduino.....                                              | 89        |
| Vườn thông minh.....                                                   | 90        |
| Kết nối.....                                                           | 90        |
| Chương trình Arduino.....                                              | 90        |
| Thùng rác thông minh.....                                              | 91        |
| Kết nối.....                                                           | 91        |
| Chương trình Arduino.....                                              | 92        |
| <b>Tài liệu tham khảo.....</b>                                         | <b>93</b> |

## Chương 1: Giới thiệu về Internet of Things

## Giới thiệu

Chương này cung cấp cho bạn một cái nhìn tổng quan về công nghệ Internet of Things (IoT), ứng dụng, các thành phần quan trọng của một hệ thống IoT. Phần thực hành cuối chương sẽ hướng dẫn bạn làm quen với bộ kit sử dụng trong giáo trình này, thiết lập các công cụ lập trình để sẵn sàng cho các dự án ở các chương sau.

Các nội dung chúng ta sẽ đề cập đến trong chương này bao gồm:

- 'Internet of Things' là gì?
- Một số ứng dụng của IoT
- Các thiết bị dành cho nhà phát triển
- Làm quen bộ kit Yolo UNO và cài đặt môi trường phát triển

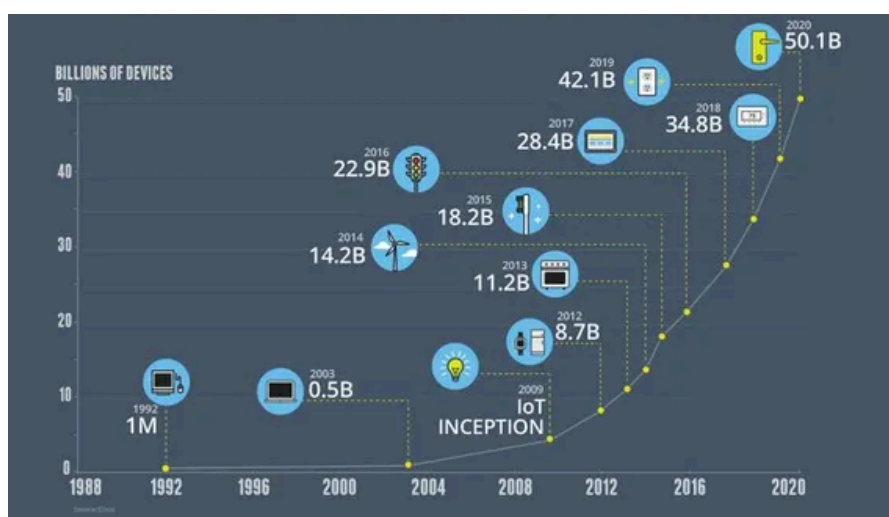
## Internet of Things (IoT) là gì?

Thuật ngữ 'Internet of Things' được [Kevin Ashton](#) đặt ra vào năm 1999, để chỉ việc kết nối Internet với các thiết bị nhỏ gọn, có khả năng đo đạc, giám sát điều kiện môi trường thông qua các cảm biến, sau đó gửi dữ liệu này về một máy chủ tập trung (server). Kể từ đó, thuật ngữ này đã được sử dụng để mô tả bất kỳ thiết bị nào tương tác với thế giới vật lý xung quanh nó, bằng cách thu thập dữ liệu từ cảm biến (sensor) hoặc cung cấp các tương tác trong thế giới thực thông qua bộ truyền động (actuator - thiết bị thực hiện một số thao tác như bật công tắc hoặc bật đèn LED). Các thiết bị này sẽ được bổ sung thêm khả năng kết nối với mạng Internet, để truyền tải dữ liệu lên điện toán đám mây (cloud server).



Cảm biến thu thập thông tin từ thế giới xung quanh, chẳng hạn như đo nhiệt độ, ánh sáng, tốc độ hoặc vị trí... Thiết bị truyền động chuyển đổi tín hiệu điện thành các tương tác trong thế giới thực như kích hoạt công tắc, bật đèn, tạo âm thanh hoặc gửi tín hiệu điều khiển đến phần cứng khác, chẳng hạn như để bật ổ cắm điện.

IoT là một lĩnh vực công nghệ đang phát triển nhanh chóng. Nhờ việc tối ưu kích thước và giá thành của các cảm biến cũng như thiết bị truyền động, mà các ứng dụng IOT có thể dễ dàng được hiện thực và tích hợp. Ước tính đến cuối năm 2020, 50 tỷ thiết bị IoT đã được triển khai và kết nối vào mạng Internet. Nhìn về tương lai, ước tính đến năm 2025, các thiết bị IoT sẽ thu thập được gần 80 zettabyte dữ liệu hoặc 80 nghìn tỷ gigabyte. Đó là rất nhiều dữ liệu, là nguồn thông tin quan trọng cho các công nghệ phía sau IOT, chẳng hạn như phân tích dữ liệu, khoa học dữ liệu hay các bộ não nhân tạo.



Sự phát triển của các thiết bị kết nối vạn vật: Hơn 50 tỷ thiết bị vào cuối năm 2020 (Kết quả tham khảo từ Cisco).

## Ứng dụng của IoT

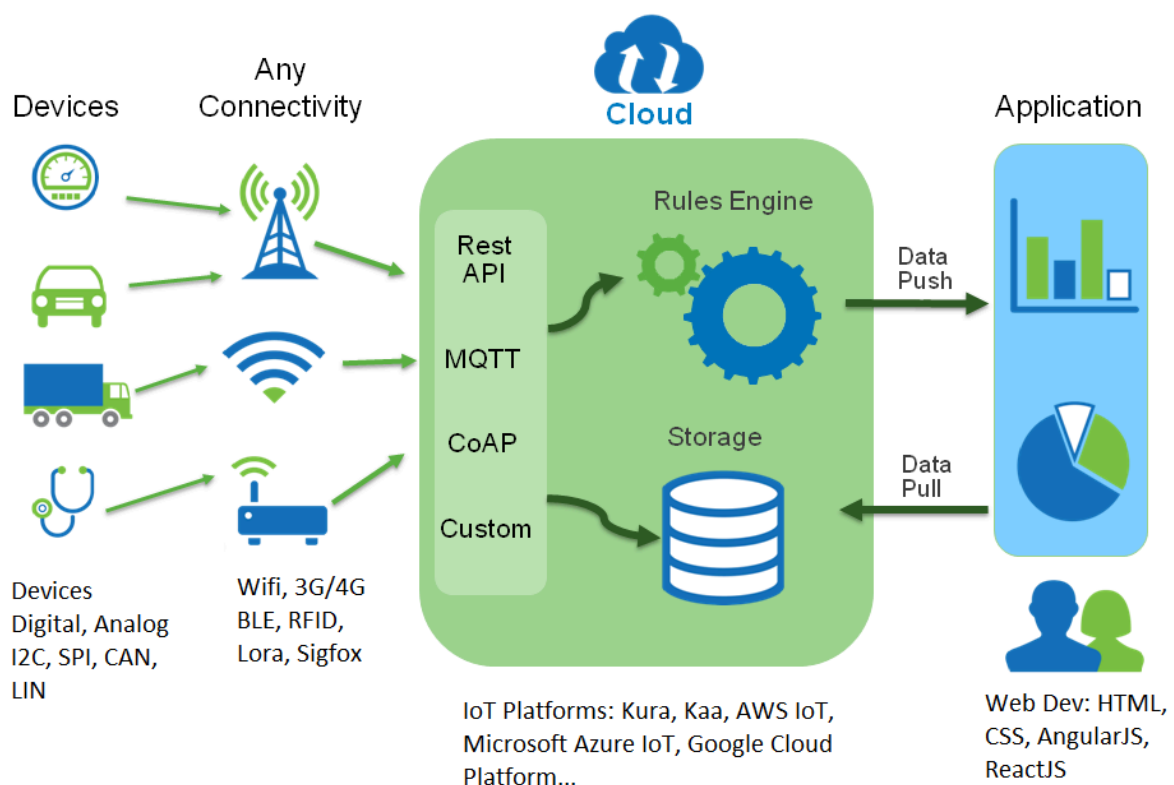
Công nghệ IoT có thể được ứng dụng trong rất nhiều lĩnh vực và có thể chia thành 1 số nhóm lớn như:

- Giải pháp tiêu dùng (smart home, wearable)
- Giải pháp thương mại, doanh nghiệp (smart tracking, building management system...)
- Công nghiệp (smart factory, smart agriculture, smart healthcare...)
- Cơ sở hạ tầng (smart city, smart grid...)



## Các thành phần trong hệ thống IoT

Dưới đây là kiến trúc một hệ thống IoT tiêu biểu:



### 1. Thiết bị (Things):

- Là các thiết bị được tích hợp cảm biến, bộ vi điều khiển và có khả năng kết nối ra Internet hoặc các gateway trong hệ thống.

- Có nhiệm vụ thu thập dữ liệu về môi trường xung quanh như nhiệt độ, độ ẩm, áp suất, chuyển động, v.v.
- Giao tiếp và chia sẻ dữ liệu với các thiết bị khác qua mạng Internet.
- Ví dụ: cảm biến nhiệt độ, camera thông minh, đồng hồ thông minh, v.v.

## 2. IoT Gateway:

- Hỗ trợ và là trung gian kết nối giữa các thiết bị IoT và mạng Internet.
- Hỗ trợ nhiều giao thức kết nối khác nhau như Wi-Fi, Bluetooth, Zigbee, v.v.
- Giúp quản lý và bảo mật dữ liệu truyền tải giữa các thiết bị.
- Giúp xử lý các dữ liệu thu thập được ở cục bộ trước khi đưa lên Internet để tăng hiệu suất và giảm chi phí xử lý (Edge computing).

## 3. IoT Platform:

- Nhận và lưu trữ các dữ liệu thu thập được từ các thiết bị IoT.
- Phân tích dữ liệu thu thập được từ các thiết bị IoT.
- Hỗ trợ ra quyết định và thực hiện các hành động tự động theo kịch bản được cấu hình sẵn (Rules Engine).
- Thường được triển khai trên các dịch vụ đám mây

## 4. Giao diện người dùng (User Interface):

- Cung cấp giao diện cho người dùng tương tác với hệ thống IoT.
- Hiển thị dữ liệu được thu thập và phân tích từ các thiết bị IoT.
- Cho phép người dùng điều khiển và quản lý các thiết bị IoT.
- Bao gồm ứng dụng di động, trang web, bảng điều khiển, v.v.
- Ví dụ: ứng dụng Smart Home, bảng điều khiển điều khiển tòa nhà, v.v.

Ngoài ra, hệ thống IoT có thể bao gồm các thành phần khác như:

- Hệ thống lưu trữ: lưu trữ dữ liệu thu thập được từ các thiết bị IoT.
- Hệ thống bảo mật: bảo vệ hệ thống IoT khỏi các mối đe dọa an ninh mạng.
- Hệ thống quản lý: quản lý và điều phối các hoạt động của hệ thống IoT.

Tóm lại, một hệ thống IoT bao gồm nhiều thành phần khác nhau hoạt động cùng nhau để thu thập, truyền tải, phân tích và xử lý dữ liệu từ các thiết bị IoT. Nhờ đó, hệ thống IoT giúp con người đưa ra quyết định sáng suốt hơn, tự động hóa các quy trình và cải thiện chất lượng cuộc sống.

## Thiết bị cho nhà phát triển (IoT Dev Kit)

---

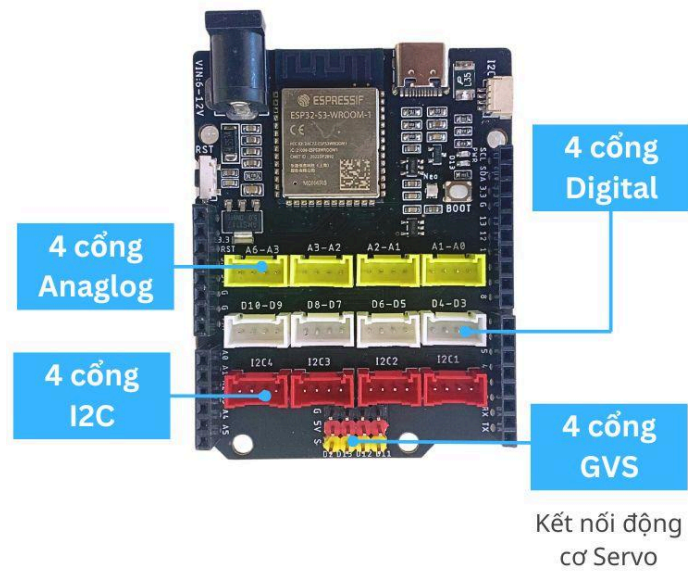
Khác với các công ty công nghiệp hoặc thương mại, thường cần một nền tảng lập trình hoàn chỉnh và đặt theo nhu cầu riêng, một người mới bắt đầu phát triển hoặc đang tìm hiểu về IoT hoặc đang nghiên cứu tạo ra các thiết bị mẫu, bạn thường sẽ cần bắt đầu với bộ công cụ dành riêng cho nhà phát triển, gọi là Dev Kit.

Đặc điểm của các bộ kit này là chúng được thiết kế để các nhà phát triển dễ dàng sử dụng và thử nghiệm cũng như dò gỡ lỗi, thường có nhiều tính năng mà có thể sẽ không có trên phiên bản để sản xuất. Sau khi việc phát triển có thể đi đến sản phẩm mẫu, một phiên bản tối giản cho ứng dụng của bạn có thể được sản xuất, các tính năng không cần thiết trên bộ Dev Kit có thể được lược bỏ để giảm chi phí.

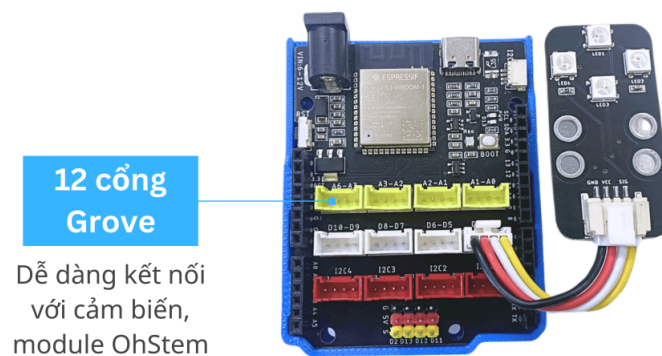
Những bộ Dev Kit này thường được chia thành hai loại - bộ vi điều khiển (MicroController Unit - MCU) và máy tính bo mạch đơn (Single Board Computer - SBC). Chúng ta sẽ tìm hiểu chi tiết hơn 2 loại này trong chương tiếp theo. Bộ công cụ dành cho nhà phát triển vi điều khiển thường đi kèm với các cảm biến và bộ truyền động tích hợp sẵn trên bo mạch hoặc đi kèm dưới dạng module riêng trong bộ kit. Một số bộ vi điều khiển có tích hợp kết nối không dây như Bluetooth hoặc WiFi hoặc có thêm bộ vi điều khiển trên bo mạch để thêm kết nối này.

**Yolo UNO Starter Kit** của OhStem cũng là một Dev Kit có nhiều các công cụ và module mở rộng giúp bạn dễ dàng bắt đầu làm quen và thực hành các chủ đề kiến thức trong việc xây dựng một hệ thống IoT. Dựa trên bộ xử lý trung tâm ESP32 thế hệ mới nhất S3, cùng các ngoại vi phong phú và đa dạng, Yolo UNO là một nền tảng lập trình tốt cho các ứng dụng dựa trên nền tảng kết nối vạn vật ở phạm vi vừa và nhỏ.

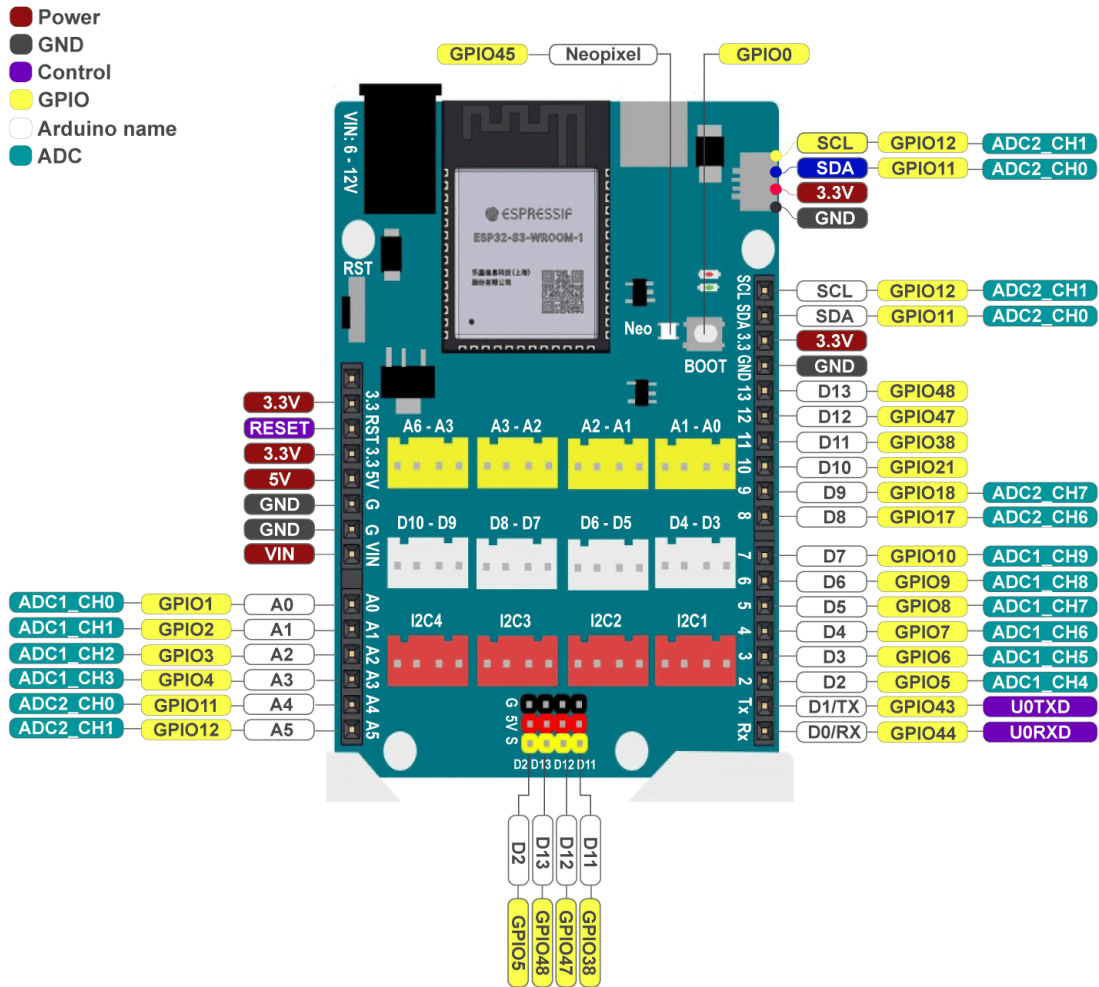




Với bộ kit này, chúng ta có thể kết nối các linh kiện điện tử với mạch xử lý dễ dàng qua cổng cắm chuẩn Grove, được thiết kế theo kiểu Plug & Play, chống cắm ngược chiều để tránh gây hư hỏng board mạch. Điều này sẽ giúp bạn rất nhiều khi mới bắt đầu làm quen với các thiết bị IoT.



Mạch Yolo UNO có kích thước và sơ đồ chân tương thích với mạch Arduino Uno truyền thống:



Bạn có thể lập trình Yolo UNO hoạt động bằng ngôn ngữ kéo thả khối lệnh, Python hoặc C đều được. Trong giáo trình này, chúng tôi sẽ giới thiệu cho bạn cả 2 ngôn ngữ là kéo thả khối lệnh và Arduino. Chúng tôi cũng sẽ bổ sung thêm hướng dẫn cho ngôn ngữ Python trong thời gian tới.



```
from machine import Pin, Timer
led = Pin(25, Pin.OUT)
timer = Timer()

def blink(timer):
    led.toggle()
```

Bạn có thể tham khảo [link tài liệu](#) của nhà sản xuất để có thông tin chi tiết hơn.

## Thực hành: Thiết lập môi trường phát triển cho Yolo UNO

---

Trước khi có thể bắt đầu làm việc với bộ kit thiết bị IoT của mình, bạn sẽ cần thực hiện cài đặt và thiết lập môi trường phát triển.

### Lập trình bằng ngôn ngữ khối lệnh

Để lập trình bằng ngôn ngữ khối lệnh, bạn tham khảo các hướng dẫn sau đây để cập nhật firmware cho Yolo UNO và các thao tác để kết nối, lập trình trên phần mềm OhStem App

- [Hướng dẫn cập nhật firmware](#)
- [Các thao tác với Yolo UNO trên OhStem App](#)

### Lập trình bằng Arduino

Để lập trình bằng Arduino, bạn cần cài đặt phần mềm Arduino IDE và thư viện hỗ trợ mạch Yolo UNO trên Arduino. Bạn tham khảo **Phụ lục 1**, phần **“Tải và cài đặt Arduino IDE”** để thực hiện các tác vụ cài đặt này.

## Chương 2: Thiết bị IoT

### Giới thiệu

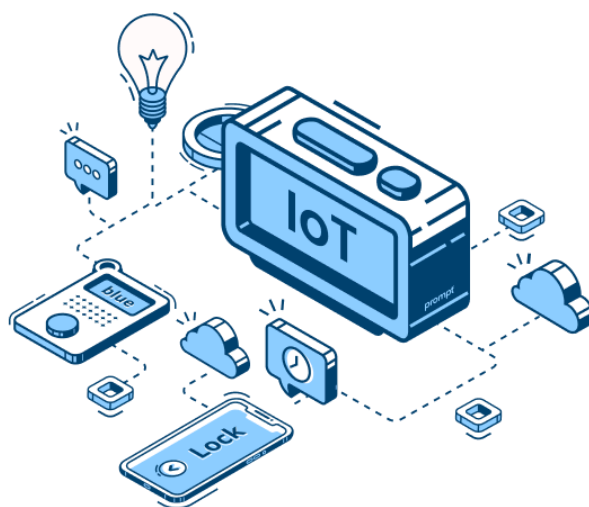
Trong khái niệm Internet of Things (IoT) - Things đại diện cho các thiết bị tương tác với thế giới vật lý xung quanh bằng cách thu thập dữ liệu từ các cảm biến và/ hoặc tương tác lại thế giới thực thông qua bộ truyền động. Ví dụ như một hệ thống tưới cây thông minh, thì độ ẩm đất là cảm biến và máy bơm (kích hoạt khi cảm biến phát hiện đất khô) là một ví dụ cho bộ truyền động.

Trong chương này chúng ta sẽ cùng tìm hiểu sâu hơn bên trong một thiết bị IoT, bộ não điều khiển của chúng và các loại phần cứng phổ biến. Bạn cũng sẽ thực hành viết chương trình đầu tiên cho thiết bị Yolo UNO.

Các chủ đề chúng ta sẽ tìm hiểu bao gồm:

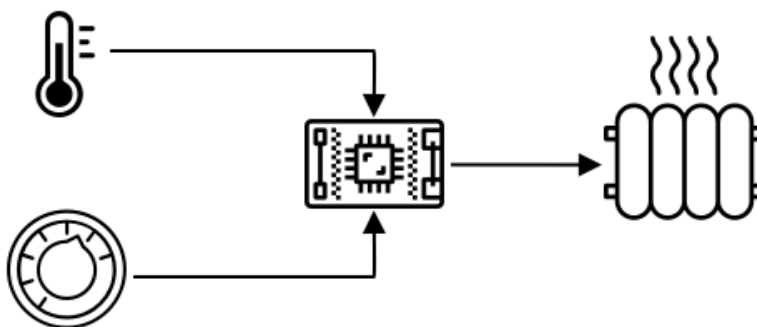
- Things trong IoT
- Vi điều khiển MCU
- Máy tính Single Board Computer - SBC

### Things



Các thiết bị trong lớp Things thường là những máy tính nhỏ, giá rẻ, chạy ở tốc độ thấp và sử dụng điện năng thấp để có thể sử dụng được lâu dài - ví dụ: bộ vi điều khiển đơn giản chỉ có hàng trăm kilobyte bộ nhớ RAM (so với hàng gigabyte trong PC), chỉ chạy ở tốc độ vài trăm MHz (so với GHz của PC), nhưng đôi khi tiêu thụ rất ít năng lượng nên chúng có thể chạy hàng tuần, hàng tháng hoặc thậm chí nhiều năm bằng pin.

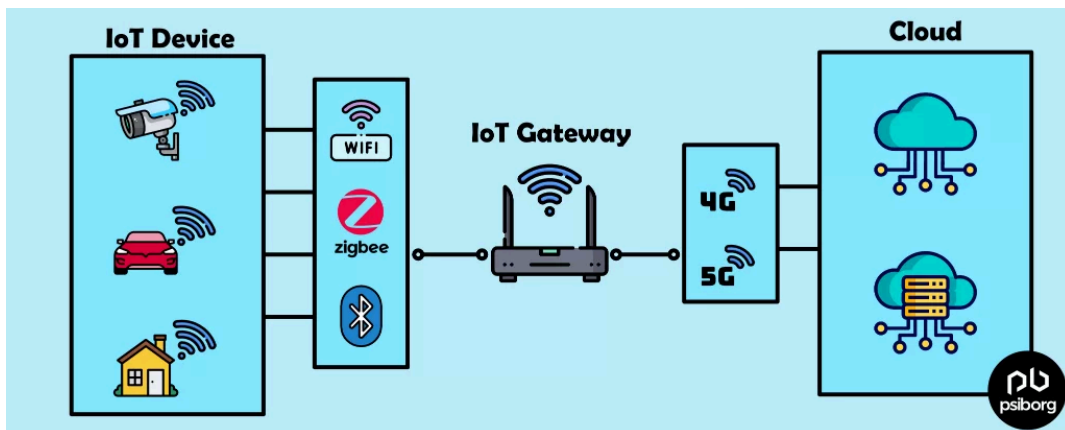
Các thiết bị này tương tác với thế giới vật lý bằng cách sử dụng cảm biến để thu thập dữ liệu từ môi trường xung quanh hoặc bằng cách điều khiển đầu ra hoặc bộ truyền động để thực hiện các thay đổi vật lý. Ví dụ điển hình cho điều này là bộ điều chỉnh nhiệt thông minh - một thiết bị có cảm biến nhiệt độ, giao diện để đặt nhiệt độ mong muốn như mặt số hoặc màn hình cảm ứng và kết nối với hệ thống sưởi hoặc làm mát có thể được bật khi phát hiện nhiệt độ nằm ngoài phạm vi mong muốn. Cảm biến nhiệt độ phát hiện phòng quá lạnh và thiết bị truyền động sẽ bật hệ thống sưởi.



Điện thoại của bạn cũng có thể được coi là một thiết bị IoT đa năng, được tích hợp sẵn các cảm biến và bộ truyền động, với các ứng dụng khác nhau sử dụng cảm biến và bộ truyền động theo những cách khác nhau với các dịch vụ đám mây khác nhau.

## Gateway - Edge devices

Mặc dù chữ I trong IoT là viết tắt của Internet nhưng những thiết bị này không nhất thiết phải kết nối với Internet. Trong một số trường hợp, thiết bị có thể kết nối với các thiết bị gateway hay 'edge' - thiết bị biên chạy trên mạng cục bộ của bạn, nghĩa là bạn có thể xử lý dữ liệu mà không cần thực hiện truyền dữ liệu qua Internet. Điều này có thể nhanh hơn khi bạn có nhiều dữ liệu hoặc kết nối Internet chậm, nó cho phép bạn chạy ngoại tuyến ở những nơi không thể kết nối Internet, chẳng hạn như trên tàu hoặc trong khu vực thảm họa khi ứng phó với khủng hoảng nhân đạo và cho phép bạn giữ dữ liệu riêng tư.



### Vai trò IoT Gateway trong hệ thống IoT

Một ví dụ về điều này là một thiết bị gia đình thông minh như Apple HomePod, Amazon Alexa hoặc Google Home, sẽ nghe giọng nói của bạn bằng các mô hình AI được đào tạo trên đám mây nhưng chạy cục bộ trên thiết bị. Các thiết bị này sẽ “thức dậy” khi một từ hoặc cụm từ nhất định được nói ra và chỉ sau đó mới gửi bài phát biểu của bạn qua Internet để xử lý. Thiết bị sẽ ngừng gửi lời nói tại một thời điểm thích hợp, chẳng hạn như khi phát hiện thấy điểm dừng trong lời nói của bạn. Mọi điều bạn nói trước khi đánh thức thiết bị bằng từ đánh thức và mọi điều bạn nói sau khi thiết bị ngừng nghe sẽ không được gửi qua internet tới nhà cung cấp thiết bị và do đó sẽ ở chế độ riêng tư.

## Vi điều khiển

Trong chương trước, chúng tôi đã nhắc đến bộ vi điều khiển. Bây giờ chúng ta hãy tìm hiểu kỹ hơn về chúng.

Bộ vi điều khiển (còn gọi là MCU, viết tắt của MicroController Unit) là một máy tính nhỏ bao gồm:

- Một hoặc nhiều bộ xử lý trung tâm (CPU) - 'bộ não' của bộ vi điều khiển chạy chương trình của bạn
- Bộ nhớ (RAM và bộ nhớ chương trình) - nơi lưu trữ chương trình, dữ liệu và biến của bạn
- Các kết nối đầu vào/đầu ra (I/O) có thể lập trình - để giao tiếp với các thiết bị ngoại vi bên ngoài (thiết bị được kết nối) như cảm biến và bộ truyền động

Bộ vi điều khiển thường là thiết bị điện toán có giá thành thấp, từ vài trăm ngàn cho đến vài ngàn VND. Tuy nhiên một bộ Dev Kit thường sẽ giá cao hơn do chúng có thêm rất nhiều tính năng và thành phần khác để hỗ trợ trong việc làm quen, thử nghiệm và dò lỗi trong quá trình phát triển.

Bộ vi điều khiển được thiết kế lập trình được để thực hiện một số tác vụ rất cụ thể, thay vì là các máy tính có nhiều mục đích như PC hoặc Mac. Ngoài trừ những trường hợp rất đặc biệt, bạn thường sẽ không thấy các MCU có kết nối màn hình, bàn phím hay chuột như máy tính.

Vi điều khiển thường được lập trình bằng C/C++. Ngoài ra, Python cũng là ngôn ngữ đang dần trở nên phổ biến vì dễ tiếp cận và được dùng trong phần lớn các dự án về AI.

## CPU

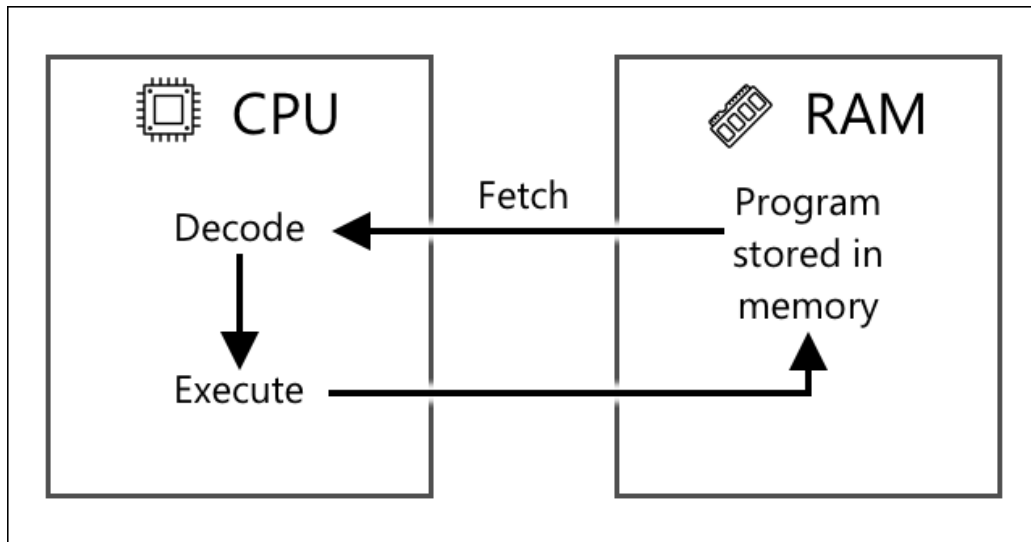
CPU chính là “bộ não” của vi điều khiển. Đó là bộ xử lý chạy mã của bạn và có thể gửi và nhận dữ liệu từ bất kỳ thiết bị được kết nối nào. CPU có thể chứa một hoặc nhiều lõi - về cơ bản là một hoặc nhiều CPU có thể hoạt động cùng nhau để chạy mã của bạn.

CPU dựa vào đồng hồ để đánh dấu hàng triệu hoặc hàng tỷ lần một giây. Mỗi tích tắc hoặc chu kỳ sẽ đồng bộ hóa các hành động mà CPU có thể thực hiện. Với mỗi tích tắc, CPU có thể thực thi một lệnh từ một chương trình, chẳng hạn như truy xuất dữ liệu từ thiết bị bên ngoài hoặc thực hiện phép tính toán học. Chu kỳ thông thường này cho phép tất cả các hành động được hoàn thành trước khi lệnh tiếp theo được xử lý.

Chu kỳ xung nhịp càng nhanh thì càng có nhiều lệnh được xử lý mỗi giây và do đó CPU càng nhanh. Tốc độ CPU được đo bằng [Hertz \(Hz\)](#) , một đơn vị tiêu chuẩn trong đó 1 Hz có nghĩa là một chu kỳ hoặc tích tắc đồng hồ mỗi giây.

Tốc độ CPU thường được tính bằng MHz hoặc GHz. 1 MHz là 1 triệu Hz, 1GHz là 1 tỷ Hz.

CPU thực thi các chương trình theo [chu trình tìm nạp-giải mã-thực thi](#). Với mỗi tích tắc đồng hồ, CPU sẽ lấy lệnh tiếp theo từ bộ nhớ, giải mã và thực thi lệnh đó như sử dụng đơn vị logic số học (ALU) để cộng 2 số. Một số lần thực thi sẽ cần nhiều tích tắc để chạy, vì vậy chu kỳ tiếp theo sẽ chạy ở tích tắc tiếp theo sau khi lệnh hoàn thành.



Bộ vi điều khiển có tốc độ xung nhịp thấp hơn nhiều so với máy tính để bàn, máy tính xách tay hoặc thậm chí hầu hết điện thoại thông minh. Ví dụ: Yolo UNO có CPU chạy ở tốc độ 240 MHz hoặc 240.000.000 chu kỳ mỗi giây.

Mỗi chu kỳ đồng hồ sẽ tiêu thụ năng lượng và tạo ra nhiệt. Tốc độ càng nhanh thì điện năng tiêu thụ càng nhiều và nhiệt sinh ra càng nhiều. PC có bộ tản nhiệt và quạt để loại bỏ nhiệt, vì nếu không có sẽ quá nóng và tắt trong vòng vài giây. Bộ vi điều khiển thường không có chức năng này vì chúng chạy mát hơn nhiều và do đó chậm hơn nhiều. Bộ vi điều khiển có thể chạy trong nhiều ngày, nhiều tháng hoặc thậm chí nhiều năm nếu sử dụng pin nhỏ. Bộ vi điều khiển cũng có thể có các nhân chạy ở tốc độ khác nhau, chuyển sang các nhân tiêu thụ điện năng thấp chậm hơn khi nhu cầu về CPU thấp để giảm mức tiêu thụ điện năng.

Một số PC và Mac đang áp dụng cùng một cách kết hợp giữa nhân năng lượng cao nhanh và nhân năng lượng thấp chậm hơn, chuyển sang tiết kiệm pin. Ví dụ: chip M1 trong máy tính xách tay mới nhất của Apple có thể chuyển đổi giữa 4 nhân hiệu suất cao và 4 nhân hiệu suất thấp để tối ưu hóa thời lượng pin hoặc tốc độ tùy thuộc vào tác vụ đang chạy.

## Bộ nhớ RAM

Bộ vi điều khiển thường có hai loại bộ nhớ - bộ nhớ chương trình và bộ nhớ truy cập ngẫu nhiên (RAM).

Bộ nhớ chương trình không dễ thay đổi, có nghĩa là mọi thứ được ghi vào nó sẽ vẫn tồn tại khi thiết bị không có nguồn điện. Đây là bộ nhớ lưu trữ mã chương trình của bạn.

RAM là bộ nhớ được chương trình sử dụng để chạy, chứa các biến do chương trình của bạn phân bổ và dữ liệu được thu thập từ các thiết bị ngoại vi. Khi mất điện, toàn bộ dữ liệu trong RAM bị mất và được reset lại khi thiết bị khởi động.

Giống như CPU, bộ nhớ RAM trên bộ vi điều khiển có độ lớn nhỏ hơn PC hoặc Mac. Một PC thông thường có thể có 8 Gigabyte (GB) RAM hoặc 8.000.000.000 byte, với mỗi byte đủ dung lượng để lưu trữ một chữ cái hoặc một số từ 0-255. Một bộ vi điều khiển sẽ chỉ có Kilobyte (KB) hoặc Megabyte (MB) RAM, với một kilobyte là 1.000 byte. MCU ESP32-S3 sử dụng trên mạch Yolo UNO có 512KB RAM, hoặc 512.000 byte - ít hơn 15.000 lần so với một PC trung bình!



*So sánh bộ nhớ 8GB (khung hình chữ nhật) và 192KB (dấu chấm)*

Bộ nhớ chương trình cũng nhỏ hơn PC. Một PC thông thường có thể có ổ cứng 500GB để lưu trữ chương trình, trong khi một bộ vi điều khiển có thể chỉ có kilobyte hoặc có thể vài megabyte (MB) dung lượng lưu trữ (1MB là 1.000KB hoặc 1.000.000 byte). Yolo UNO có 8MB dung lượng để lưu trữ chương trình.

## Input - Output (IO)

Bộ vi điều khiển cần kết nối đầu vào và đầu ra (I/O) để đọc dữ liệu từ cảm biến và gửi tín hiệu điều khiển đến bộ truyền động. Chúng thường chứa một số chân đầu vào/đầu ra dùng cho nhiều mục đích (General Purpose Input Output - GPIO). Các chân này có thể được cấu hình trong chương trình lập trình để làm Input (nhận tín hiệu) hoặc Output (xuất ra tín hiệu).

- Chân Input được sử dụng để đọc giá trị từ cảm biến
- Chân Output gửi tín hiệu lệnh đến bộ truyền động

## Framework và hệ điều hành

Do tốc độ và kích thước bộ nhớ thấp, bộ vi điều khiển không chạy hệ điều hành (OS) như máy tính để bàn. Các hệ điều hành phổ biến chạy trên các máy tính như

Windows, Linux hoặc MacOS cần nhiều bộ nhớ và sức mạnh xử lý để chạy các tác vụ hoàn toàn không cần thiết đối với bộ vi điều khiển. Bộ vi điều khiển thường được lập trình chỉ để thực hiện một hoặc vài tác vụ rất cụ thể, không giống như máy tính đa năng như PC hoặc Mac cần hỗ trợ giao diện người dùng, phát nhạc hoặc phim, cung cấp công cụ để viết tài liệu hoặc mã, chơi trò chơi hoặc lên mạng.

Để lập trình một bộ vi điều khiển không có hệ điều hành, bạn cần một số công cụ cho phép bạn viết code theo cách mà bộ vi điều khiển có thể chạy, sử dụng các API có thể giao tiếp với bất kỳ thiết bị ngoại vi nào. Mỗi bộ vi điều khiển đều khác nhau, vì vậy các nhà sản xuất thường hỗ trợ các framework riêng cho phép bạn xây dựng chương trình của mình và để chạy trên bất kỳ bộ vi điều khiển được framework đó hỗ trợ.

Bạn có thể lập trình bộ vi điều khiển bằng HĐH - thường được gọi là hệ điều hành thời gian thực (RTOS), vì chúng được thiết kế để xử lý việc gửi dữ liệu đến và đi từ các thiết bị ngoại vi trong thời gian thực. Các hệ điều hành này rất nhẹ và cung cấp các tính năng như:

- Đa luồng, cho phép code của bạn chạy nhiều nhiệm vụ cùng lúc, trên nhiều lõi hoặc bằng cách thay phiên nhau trên một lõi
- Kết nối mạng để cho phép giao tiếp qua Internet một cách an toàn
- Các thành phần giao diện đồ họa người dùng (GUI) để xây dựng giao diện người dùng (UI) trên các thiết bị có màn hình.

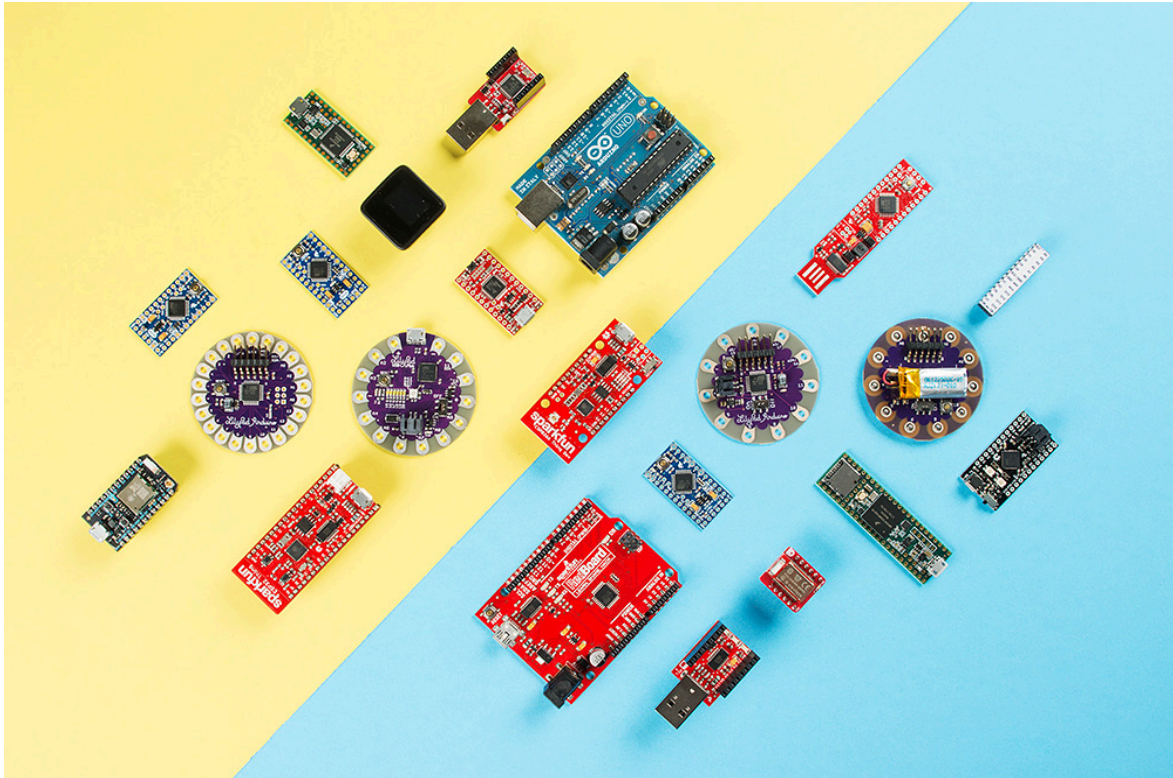
✅ Đọc thêm một số RTOS khác nhau: [Azure RTOS](#) , [FreeRTOS](#) , [Zephyr](#)

## Arduino



[Arduino](#) có lẽ là framework vi điều khiển phổ biến nhất, đặc biệt là trong giới sinh viên, cộng đồng maker và các nhà sản xuất thiết bị.

Arduino là một nền tảng nguồn mở kết hợp cả phần mềm và phần cứng. Bạn có thể mua bo mạch tương thích với Arduino từ chính Arduino hoặc từ các nhà sản xuất khác, sau đó lập trình bằng công cụ phần mềm Arduino IDE.



*Có rất nhiều loại bo mạch tương thích với Arduino*

Yolo UNO cũng là bo mạch có thể lập trình và tương thích hoàn toàn với Arduino IDE.

Bo mạch Arduino có thể lập trình bằng C hoặc C++. Việc sử dụng C/C++ cho phép chương trình của bạn được biên dịch rất nhỏ và chạy nhanh, điều này cần thiết trên một thiết bị bị hạn chế như bộ vi điều khiển.

Arduino cung cấp các thư viện tiêu chuẩn để tương tác với các bộ vi điều khiển và các chân I/O, với các cách triển khai khác nhau để chạy trên các bộ vi điều khiển khác nhau. Ví dụ: hàm [delay](#) sẽ tạm dừng chương trình trong một khoảng thời gian nhất định, hàm [digitalRead](#) sẽ đọc giá trị HIGH hoặc LOW từ chân IO đã cho, bất kể mã được chạy trên bo mạch nào. Các thư viện tiêu chuẩn này có nghĩa là mã Arduino được viết cho một bo mạch có thể được biên dịch lại cho bất kỳ bo mạch Arduino nào khác và sẽ chạy, giả sử rằng các chân giống nhau và các bo mạch hỗ trợ các tính năng giống nhau.

Arduino có một hệ sinh thái đồ sộ các thư viện của bên thứ ba cho phép bạn dễ dàng thêm các tính năng bổ sung vào các dự án Arduino của mình, chẳng hạn như sử dụng cảm biến và bộ truyền động hoặc kết nối với các dịch vụ IoT trên đám mây.

## Single Board Computer - SBC

Trong chương trước chúng ta cũng đã nhắc đến máy tính SBC. Bây giờ hãy cùng nhau tìm hiểu kỹ hơn về loại máy tính đặc biệt này.

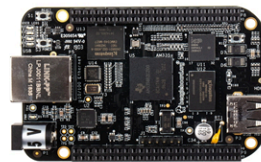
SBC là một thiết bị điện toán nhỏ có tất cả các thành phần của một máy tính hoàn chỉnh được chứa trên một bảng mạch có kích thước nhỏ. Đây là những thiết bị có thông số kỹ thuật gần giống với máy tính để bàn hoặc máy tính xách tay PC hoặc Mac, chạy hệ điều hành đầy đủ nhưng nhỏ, sử dụng ít năng lượng hơn và rẻ hơn đáng kể.



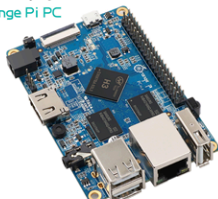
Raspberry Pi Model B



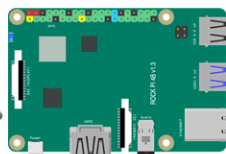
Nvidia Jetson family



BeagleBone Board



Orange Pi



Rock Pi 4



HiKey970



Coral.ai

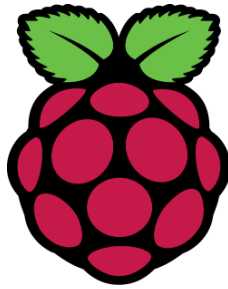
### *Một số loại SBC có trên thị trường*

Giống như vi điều khiển, máy tính SBC có CPU, bộ nhớ và các chân IO nhưng chúng có các tính năng bổ sung như chip đồ họa cho phép bạn kết nối màn hình, đầu ra âm thanh và cổng USB để kết nối chuột, bàn phím và USB tiêu chuẩn khác như webcam hoặc ổ cứng flash. Các chương trình được lưu trữ trên thẻ SD hoặc ổ cứng cùng với hệ điều hành, thay vì chip nhớ được tích hợp trong bo mạch.

Bạn có thể coi SBC như một phiên bản nhỏ hơn, rẻ hơn của PC hoặc Mac mà bạn đang đọc bài viết này, có bổ sung các chân GPIO (đầu vào/đầu ra đa năng) để tương tác với các cảm biến và bộ truyền động.

SBC là loại máy tính có đầy đủ tính năng nên có thể lập trình bằng bất kỳ ngôn ngữ nào. Các thiết bị IoT thường được lập trình bằng Python.

## Raspberry Pi



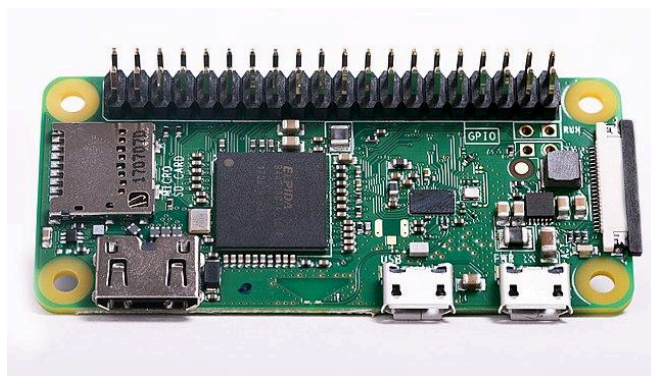
Raspberry [Pi Foundation](#) là tổ chức từ thiện đến từ Vương quốc Anh được thành lập năm 2009 nhằm thúc đẩy nghiên cứu về khoa học máy tính, đặc biệt là ở cấp trường học. Là một phần của sứ mệnh này, họ đã phát triển một máy tính SBC, được gọi là Raspberry Pi. Raspberry Pi hiện có sẵn 3 biến thể - phiên bản kích thước đầy đủ, Pi Zero nhỏ hơn và mô-đun điện toán có thể được tích hợp vào thiết bị IoT cuối cùng của bạn.



*Raspberry Pi là một trong những máy tính SBC phổ biến nhất*

Phiên bản mới nhất của Raspberry Pi kích thước đầy đủ là Raspberry Pi 5, có CPU lõi tứ (4 lõi) chạy ở tốc độ 2.4 GHz, RAM 2, 4 hoặc 8GB, ethernet gigabit, WiFi, 2 cổng HDMI hỗ trợ màn hình 4k, cổng đầu ra âm thanh và video tổng hợp, cổng USB (2 USB 2.0, 2 USB 3.0), 40 chân GPIO, đầu nối camera cho mô-đun máy ảnh Raspberry Pi và khe cắm thẻ SD. Tất cả điều này diễn ra trên một bo mạch có kích thước 88mm x 58mm x 19,5mm và được cấp nguồn bằng nguồn điện 3A USB-C. Raspberry Pi có giá khởi điểm chỉ khoảng 35-40 USD, rẻ hơn nhiều so với PC hoặc Mac.

Ngoài ra còn có máy tính Pi400 all in one được tích hợp Pi4 trên bàn phím.



*Raspberry Pi Zero*

Pi Zero nhỏ hơn nhiều và có công suất thấp hơn. Nó có CPU lõi đơn 1GHz, RAM 512MB, WiFi (ở mẫu Zero W), một cổng HDMI, một cổng micro-USB, 40 chân GPIO, đầu nối camera cho mô-đun máy ảnh Raspberry Pi và thẻ SD chổ. Nó có kích thước 65mm x 30mm x 5mm và tiêu thụ rất ít năng lượng. Zero có giá 5 USD, với phiên bản W có WiFi là 10 USD.

CPU trong cả hai loại này đều là bộ xử lý ARM, trái ngược với bộ xử lý Intel/AMD x86 hoặc x64 mà bạn thấy trong hầu hết PC và Mac. Chúng tương tự như CPU bạn tìm thấy trong một số bộ vi điều khiển, cũng như gần như tất cả các điện thoại di động, Microsoft Surface X và Apple Mac mới dựa trên Apple Silicon.

Tất cả các biến thể của Raspberry Pi đều chạy phiên bản Debian Linux có tên Raspberry Pi OS. Đây là phiên bản rút gọn không giao diện đồ họa GUI, phù hợp cho các dự án mà bạn không cần màn hình hoặc phiên bản đầy đủ với môi trường máy tính để bàn hoàn chỉnh, với trình duyệt web, ứng dụng văn phòng, công cụ mã hóa và trò chơi. Vì HĐH là phiên bản của Debian Linux nên bạn có thể cài đặt bất kỳ ứng dụng hoặc công cụ nào chạy trên Debian và được xây dựng cho bộ xử lý ARM bên trong Pi.

## **Lập trình máy tính SBC**

Máy tính SBC là máy tính đầy đủ, chạy hệ điều hành đầy đủ. Điều này có nghĩa là có rất nhiều ngôn ngữ lập trình, framework và công cụ mà bạn có thể sử dụng để lập trình, không giống như các bộ vi điều khiển dựa vào sự hỗ trợ của các framework như Arduino. Hầu hết các ngôn ngữ lập trình đều có thư viện có thể truy cập vào các chân GPIO để giao tiếp với các cảm biến và bộ truyền động.

Ngôn ngữ lập trình phổ biến nhất để xây dựng các ứng dụng IoT trên Raspberry Pi là Python. Có một hệ sinh thái phần cứng khổng lồ được thiết kế cho Pi và gần như tất cả chúng đều có các thư viện cần thiết để sử dụng với Python. Một số hệ sinh thái này dựa trên các mạch mở rộng gắn lên trên mạch Raspberry Pi và kết nối

thông qua 40 chân GPIO, gọi là các shield HAT. Những chiếc shield này cung cấp các tính năng bổ sung, chẳng hạn như màn hình, cảm biến, điều khiển từ xa hoặc các shield mở rộng để cho phép bạn cảm biến bằng các cấp tiêu chuẩn.

## Máy tính SBC trong các dự án IoT chuyên nghiệp

SBC thường được sử dụng để triển khai các dự án IoT chuyên nghiệp, không như các bộ Dev kit chủ yếu dành cho nhà phát triển và quá trình làm các dự án mẫu. Chúng có thể cung cấp năng lực tính toán đủ mạnh mẽ để chạy các tác vụ phức tạp như chạy các mô hình học máy.

Ví dụ: một [mô-đun tính toán Raspberry Pi 4](#) (compute module) cung cấp toàn bộ sức mạnh của Raspberry Pi 4 nhưng ở dạng nhỏ gọn và rẻ hơn, không có hầu hết các cổng, được thiết kế để cài đặt vào phần cứng tùy chỉnh.

## Thực hành: Viết chương trình cho vi điều khiển

---

Trong phần thực hành này, bạn sẽ tìm hiểu cách viết một chương trình đầu tiên cơ bản nhất của thế giới điện tử cho mạch Yolo UNO của bạn, đó là chương trình bật tắt đèn LED (blink led).

### Lập trình bằng khối lệnh và OhStem App

Bạn tham khảo các hướng dẫn ở link sau:

- [https://docs.ohstem.vn/en/latest/yolo\\_uno/lam\\_quen/cac\\_thao\\_tac\\_co\\_ban.html](https://docs.ohstem.vn/en/latest/yolo_uno/lam_quen/cac_thao_tac_co_ban.html)
- [https://docs.ohstem.vn/en/latest/yolo\\_uno/lam\\_quen/bat\\_tat\\_den.html](https://docs.ohstem.vn/en/latest/yolo_uno/lam_quen/bat_tat_den.html)

### Lập trình bằng Arduino

Bạn tham khảo **Phụ lục 1**, phần **Upload code trong Arduino IDE**.

## Chương 3: Sensor & Actuator

### Giới thiệu

---

Trong phần này, bạn sẽ tìm hiểu 2 khái niệm quan trọng của một thiết bị IoT là sensor (cảm biến) và actuator (truyền động hay điều khiển). Chúng đóng vai trò quan trọng trong việc thu thập dữ liệu từ môi trường xung quanh và điều khiển các thiết bị vật lý.

Bạn cũng sẽ thực hành viết chương trình để mạch Yolo UNO làm việc với các loại thiết bị này trong dự án đèn thông minh bằng cách kết hợp cảm biến ánh sáng và module đèn LED.

Trong bài học này chúng ta sẽ đề cập đến:

- Cảm biến là gì?
- Sử dụng cảm biến
- Các loại cảm biến
- Thiết bị truyền động là gì?
- Sử dụng bộ truyền động
- Các loại thiết bị truyền động

### Cảm biến là gì?

---

Cảm biến là thiết bị phản ứng cảm nhận thế giới vật lý - nghĩa là chúng đo lường một hoặc nhiều thuộc tính xung quanh chúng và gửi thông tin đến thiết bị IoT. Cảm biến bao phủ rất nhiều loại thiết bị vì có rất nhiều thứ có thể đo được, từ các đặc tính tự nhiên như nhiệt độ không khí đến các tương tác vật lý như chuyển động.

Một số cảm biến phổ biến bao gồm:

- Cảm biến nhiệt độ - những cảm biến này cảm nhận nhiệt độ không khí hoặc nhiệt độ của vật thể mà chúng được nhúng vào.
- Nút nhấn - phát hiện được khi chúng được nhấn.

- Cảm biến ánh sáng - phát hiện các mức ánh sáng và có thể các màu cụ thể, tia UV, tia hồng ngoại hoặc ánh sáng nhìn thấy chung.
- Camera - chụp được hình ảnh hoặc truyền phát video.
- Gia tốc kế - cảm nhận được chuyển động theo nhiều hướng.
- Micro - phát hiện các âm thanh hay mức độ ồn của môi trường.

Tất cả các cảm biến đều có một điểm chung - chúng chuyển đổi bất cứ thứ gì chúng cảm nhận được thành tín hiệu điện có thể đọc được bởi thiết bị IoT. Cách đọc tín hiệu điện này tùy thuộc vào loại cảm biến, cũng như giao thức được sử dụng để giao tiếp với thiết bị IoT.

## Các loại cảm biến

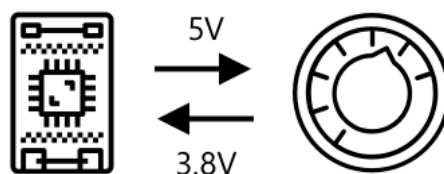
Cảm biến có thể là loại analog hoặc digital.

### Cảm biến analog

Một số cảm biến cơ bản nhất là cảm biến analog. Các cảm biến này nhận được điện áp từ thiết bị IoT, các thành phần cảm biến sẽ điều chỉnh điện áp này và điện áp trả về từ cảm biến sẽ được đo để đưa ra giá trị cảm biến.

Điện áp là thước đo lực đẩy để di chuyển dòng điện từ nơi này sang nơi khác, chẳng hạn như từ cực dương của pin sang cực âm. Ví dụ: pin AA tiêu chuẩn là 1,5V (V là ký hiệu của vôn) và có thể đẩy dòng điện với lực 1,5V từ cực dương đến cực âm của nó. Phần cứng khác nhau yêu cầu điện áp khác nhau để hoạt động, ví dụ, đèn LED có thể phát sáng trong khoảng 2-3V, nhưng bóng đèn dây tóc 100W sẽ cần 240V. Bạn có thể đọc thêm về điện áp trên [trang Wikipedia](#).

Một ví dụ về điều này là một chiết áp. Đây là thiết bị có núm xoay mà bạn có thể xoay giữa hai vị trí và cảm biến sẽ đo góc quay.



Thiết bị IoT sẽ gửi tín hiệu điện đến chiết áp ở điện áp ví dụ như 5 volt (5V). Khi chiết áp được xoay, nó sẽ thay đổi điện áp đi ra từ phía bên kia. Hãy tưởng tượng bạn có một chiết áp được gắn nhãn là mặt số đi từ 0 đến [11](#), chẳng hạn như núm

âm lượng trên bộ khuếch đại. Khi chiết áp ở vị trí tắt hoàn toàn (0) thì 0V (0 volt) sẽ xuất ra. Khi nó ở vị trí tối đa (11), 5V (5 volt) sẽ xuất ra.

Đây là một cách giải thích được đơn giản hóa giúp bạn dễ hiểu. Bạn có thể đọc thêm về chiết áp và điện trở thay đổi trên [trang Wikipedia về chiết áp](#).

Sau đó, điện áp phát ra từ cảm biến sẽ được thiết bị IoT đọc và thiết bị có thể phản hồi với nó. Tùy thuộc vào cảm biến, điện áp này có thể là một giá trị tùy ý hoặc có thể ánh xạ tới một đơn vị tiêu chuẩn. Ví dụ, một cảm biến nhiệt độ tương tự dựa trên [nhiệt điện trở](#) sẽ thay đổi điện trở tùy thuộc vào nhiệt độ. Sau đó, điện áp đầu ra có thể được chuyển đổi thành nhiệt độ tính bằng Kelvin và tương ứng thành °C hoặc °F, bằng cách tính toán trong chương trình.

### **Bộ chuyển đổi Analog to Digital Converter (ADC)**

Các thiết bị IoT là digital - chúng không thể hoạt động với các giá trị analog, mà chỉ hoạt động với các số 0 và 1. Điều này có nghĩa là các giá trị cảm biến analog cần phải được chuyển đổi thành tín hiệu số trước khi chúng có thể được xử lý. Nhiều thiết bị IoT có bộ chuyển đổi Analog sang Digital (ADC) để chuyển đổi đầu vào analog thành giá trị digital. Cảm biến cũng có thể hoạt động với ADC thông qua bảng kết nối.

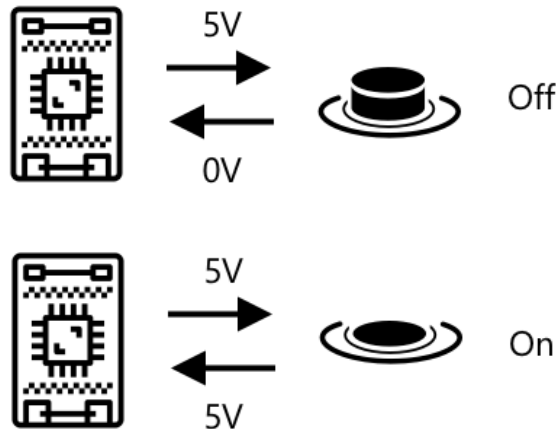
Hãy tưởng tượng bạn có một cảm biến ánh sáng analog được kết nối với thiết bị IoT sử dụng 3,3V và trả về giá trị 1V. 1V này không có ý nghĩa gì trong thế giới digital, vì vậy cần phải được chuyển đổi. Điện áp sẽ được chuyển đổi thành giá trị analog bằng thang đo tùy thuộc vào thiết bị và cảm biến. Một ví dụ là cảm biến ánh sáng trong bộ Yolo UNO Starter Kit tạo ra các giá trị từ 0 đến 4.095. Đối với cảm biến chạy ở điện áp 3,3V này, đầu ra 1V sẽ có giá trị 1.241.

Thông thường, tất cả điều này thường được xử lý bởi firmware bên dưới của bộ vi điều khiển cũng như các thư viện đi kèm với cảm biến, vì vậy bạn không cần phải lo lắng về việc chuyển đổi này.

### **Cảm biến digital**

Cảm biến digital, giống như cảm biến analog, phát hiện thế giới xung quanh bằng cách sử dụng những thay đổi về điện áp. Sự khác biệt là chúng phát ra tín hiệu số, chỉ bằng cách đo hai trạng thái hoặc bằng cách sử dụng ADC tích hợp.

Cảm biến digital đơn giản nhất là một nút nhấn (push button) hoặc công tắc (switch). Đây là cảm biến có hai trạng thái bật hoặc tắt.

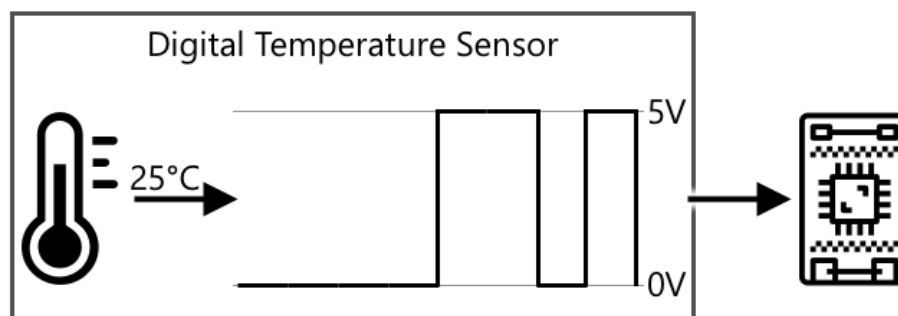


Các chân trên thiết bị IoT như chân GPIO có thể đo tín hiệu này trực tiếp dưới dạng 0 hoặc 1. Nếu điện áp gửi giống với điện áp trả về thì giá trị đọc là 1, nếu không thì giá trị đọc là 0. Không cần chuyển đổi tín hiệu, chỉ có thể là 1 hoặc 0.

👤 Điện áp không bao giờ chính xác, đặc biệt là các thành phần trong cảm biến sẽ có một số điện trở nên thường có dung sai. Ví dụ: các chân GPIO trên Raspberry Pi hoạt động ở điện áp 3,3V và khi đọc tín hiệu trả về lớn hơn 1,8V sẽ là 1, dưới 1,8V sẽ là 0.

- 3,3V đi vào nút. Nút tắt nên 0V xuất hiện, cho giá trị 0
- 3,3V đi vào nút. Nút bật nên 3,3V xuất hiện, cho giá trị 1

Các cảm biến digital loại cao cấp hơn thì sẽ đọc các giá trị analog, sau đó tự chuyển đổi chúng bằng cách sử dụng ADC tích hợp của mình thành tín hiệu digital. Ví dụ, cảm biến nhiệt độ digital vẫn sẽ sử dụng cặp nhiệt điện giống như cảm biến analog và vẫn đo sự thay đổi điện áp do điện trở của cặp nhiệt điện ở nhiệt độ hiện tại. Thay vì trả về một giá trị analog, ADC được tích hợp trong cảm biến sẽ chuyển đổi thành giá trị digital và gửi nó đến thiết bị IoT.



Việc gửi dữ liệu digital này cho phép các cảm biến trở nên phức tạp hơn và gửi được nhiều dữ liệu chi tiết hơn.

## Thực hành: Lập trình đọc cảm biến

---

Nếu sử dụng khối lệnh và OhStem App, bạn tham khảo hướng dẫn ở link sau:

- [https://docs.ohstem.vn/en/latest/yolo\\_uno/lam\\_quen/dht20.html](https://docs.ohstem.vn/en/latest/yolo_uno/lam_quen/dht20.html)

Nếu sử dụng Arduino, bạn hãy tham khảo hướng dẫn trong **Phụ lục 1**, phần **Làm việc với cảm biến ánh sáng** để viết chương trình đọc cảm biến ánh sáng với Yolo UNO.

## Thiết bị truyền động là gì?

---

Thiết bị truyền động trái ngược với cảm biến - chúng chuyển đổi tín hiệu điện từ thiết bị IoT của bạn thành tín hiệu tương tác với thế giới vật lý như phát ra ánh sáng hoặc âm thanh hoặc di chuyển động cơ.

Một số thiết bị truyền động phổ biến bao gồm:

- Đèn LED - phát ra ánh sáng khi bật
- Loa - phát ra âm thanh dựa trên tín hiệu được gửi tới chúng, từ còi cơ bản đến loa âm thanh có thể phát nhạc
- Động cơ bước - chuyển đổi tín hiệu thành một lượng vòng quay xác định, chẳng hạn như quay 1 góc 90°
- Role (relay) - đây là những công tắc có thể bật hoặc tắt bằng tín hiệu điện. Chúng cho phép một điện áp nhỏ từ thiết bị IoT bật lên điện áp lớn hơn.
- Màn hình - đây là 1 loại bộ truyền động phức tạp hơn và hiển thị thông tin trên màn hình nhiều đoạn. Màn hình có nhiều loại khác nhau từ màn hình LED đơn giản đến màn hình video có độ phân giải cao.

## Các loại thiết bị truyền động

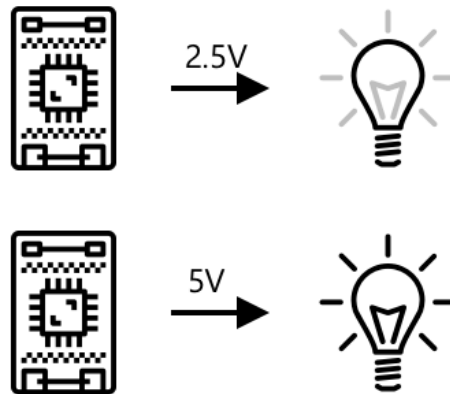
---

Giống như cảm biến, bộ truyền động có loại analog hoặc digital.

### Thiết bị truyền động Analog

Bộ truyền động tương tự lấy tín hiệu analog và chuyển đổi nó thành một loại tương tác nào đó, trong đó sự tương tác thay đổi dựa trên điện áp được cung cấp.

Một ví dụ là đèn có thể điều chỉnh độ sáng, chẳng hạn như đèn bạn có thể có trong nhà. Lượng điện áp cung cấp cho đèn quyết định độ sáng của đèn.



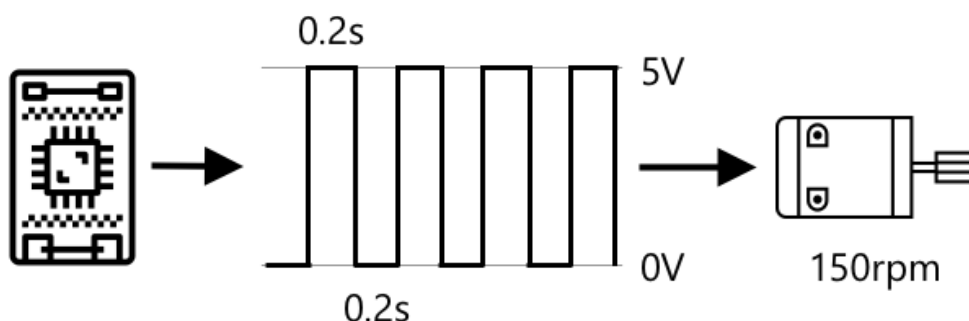
Giống như cảm biến, thiết bị IoT thực tế hoạt động trên tín hiệu số chứ không phải tín hiệu analog. Điều này có nghĩa là để gửi tín hiệu analog, thiết bị IoT cần có bộ chuyển đổi digital sang analog (DAC), trực tiếp trên thiết bị IoT hoặc trên bảng mở rộng. Điều này sẽ chuyển đổi số 0 và số 1 từ thiết bị IoT thành điện áp tương tự mà bộ truyền động có thể sử dụng.

## PWM - Điều chế độ rộng xung

Một tùy chọn khác để chuyển đổi tín hiệu số từ thiết bị IoT sang tín hiệu tương tự là PWM - điều chế độ rộng xung. Đây là kỹ thuật gửi nhiều xung digital ngắn để giả lập tín hiệu analog.

Ví dụ: bạn có thể sử dụng xung điện để điều khiển tốc độ của động cơ.

Hãy tưởng tượng bạn đang điều khiển một động cơ có nguồn điện 5V. Bạn gửi một xung ngắn đến động cơ của mình, bật điện áp lên mức cao (5V) trong khoảng 20% giây (0,02 giây). Trong thời gian đó, động cơ của bạn có thể quay 1/10 vòng quay, hay 36°. Sau đó, tín hiệu sẽ tạm dừng trong 20% giây (0,02 giây), gửi tín hiệu ở mức thấp (0V). Mỗi chu kỳ bật rồi tắt như vậy kéo dài 0,04 giây và lặp lại liên tục.

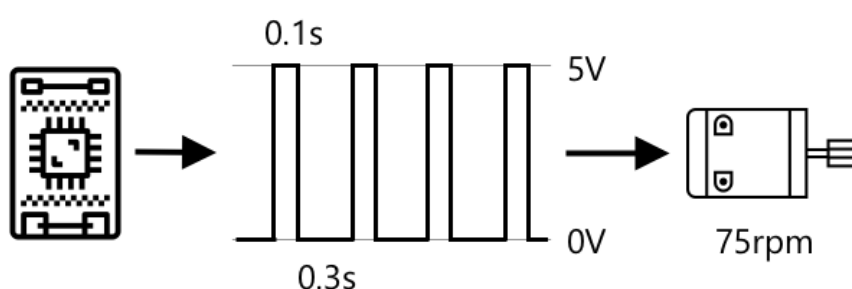


Điều này có nghĩa là trong một giây, bạn có 25 xung 5V 0,02 giây làm quay động cơ, mỗi xung tiếp theo là 0,02 giây tạm dừng 0V không làm quay động cơ. Mỗi xung làm

quay động cơ 1/10 vòng quay, nghĩa là động cơ hoàn thành 2,5 vòng quay mỗi giây. Bạn đã sử dụng tín hiệu số để quay động cơ với tốc độ 2,5 vòng/giây hoặc 150 vòng/phút (một thước đo không chuẩn về tốc độ quay).

$$\begin{aligned} 25 \text{ pulses per second} \times 0.1 \text{ rotations per pulse} &= 2.5 \text{ rotations per second} \\ 2.5 \text{ rotations per second} \times 60 \text{ seconds in a minute} &= 150 \text{rpm} \end{aligned}$$

Khi tín hiệu xung điện bật trong một nửa thời gian và tắt trong một nửa thời gian, tín hiệu này được gọi là duty cycle (chu kỳ nhiệm vụ) 50% . Chu kỳ hoạt động được đo bằng phần trăm thời gian tín hiệu ở trạng thái bật so với trạng thái tắt.



Bạn có thể thay đổi tốc độ động cơ bằng cách thay đổi kích thước của xung. Ví dụ: với cùng một động cơ, bạn có thể giữ nguyên thời gian chu kỳ là 0,04 giây, với xung bật giảm một nửa còn 0,01 giây và xung tắt tăng lên 0,03 giây. Bạn có cùng số xung mỗi giây (25), nhưng mỗi xung trên có độ dài bằng một nửa. Một nửa chiều dài xung chỉ quay động cơ một phần hai mươi vòng quay và ở 25 xung một giây sẽ hoàn thành 1,25 vòng quay mỗi giây hoặc 75 vòng/phút. Bằng cách thay đổi tốc độ xung của tín hiệu số, bạn đã giảm một nửa tốc độ của động cơ analog.

$$\begin{aligned} 25 \text{ pulses per second} \times 0.05 \text{ rotations per pulse} &= 1.25 \text{ rotations per second} \\ 1.25 \text{ rotations per second} \times 60 \text{ seconds in a minute} &= 75 \text{rpm} \end{aligned}$$

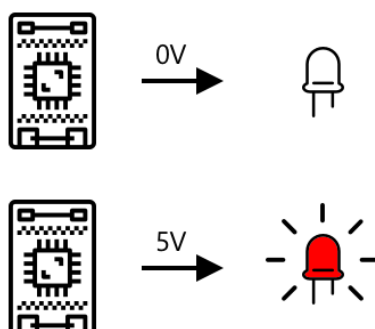
Một số cảm biến còn sử dụng xung điện để chuyển đổi tín hiệu analog sang tín hiệu số.

Bạn có thể đọc thêm về PWM trên [Wikipedia](https://en.wikipedia.org/wiki/Pulse-width_modulation) .

## Thiết bị truyền động digital

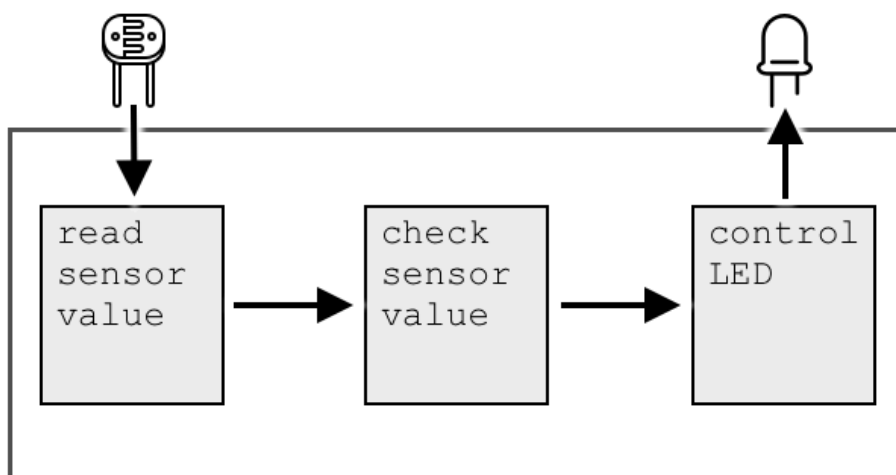
Các bộ truyền động digital, giống như cảm biến digital, có hai trạng thái được điều khiển bởi điện áp cao hoặc thấp hoặc có tích hợp DAC để có thể chuyển đổi tín hiệu kỹ thuật số sang tín hiệu tương tự.

Một thiết bị truyền động kỹ thuật số đơn giản là đèn LED. Khi một thiết bị gửi tín hiệu số 1, một điện áp cao sẽ được gửi đến để làm sáng đèn LED. Khi tín hiệu số 0 được gửi đi, điện áp giảm xuống 0V và đèn LED tắt.



## Thực hành: Lập trình bộ truyền động

Trong hướng dẫn này, bạn sẽ bổ sung thêm đèn LED là 1 loại bộ truyền động vào thiết bị IoT của bạn là đèn LED thông minh, giúp đèn LED này được điều khiển tự động bởi cảm biến. Nó sẽ đọc mức ánh sáng từ cảm biến ánh sáng và sử dụng bộ truyền động dưới dạng đèn LED để phát ra ánh sáng khi mức ánh sáng phát hiện quá thấp.



Nếu sử dụng khối lệnh và OhStem App, bạn tham khảo:

- [https://docs.ohstem.vn/en/latest/yolo\\_uno/lam\\_quen/den\\_thong\\_minh.html](https://docs.ohstem.vn/en/latest/yolo_uno/lam_quen/den_thong_minh.html)

Nếu sử dụng Arduino, bạn hãy tham khảo hướng dẫn trong **Phụ lục 1**, phần **Lập trình đèn LED tự động** để viết chương trình điều khiển đèn LED bằng cảm biến ánh sáng với Yolo UNO.

## Sự kết hợp giữa cảm biến và bộ truyền động

---

Cảm biến và bộ truyền động thường được sử dụng kết hợp với nhau trong các thiết bị IoT. Cảm biến thu thập dữ liệu từ môi trường, sau đó gửi đến bộ xử lý trung tâm để xử lý và phân tích. Dựa trên dữ liệu thu thập được, bộ xử lý trung tâm sẽ gửi tín hiệu đến bộ truyền động (trực tiếp hoặc thông qua các giao thức khác nhau trong hệ thống IoT) để thực hiện hành động phù hợp.

Ví dụ về ứng dụng kết hợp cảm biến và bộ truyền động:

- Hệ thống tưới tiêu thông minh: Sensor độ ẩm đất có thể đo lường độ ẩm của đất và gửi tín hiệu đến bộ xử lý trung tâm. Bộ xử lý trung tâm sau đó sẽ gửi tín hiệu đến Actuator (van nước) để tưới nước cho cây khi cần thiết.
- Hệ thống đèn chiếu sáng thông minh: Sensor ánh sáng có thể đo lường cường độ ánh sáng và gửi tín hiệu đến bộ xử lý trung tâm. Bộ xử lý trung tâm sau đó sẽ gửi tín hiệu đến Actuator (đèn) để bật hoặc tắt đèn tùy theo điều kiện ánh sáng.
- Hệ thống an ninh thông minh: Sensor chuyển động có thể phát hiện chuyển động và gửi tín hiệu đến bộ xử lý trung tâm. Bộ xử lý trung tâm sau đó sẽ gửi tín hiệu đến Actuator (báo động) để cảnh báo người dùng về sự xâm nhập trái phép.

Sự kết hợp giữa cảm biến và bộ truyền động giúp cho các thiết bị IoT trở nên thông minh và có khả năng tự động hóa nhiều tác vụ trong cuộc sống hàng ngày. Nhờ đó, IoT đang ngày càng được ứng dụng rộng rãi trong nhiều lĩnh vực khác nhau như nhà thông minh, công nghiệp, y tế, nông nghiệp, v.v.

# Chương 4: Kết nối Internet

## Giới thiệu

---

Chữ I trong IoT là Internet - liên quan đến kết nối ra Internet và các dịch vụ đám mây (cloud), hỗ trợ rất nhiều tính năng của thiết bị IoT, từ việc thu thập số liệu đo từ các cảm biến được kết nối với thiết bị, đến gửi tin nhắn để điều khiển bộ truyền động.

Các thiết bị IoT thường kết nối với một dịch vụ IoT trên cloud bằng các giao thức chuẩn và dịch vụ đó được kết nối với phần còn lại của ứng dụng IoT của bạn, từ các dịch vụ xử lý AI để đưa ra quyết định thông minh về dữ liệu của bạn, đến các ứng dụng web để theo dõi hoặc báo cáo.

Dữ liệu được thu thập từ các cảm biến và gửi lên đám mây được gọi là Telemetry.

Thiết bị IoT có thể nhận các thông báo từ đám mây. Thông thường, các thông báo chứa các lệnh yêu cầu thực hiện một hành động (chẳng hạn như reset lại hoặc cập nhật firmware của thiết bị) hoặc điều khiển bộ truyền động (chẳng hạn như bật tắt đèn).

Chương này giới thiệu một số giao thức liên lạc (communication protocol) mà các thiết bị IoT có thể sử dụng để kết nối với đám mây và các loại dữ liệu mà chúng có thể gửi hoặc nhận. Bạn cũng sẽ lập trình cả 2 tính năng này trong phần thực hành dự án theo dõi nhiệt độ phòng và bật tắt quạt qua Internet.

Trong bài học này chúng ta sẽ đề cập đến:

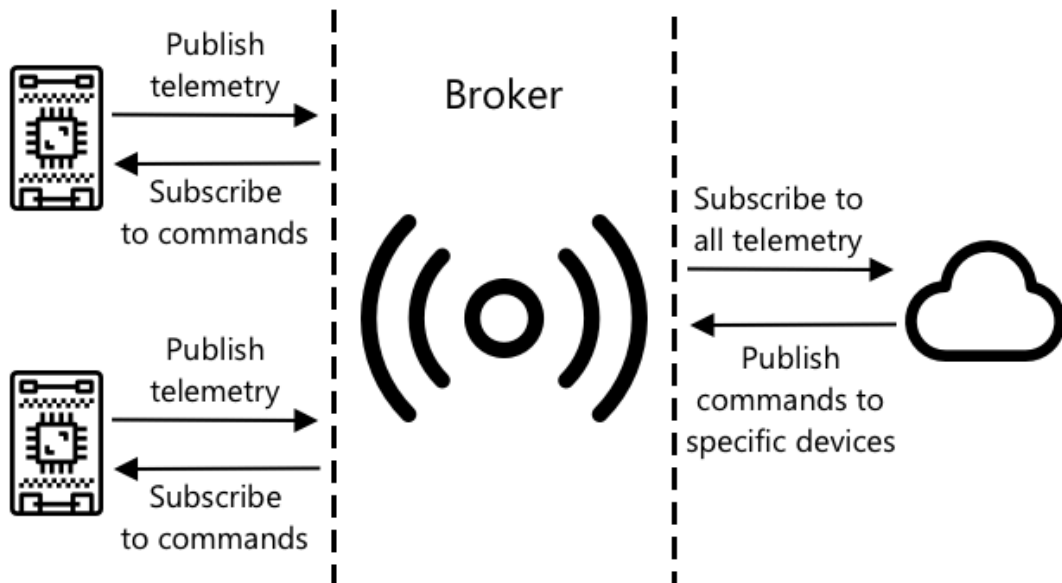
- Giao thức truyền thông (Communication Protocols)
- Giao thức MQTT
- Telemetry
- Command

## Giao thức truyền thông (Communication Protocols)

---

Có một số giao thức truyền thông phổ biến được các thiết bị IoT sử dụng để liên lạc qua Internet. Phổ biến nhất là dựa trên mô hình xuất bản/đăng ký (publish/subscribe) thông qua máy chủ trung gian gọi là broker. Các thiết bị IoT kết nối với broker và publish dữ liệu đo được cũng như đăng ký nhận các lệnh. Các dịch

vụ đám mây cũng kết nối với broker và đăng ký nhận các dữ liệu đó cũng như xuất bản các lệnh điều khiển tới các thiết bị IoT.



MQTT là một giao thức truyền thông phổ biến nhất dành cho các thiết bị IoT và được đề cập trong bài học này. Ngoài ra còn có nhiều các giao thức khác như AMQP và HTTP/HTTPS...

## Giao thức MQTT

[MQTT](#) (Message Queue Telemetry Transport) là một giao thức truyền tin tiêu chuẩn mở, nhẹ, có thể gửi tin nhắn giữa các thiết bị. Nó được thiết kế vào năm 1999 để giám sát các đường ống dẫn dầu, trước khi được IBM phát hành dưới dạng tiêu chuẩn mở 15 năm sau.

MQTT là một giao thức truyền tin (message) theo mô hình publish – subscribe (xuất bản – đăng ký nhận tin), sử dụng băng thông thấp, độ tin cậy cao và có khả năng hoạt động trong điều kiện đường truyền không ổn định. Do đó MQTT cực kỳ thích hợp cho các hệ thống IoT.

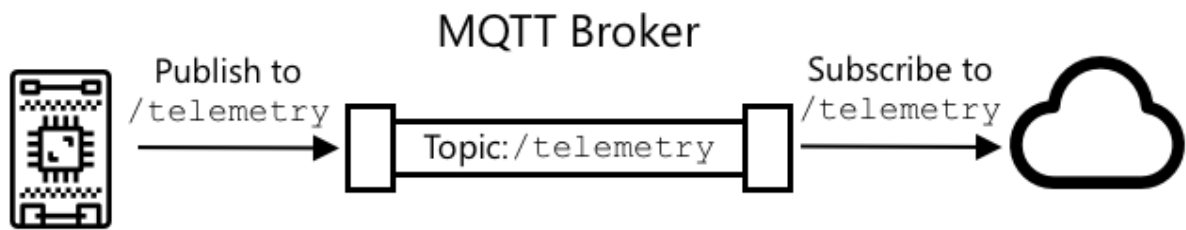
Sau đây ta cũng làm quen với các khái niệm cơ bản trong MQTT.

### Publish - Subscribe

Mô hình MQTT gồm 2 phần chính là Broker và Client.

Mỗi Client sẽ đăng ký một vài kênh (topic) dữ liệu, ví dụ như “house/living-room/temperature”, “house/kitchen/temperature”. Quá trình đăng ký này gọi là subscribe, giống như chúng ta đăng ký nhận tin trên một kênh Youtube

vậy. Mỗi Client sẽ nhận được dữ liệu khi bất kỳ Client nào khác gửi dữ liệu đến kênh đã đăng ký. Khi một Client gửi dữ liệu tới kênh đó, gọi là publish.



Broker được coi như trung tâm, nó là điểm giao của tất cả các kết nối đến từ Client. Nhiệm vụ chính của broker là nhận message từ các Client, xếp các message theo hàng đợi rồi chuyển chúng tới một địa chỉ (topic) cụ thể. Nhiệm vụ phụ của broker là nó có thể đảm nhận thêm một vài tính năng liên quan tới quá trình truyền thông như: bảo mật message, lưu trữ message, logs...

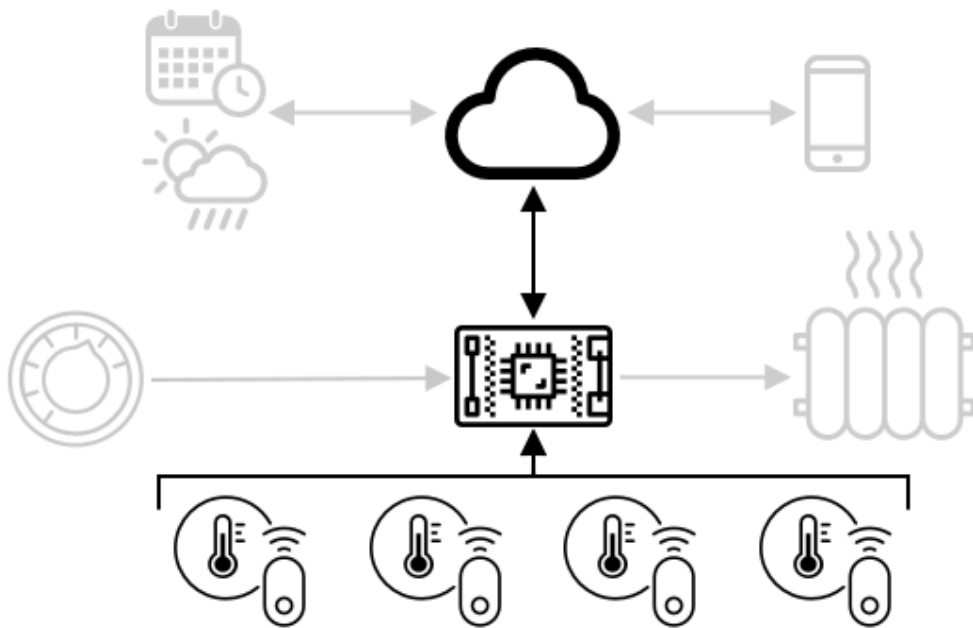
Các topic để gửi và nhận thông tin có thể được phân cấp và các client có thể đăng ký các cấp khác nhau của hệ thống phân cấp này bằng cách sử dụng ký tự đại diện (wildcards) trong tên topic. Ví dụ: bạn có thể gửi tin nhắn chứa thông tin về nhiệt độ đến topic **/telemetry/temperature** và tin nhắn độ ẩm đến topic **/telemetry/humidity**. Sau đó trong ứng dụng đám mây của bạn, hãy đăng ký topic **/telemetry/\*** để có thể nhận cả 2 tin nhắn nhiệt độ và độ ẩm.

## Telemetry (dữ liệu đo từ xa)

Cái tên Telemetry có nguồn gốc từ tiếng Hy Lạp có nghĩa là đo và gửi kết quả được từ xa. Trong hệ thống IoT, thuật ngữ telemetry dùng để gọi các dữ liệu thu thập được từ các thiết bị IoT và gửi về bộ xử lý trung tâm (chạy trên server hoặc dịch vụ cloud xử lý).

👤 Một trong những thiết bị đo từ xa sớm nhất được phát minh ở Pháp vào năm 1874 và gửi thông tin thời tiết và độ sâu tuyết theo thời gian thực từ Mont Blanc đến Paris. Nó sử dụng dây vật lý vì công nghệ không dây chưa có sẵn vào thời điểm đó.

Chúng ta cùng xem lại ví dụ về máy điều nhiệt thông minh ở chương 1.



Bộ điều chỉnh nhiệt có cảm biến nhiệt độ để thu thập dữ liệu từ xa. Rất có thể nó sẽ có một cảm biến nhiệt độ được tích hợp và nó có thể kết nối với nhiều cảm biến nhiệt độ bên ngoài qua giao thức không dây như [Bluetooth Low Energy](#) (BLE).

Một ví dụ về dữ liệu đo từ xa mà nó sẽ gửi có thể là:

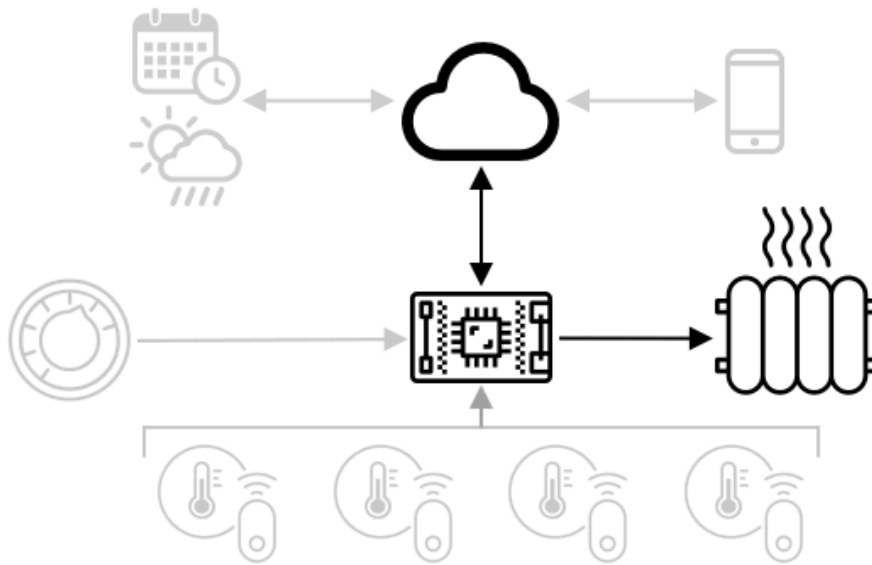
| Tên                    | Giá trị | Mô tả                                                                                          |
|------------------------|---------|------------------------------------------------------------------------------------------------|
| thermostat_temperature | 18°C    | Nhiệt độ được đo bằng cảm biến nhiệt độ tích hợp của bộ điều chỉnh nhiệt                       |
| livingroom_temperature | 19°C    | Nhiệt độ được đo bằng cảm biến nhiệt độ từ xa được đặt tên livingroom để xác định phòng đang ở |
| bedroom_temperature    | 21°C    | Nhiệt độ được đo bằng cảm biến nhiệt độ từ xa được đặt tên bedroom để xác định phòng đang ở    |

Sau đó, dịch vụ đám mây có thể sử dụng dữ liệu đo từ xa này để đưa ra quyết định về những lệnh cần gửi để kiểm soát hệ thống sưởi.

## Command (lệnh)

Command (lệnh) là các tin nhắn được bộ xử lý trung tâm (server, cloud) gửi tới thiết bị IoT, hướng dẫn thiết bị thực hiện điều gì đó. Các lệnh này phần lớn liên quan đến việc điều khiển một số loại đầu ra thông qua bộ truyền động, nhưng cũng có thể là

một hướng dẫn cho chính thiết bị đó, chẳng hạn như khởi động lại hoặc cập nhật firmware...



Bộ điều nhiệt có thể nhận lệnh từ đám mây để bật hệ thống sưởi. Dựa trên dữ liệu đo từ xa từ tất cả các cảm biến, nếu dịch vụ đám mây đã quyết định bật hệ thống sưởi thì nó sẽ gửi lệnh liên quan.

## QoS

Có 3 tùy chọn QoS (Quality of service) khi publish và subscribe:

- QoS 0: Đây là mức dịch vụ thấp nhất trong MQTT. Người gửi chỉ gửi tin nhắn một lần duy nhất và không được lưu trữ và truyền lại bởi người gửi. Người nhận không xác nhận đã nhận được tin nhắn.
- QoS 1: QoS 1 đảm bảo rằng một tin nhắn được gửi ít nhất một lần cho người nhận. Người gửi lưu trữ tin nhắn và có thể gửi nhiều lần cho đến khi nó nhận được phản hồi từ người nhận xác nhận rằng đã nhận được tin nhắn.
- QoS 2: Đây là mức dịch vụ cao nhất trong MQTT. Mức này đảm bảo rằng mỗi tin nhắn chỉ được nhận đúng một lần bởi người nhận. QoS 2 là mức dịch vụ an toàn và chậm nhất, quá trình này phải trải qua 4 bước bắt tay.

## Retain

Nếu RETAIN được set bằng 1, khi tin nhắn được publish từ Client, Broker PHẢI lưu trữ lại gói tin với QoS, và nó sẽ được gửi đến bất kỳ Client nào subscribe cùng kênh trong tương lai. Khi một Client kết nối tới Broker và subscribe, nó sẽ nhận được gói tin cuối cùng có RETAIN = 1 với bất kỳ topic nào mà nó đăng ký trùng.

## LWT

Do MQTT thường được sử dụng trong điều kiện đường truyền không ổn định, nên cần phải đưa ra những kịch bản để xử lý trong các tình huống bị mất kết nối không đúng lúc. LWT (Last Will and Testament) được sử dụng trong trường hợp này.

Quá trình diễn ra như sau:

1. Publisher đăng ký với broker một topic để gửi gói tin LWT cho các subscriber trong trường hợp bị ngắt kết nối.
2. Nếu broker phát hiện một kết nối bị ngắt, nó sẽ gửi gói tin LWT sẽ cho tất cả các subscriber đã đăng ký topic ở trên.

[Mosquitto từ Eclipse Foundation](#) có một MQTT broker miễn phí mà bạn có thể tự cài đặt và chạy trên máy tính của mình để thử nghiệm, cùng với một broker MQTT dưới dạng public mà bạn có thể sử dụng miễn phí để thử nghiệm chương trình của mình tại [test.mosquitto.org](https://test.mosquitto.org).

MQTT giao tiếp qua TCP/IP, gần giống giao thức cơ bản HTTP, nhưng trên một port khác. Bạn cũng có thể sử dụng MQTT qua giao thức websocket để chạy với các ứng dụng web trên trình duyệt.

## Tần suất gửi dữ liệu

Một điều quan trọng cần cân nhắc là tần suất đo và gửi dữ liệu. Câu trả lời phụ thuộc vào nhiều thứ. Nếu đo lường thường xuyên, bạn có thể phản hồi nhanh hơn trước những thay đổi về số đo nhưng lại sử dụng nhiều năng lượng hơn, nhiều băng thông hơn, tạo ra nhiều dữ liệu hơn và cần nhiều tài nguyên đám mây hơn để xử lý. Bạn cần đo đủ thường xuyên, nhưng không nên quá thường xuyên.

Đối với bộ điều nhiệt, việc đo vài phút một lần có lẽ là quá đủ vì nhiệt độ không thay đổi thường xuyên. Nếu bạn chỉ đo mỗi ngày một lần thì có khả năng bạn sẽ sưởi ấm ngôi nhà của mình theo nhiệt độ ban đêm vào giữa một ngày nắng, trong khi nếu bạn đo mỗi giây, bạn sẽ có hàng nghìn phép đo nhiệt độ trùng lặp không cần thiết sẽ ảnh hưởng đến tốc độ Internet của người dùng và băng thông (vấn đề đối với những người có gói băng thông hạn chế), sử dụng nhiều năng lượng hơn, điều này sẽ gây ra vấn đề đối với các thiết bị chạy bằng pin như cảm biến từ xa và làm tăng chi phí xử lý và lưu trữ tài nguyên điện toán đám mây của nhà cung cấp.

Nếu bạn đang theo dõi dữ liệu xung quanh một bộ phận máy móc trong nhà máy và nếu nó bị hỏng có thể gây ra thiệt hại nghiêm trọng và mất rất nhiều tiền thì việc đo lường nhiều lần trong một giây có thể là cần thiết. Thà lãng phí băng thông còn hơn bỏ lỡ những số liệu cho biết rằng máy cần phải dừng và sửa trước khi nó bị hỏng.

Trong tình huống này, chúng ta có thể cân nhắc việc sử dụng một thiết bị điện toán biên (edge devices) để xử lý dữ liệu đo được ở cục bộ trước khi gửi về trung tâm nhằm giảm sự phụ thuộc vào Internet và tăng hiệu suất vận hành.

## Mất kết nối

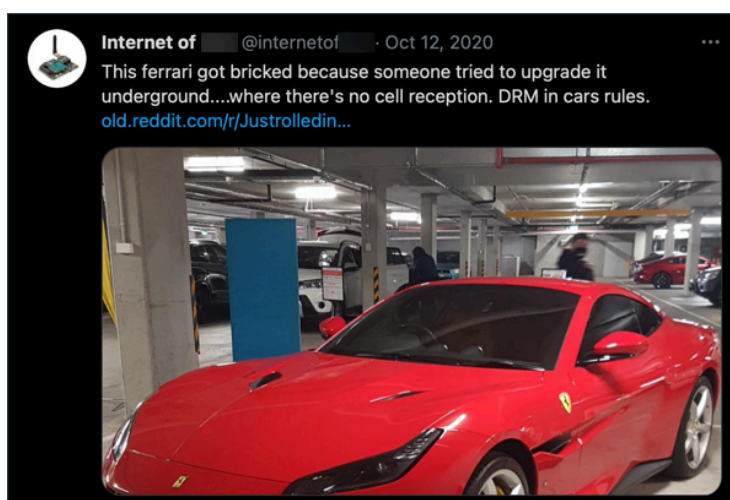
Kết nối Internet có thể không đáng tin cậy và thường xuyên bị ngừng hoạt động. Thiết bị IoT nên làm gì trong những trường hợp này - liệu nó có bị mất dữ liệu hay nên lưu trữ dữ liệu cho đến khi kết nối được khôi phục? Câu trả lời vẫn là còn tùy thuộc vào nhiều yếu tố.

Đối với bộ điều khiển nhiệt, dữ liệu có thể bị mất ngay sau khi thực hiện phép đo nhiệt độ mới. Hệ thống sưởi không quan tâm rằng 20 phút trước nhiệt độ là 20,5°C nếu nhiệt độ bây giờ là 19°C, mà nhiệt độ hiện tại mới quyết định nên bật hay tắt hệ thống sưởi.

Đối với máy móc, bạn có thể muốn giữ lại các dữ liệu, đặc biệt nếu dữ liệu đó được sử dụng để phân tích xu hướng. Có các mô hình học máy có thể giúp phát hiện sự bất thường từ các dữ liệu này bằng cách phân tích trong khoảng thời gian xác định (chẳng hạn như 1 giờ trước) và phát hiện các bất thường.

Điều này thường được sử dụng trong các giải pháp bảo trì được dự đoán trước (predictive maintenance), tìm kiếm các dấu hiệu cho thấy phần nào đó có thể sớm sẽ bị hỏng để bạn có thể sửa chữa hoặc thay thế nó trước khi điều đó xảy ra. Vì vậy, khi thiết bị IoT có thể kết nối lại, nó sẽ gửi tất cả dữ liệu đo được trong thời gian Internet ngừng hoạt động.

Khi thiết kế thiết bị IoT, chúng ta cũng nên xem xét liệu thiết bị có thể được sử dụng trong thời gian Internet ngừng hoạt động hoặc mất tín hiệu do vị trí hay không. Bộ điều chỉnh nhiệt thông minh sẽ có thể đưa ra một số quyết định có giới hạn để kiểm soát hệ thống sưởi nếu nó không thể gửi dữ liệu lên đám mây do mất điện.



## Thực hành thử nghiệm giao thức MQTT

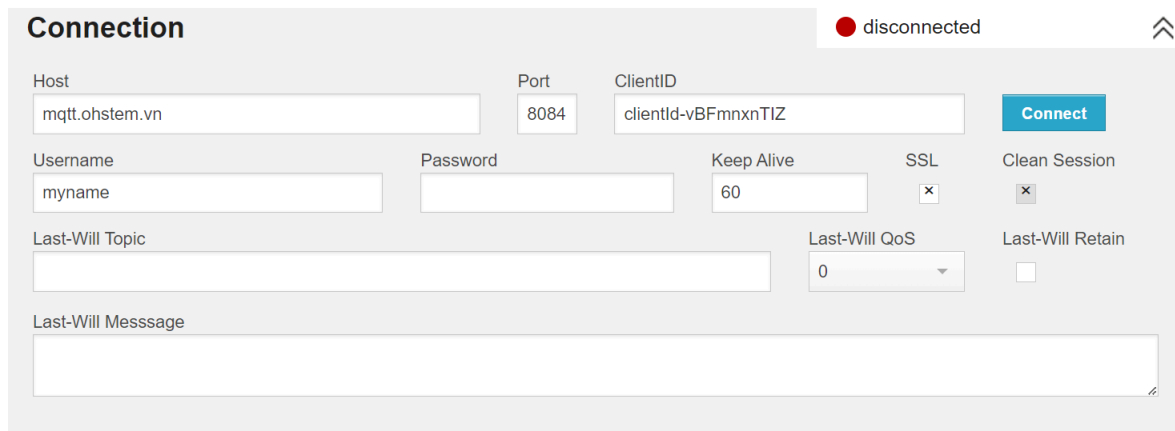
Với sự phổ biến của MQTT trong các ứng dụng kết nối vạn vật, các phần mềm hỗ trợ cho việc kết nối MQTT để thử nghiệm và kiểm tra trước khi hiện thực dự án dựa trên kết nối vạn.

Các phần mềm MQTT khá phong phú và đa dạng, bao gồm phiên bản cài đặt và chạy trực tiếp trên máy tính (ví dụ như MQTT Explorer - <https://mqtt-explorer.com/>) hay các phiên bản online từ EMQX (<http://www.emqx.io/online-mqtt-client>) hoặc HIVEMQ (<https://www.hivemq.com/demos/websocket-client/>).

Trong hướng dẫn này, chúng tôi chọn phần mềm trực tuyến từ HIVEMQ để minh họa cho các tính năng kết nối, gửi dữ liệu (publish) hoặc nhận dữ liệu (từ cơ chế subscribe của MQTT). Server MQTT dùng trong hướng dẫn này được phát triển bởi OhStem Education.

### Kết nối với server MQTT

Khi vào đường dẫn chính của công cụ online HIVEMQ (<https://www.hivemq.com/demos/websocket-client/>), giao diện sau đây sẽ hiện ra



Các thông tin quan trọng liên quan đến việc kết nối tới server MQTT bao gồm:

**Host:** Đây là địa chỉ của MQTT Broker. Khi sử dụng server từ OhStem, địa chỉ này là mqtt.ohstem.vn

**Port:** Cổng kết nối đến MQTT Broker, trong trường hợp này là **8084**

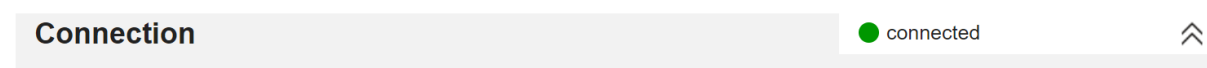
**ClientID:** Thông tin này được sinh tự động bởi công cụ HIVEMQ. Về ý nghĩa, đây là mã định danh khi một thiết bị kết nối đến MQTT Broker. Mã định danh này cần phải duy nhất trong phiên làm việc hiện tại của server. Nó khá là gần với ý nghĩa của SessionID. Bạn có thể thay đổi thông tin này, nhưng nó nên là 1 chuỗi dài (trên 10 kí

tự), bao gồm cả chữ và số, để giảm thiểu tối đa xác suất trùng nhau của ClientID. Tuy nhiên, bạn hoàn toàn có thể sử dụng thông tin mặc định của nó

**Username:** Thông tin này là username để đăng nhập vào server MQTT. Nó thuộc lớp bảo mật cấp cao hơn so với ClientID. Thông tin này khá quan trọng và bạn phải cung cấp vào công cụ online này. Bạn hãy chọn một username cho riêng mình, có kí tự chuỗi và 1 dãy các mã số cá nhân, cũng nhằm mục đích hạn chế tối đa xác suất trùng username trên hệ thống. Trong ví dụ này, chúng tôi chỉ chọn một username đơn giản, là **myname**

**Password:** Đây thường là thông tin thứ 2, đi kèm với username ở trên, để kết nối vào server MQTT. Với OhStem server, nhằm tạo điều kiện thuận lợi cho bạn đọc thử nghiệm, chúng tôi không kiểm tra thông tin này khi kết nối. Bạn đọc có thể điền bất kì thông tin gì, hoặc thậm chí bỏ trống ở trường này. Tuy nhiên đối với các server khác (chẳng hạn như Adafruit IO hoặc Core IoT) trường key hoặc token sẽ được điền vào khung password này.

Khi các thông tin đã được điền đầy đủ, bạn có thể nhấn nút Connect để kết nối. Khi kết nối được thành công, sẽ có biểu tượng màu xanh ở góc bên phải màn hình, như sau:



*Biểu tượng kết nối thành công tới OhStem Server*

## Publish và Subscribe

Để có thể kiểm tra việc gửi nhận thông tin trên server, chúng ta cần phải cấu hình cho cả 2 tác vụ này:

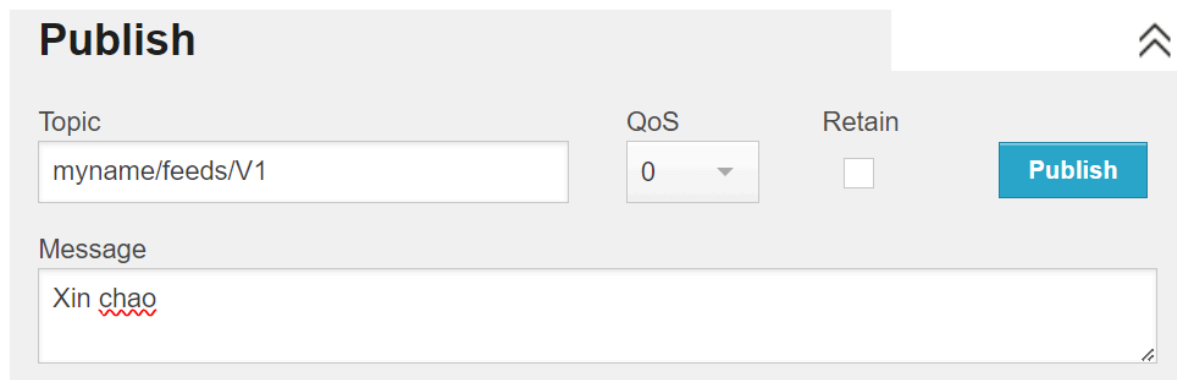
- Publish: Gửi thông tin đến một kênh dữ liệu (hay còn gọi là topic)
- Subscribe: Nhận thông tin từ một kênh dữ liệu đã đăng kí

Tùy vào ứng dụng sẽ được hiện thực về sau, mà thông tin của kênh dữ liệu sẽ khác nhau. Ví dụ như thông tin về nhiệt độ sẽ được gửi lên kênh V1, còn thông tin để điều khiển một thiết bị (bật/tắt bóng đèn) sẽ được lưu ở kênh V2. Tuy nhiên, trong thử nghiệm đầu tiên này, chúng ta sẽ dùng cùng 1 kênh dữ liệu để kiểm tra việc gửi nhận thông tin trên MQTT Broker. Các bước để thiết lập môi trường cho việc Publish và Subscribe như sau:

## Publish

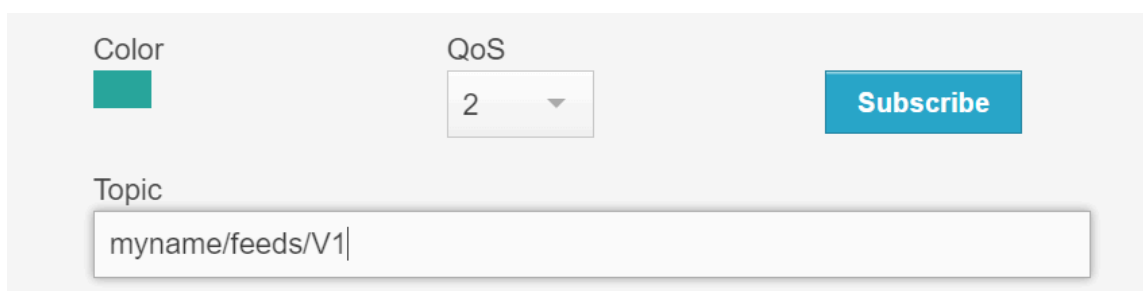
Ở phần Publish, chúng ta cần điền thông tin của Topic, theo định dạng thông dụng của server MQTT, là **username/feeds/tên topic**, trong trường hợp này là **myname/feeds/V1**. Giá trị QoS mặc định khi gửi là 0, với tốc độ gửi nhanh nhất,

nhưng sẽ không đảm bảo bên nhận sẽ nhận thành công dữ liệu. Tuy nhiên, với gói tin nhỏ, QoS = 0 là giá trị hợp lý cho một ứng dụng IOT. Thông tin muốn gửi lên để kiểm tra, chúng ta để mặc định là **Xin chào**, như minh họa ở hình bên dưới.

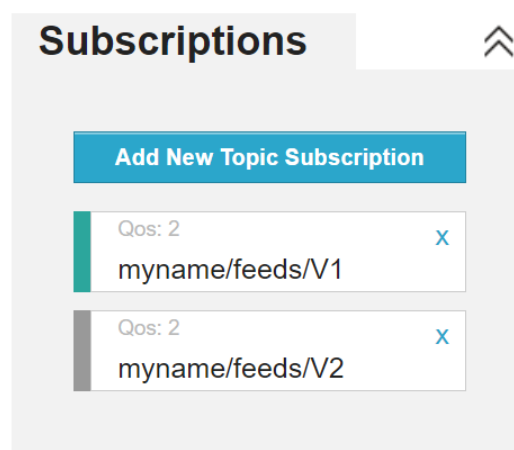


## Subscribe

Ở phần Subscriptions, nhấn vào nút **Add New Topic Subscription**, bảng sau sẽ hiện lên, và bạn điền các thông tin như gợi ý sau và nhấn tiếp vào nút **Subscribe**



QoS mặc định cho phần này là 2, là mức độ cao nhất, đảm bảo thông tin được nhận chính xác nhất nhưng cũng sẽ chậm nhất. Lặp lại quá trình trên, nhưng lần này chúng ta sẽ thêm vào kênh V2. Kết quả cuối cùng của bước đăng kí này, chúng ta sẽ có 2 kênh được đăng kí:



*Kết quả sau khi đăng kí 2 kênh dữ liệu*

Bây giờ, bạn đã có thể nhấn vào nút Publish, để gửi dữ liệu lên kênh V1. Vì chúng ta cũng đăng kí nhận dữ liệu từ V1, nên sẽ nhận lại được dữ liệu mà chúng ta gửi lên (thường được gọi là cơ chế loopback), như sau:

The screenshot displays the MQTT client interface with three main panels:

- Publish:** Contains a 'Topic' field with 'myname/feeds/V1', a 'QoS' dropdown set to '0', an unchecked 'Retain' checkbox, and a 'Publish' button. The 'Message' field contains the text 'Xin chào'.
- Subscriptions:** Features an 'Add New Topic Subscription' button and a list of two subscriptions: 'myname/feeds/V1' and 'myname/feeds/V2', both with 'Qos: 2'.
- Messages:** Shows a list of three received messages, each with a timestamp of '2024-04-24 10:22:16' (or '15' for the last one), the topic 'myname/feeds/V1', and the message 'Xin chào'.

Hãy chú ý vào thông tin ở ô Messages, để thấy lại thông tin mà chúng ta nhận về ở kênh V1. Bạn có thể chủ động thay đổi thông tin ở phần Message, thay đổi kênh dữ liệu (từ V1 sang V2) và tiếp tục trải nghiệm cơ chế gửi (publish) và nhận (subscribe) của một server MQTT. Trong trường hợp gửi lên kênh dữ liệu khác (chẳng hạn như V3), bạn sẽ không nhận được thông tin, vì không đăng kí vào kênh này.

Cần lưu ý rằng, bạn có thể chọn bất kì tên nào cho kênh dữ liệu của mình. Tuy nhiên, chúng tôi chọn ở đây là V1, V2 cho đến V20, vốn là các kênh thông dụng, để tích hợp thêm các chức năng với server của OhStem về sau. Đối với một ứng dụng minh họa cho ứng dụng dựa trên kết nối vạn vật, thì 20 kênh dữ liệu thực sự là đủ cho việc kết nối và điều khiển thiết bị.

## Dự án: Giám sát nhiệt độ phòng và bật tắt quạt qua Internet

Trong phần này của bài học, bạn sẽ xây dựng một hệ thống IoT ở mức đơn giản nhưng đầy đủ, kết hợp cảm biến để thu thập dữ liệu và điều khiển từ xa qua kết nối Internet bằng giao thức MQTT.

Trong dự án này, thiết bị (Things) sẽ là mạch Yolo Uno, kết nối với cảm biến nhiệt độ và độ ẩm. Hai thông tin cảm biến sẽ được gửi định kì lên server MQTT OhStem. Đồng thời, hai thiết bị truyền động (bóng đèn và máy quạt) sẽ được điều khiển từ xa, thông qua kết nối MQTT. Một màn hình giám sát (Dashboard) cũng sẽ được hiện thực để theo dõi và điều khiển thiết bị từ xa.

Bản chất mạch lập trình Yolo Uno hay màn hình giám sát (Dashboard) sẽ là client của server MQTT OhStem. Do đó, nếu nó muốn gửi dữ liệu lên, thì sẽ thực hiện cơ chế Publish, ngược lại, nó cần phải đăng kí kênh dữ liệu tương ứng để có thể nhận được thông tin. Không ngoại lệ, nếu công cụ trực tuyến HIVEMQ được sử dụng, nó cũng sẽ là một client với server OhStem. Bạn đọc có thể biến HIVEMQ thành một công cụ để giám sát dữ liệu trên server MQTT, bằng cách đăng kí vào cả 4 kênh dữ liệu, bao gồm V1, V2, V3 và V4. Username sử dụng trong bài hướng dẫn này là **user01234**, như sau

The screenshot displays a web-based MQTT client interface. At the top, the 'Connection' section shows the client is 'connected' (indicated by a green dot). It includes fields for Host (mqtt.ohstem.vn), Port (8084), ClientID (clientId-vBFmnnTIZ), Username (user01234), Password (empty), Keep Alive (60), SSL (disabled), Clean Session (disabled), Last-Will Topic (empty), Last-Will QoS (0), Last-Will Retain (disabled), and Last-Will Message (empty). Below this is the 'Publish' section with a Topic (user01234/feeds/V1), QoS (0), Retain (disabled), and a Message (Xin chào). To the right is the 'Subscriptions' section, which lists four subscriptions: user/feeds/V1, user/feeds/V2, user/feeds/V3, and user/feeds/V4, all with QoS: 2. At the bottom is the 'Messages' section, showing a single message received at 2024-04-24 13:04:25 on the topic user/feeds/V4 with QoS: 0 and Retained, with a payload of #ffa500.

Các bước tiếp theo để hiện thực chương trình trên mạch YoloUno và màn hình điều khiển, bạn có thể tham khảo hướng dẫn ở link sau để hoàn thiện dự án:

[https://docs.ohstem.vn/en/latest/yolo\\_uno/smart\\_home\\_uno/iot.html](https://docs.ohstem.vn/en/latest/yolo_uno/smart_home_uno/iot.html)

## Chương 5: IoT Platform

IoT Platform là một thành phần không thể thiếu trong bất kỳ hệ thống IoT nào, đóng vai trò như là đầu não, con tim của cả hệ thống. Trong chương này, chúng ta cùng tìm hiểu vai trò của IoT platform và thực hành làm quen với một platform tiêu biểu là Core IoT, nền tảng nguồn mở và hoàn toàn miễn phí.

### IoT Platform là gì?

Nói một cách đơn giản, IoT platform là một hệ thống phần mềm toàn diện giúp kết nối, quản lý và phân tích dữ liệu từ các thiết bị IoT. Nó cung cấp một bộ công cụ và dịch vụ cho phép các doanh nghiệp và tổ chức xây dựng, triển khai và vận hành các giải pháp IoT một cách hiệu quả.

Các thành phần chính của một nền tảng IoT thường bao gồm:

- Quản lý kết nối: Cho phép các thiết bị IoT kết nối với nền tảng qua các mạng khác nhau như Wi-Fi, Bluetooth, mạng di động,...
- Quản lý thiết bị: Cung cấp các công cụ để quản lý vòng đời của thiết bị IoT, bao gồm cấu hình, cập nhật phần mềm và giám sát hiệu suất.
- Lưu trữ và phân tích dữ liệu: Thu thập, xử lý và phân tích dữ liệu từ các thiết bị IoT để tạo ra những thông tin có giá trị.
- Giao diện người dùng: Cung cấp giao diện để người dùng tương tác với các thiết bị IoT và xem dữ liệu được thu thập.
- Bảo mật: Bảo vệ các thiết bị và dữ liệu IoT khỏi các mối đe dọa an ninh mạng.

Nền tảng IoT mang lại nhiều lợi ích cho các doanh nghiệp và tổ chức, bao gồm:

- Tăng hiệu quả: Tự động hóa các quy trình thủ công, giảm thời gian chết và nâng cao năng suất.
- Cải thiện khả năng ra quyết định: Cung cấp dữ liệu chi tiết để đưa ra quyết định sáng suốt hơn.
- Tạo ra các mô hình kinh doanh mới: Mở ra cơ hội cho các sản phẩm và dịch vụ mới dựa trên dữ liệu IoT.
- Giảm chi phí: Giảm chi phí vận hành và bảo trì.
- Tăng cường sự hài lòng của khách hàng: Cung cấp trải nghiệm khách hàng được cá nhân hóa và tốt hơn.

Ngoài Core IoT Platform, có rất nhiều các nền tảng IoT phổ biến khác như:

- Amazon Web Services (AWS) IoT Core

- Microsoft Azure IoT Suite
- Google Cloud IoT Core
- Arduino IoT
- Blynk IoT

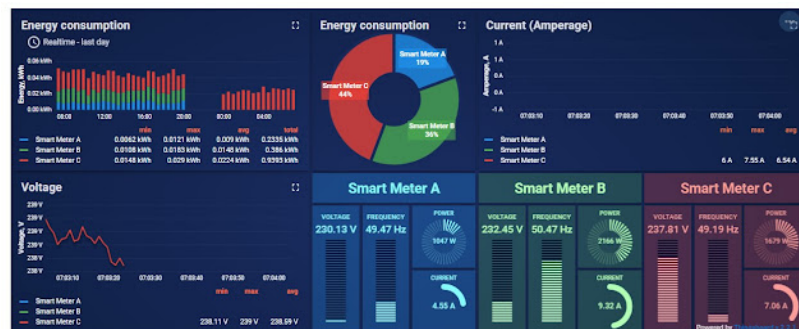
Tuy nhiên, các nền tảng trên đều cần trả phí và việc sử dụng nó vào các dự án IoT là phức tạp hơn so với Core IoT.

## Giới thiệu nền tảng Core IoT

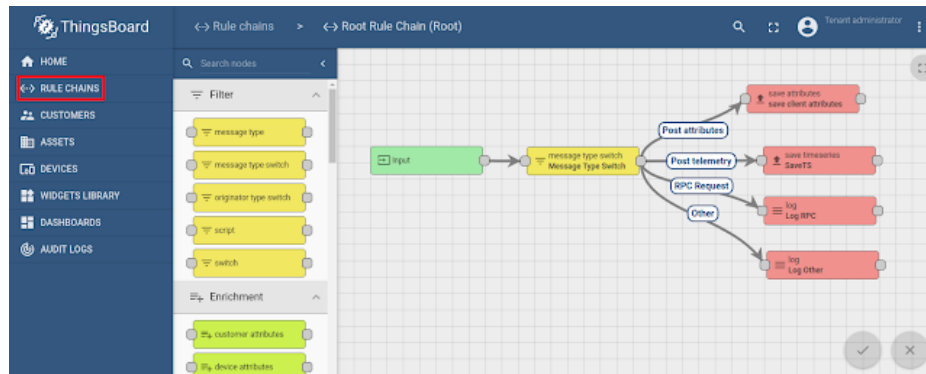
Core IoT là một nền tảng IoT mã nguồn mở, được phát triển dựa trên nền tảng Thingsboard nhưng được tùy chỉnh và triển khai miễn phí không giới hạn, phù hợp cho giáo dục và nghiên cứu cũng như nhu cầu làm các hệ thống thử nghiệm (prototyping). Nó cho phép phát triển nhanh chóng, quản lý và mở rộng các dự án IoT. Với nền tảng Core IoT, bạn có thể thu thập, xử lý, hiển thị trực quan và quản lý thiết bị.

Core IoT cho phép kết nối thiết bị thông qua các giao thức IoT tiêu chuẩn công nghiệp trong đó có MQTT, một giao thức phổ biến để thu thập dữ liệu và điều khiển thiết bị thông qua mạng Internet.

Một trong những ưu điểm của Core IoT, là nó hỗ trợ nhiều dịch vụ tiện ích mạnh mẽ, theo khuynh hướng của một ứng dụng công nghiệp. Chẳng hạn như màn hình điều hành của Core IoT, hỗ trợ rất nhiều giao diện phức tạp và hiện đại, như ví dụ sau đây:



Một dịch vụ khác, khiến người dùng chọn Core IoT, là khả năng xây dựng một chuỗi các tương tác để hệ thống có thể vận hành tự động. Khi một điều kiện xảy ra, nó sẽ tự động kích hoạt các tác vụ tiếp theo. Khái niệm này khá tương đồng với IFTTT (IF THIS THEN THAT). Trên Core IoT, dịch vụ này gọi là Rule Chains, như minh họa ở hình sau đây:



## Thực hành: Tìm hiểu và thao tác cơ bản với Core IoT Platform

Mục tiêu của phần thực hành này là giúp bạn biết cách sử dụng các tính năng phổ biến nhất của Core IoT, bao gồm:

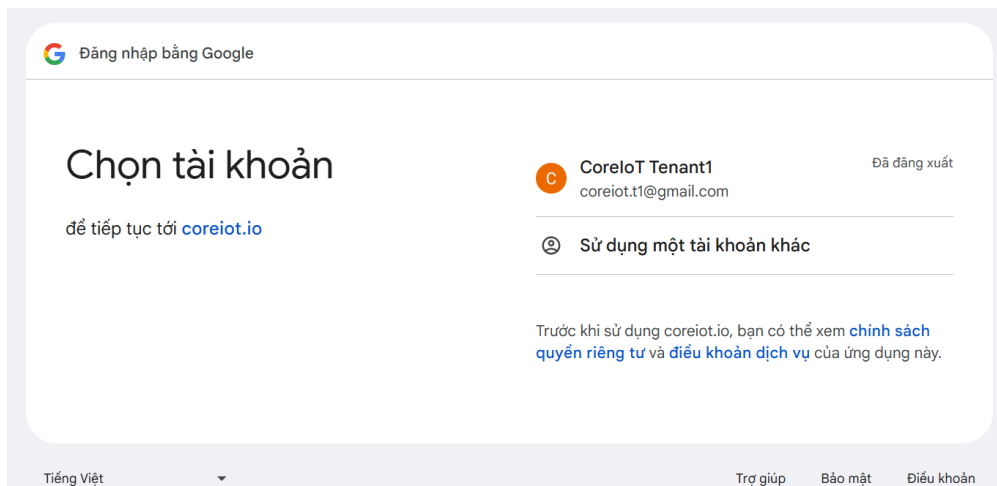
- Kết nối thiết bị với Core IoT;
- Lập trình gửi dữ liệu từ thiết bị lên Core IoT;
- Xây dựng dashboard người dùng để hiển thị dữ liệu theo thời gian thực;

Phần thực hành này sẽ sử dụng lại dự án Giám sát nhiệt độ phòng và bật tắt đèn từ xa đã xây dựng ở chương trước.

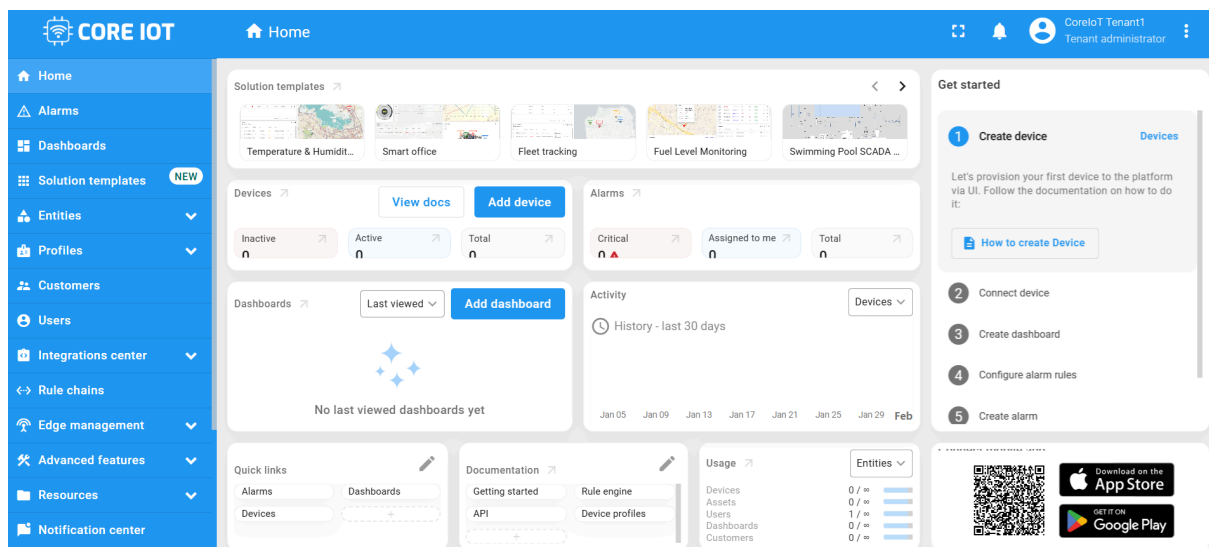
### Tạo tài khoản trên Core IoT

Bạn vào link <https://app.coreiot.io/> và chọn Login bằng Google hoặc Facebook hoặc Github để đăng ký một tài khoản mới.

Ví dụ nếu bạn sử dụng tài khoản Google để login thì bạn cần xác nhận tài khoản để sử dụng ở màn hình tiếp theo sau đó:

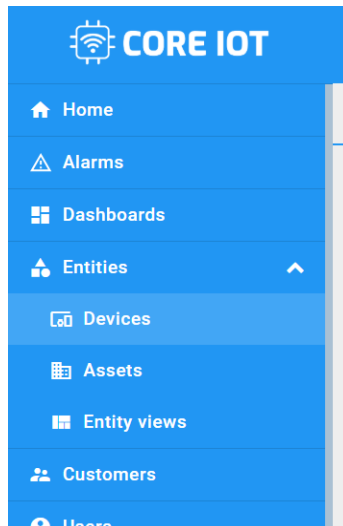


Tương tự cho Facebook và Github, sau khi xác nhận chọn tài khoản sử dụng xong, một tài khoản mới sẽ được tạo trên Core IoT và sau bước này bạn đã có thể đăng nhập vào hệ thống với tài khoản của mình. Giao diện sau khi login sẽ như sau:

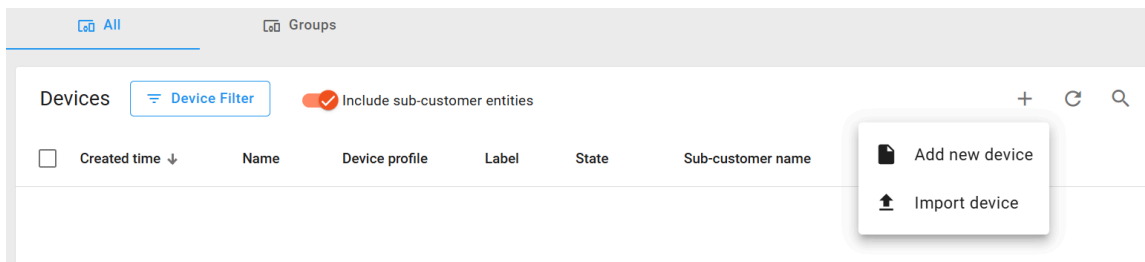


## Tạo Device trên Core IoT

Device là một tài nguyên sẽ được tạo ra trên Core IoT để lưu trữ các thông tin. Thông thường, chúng ta sẽ dùng dịch vụ Device này để lưu trữ thông tin cảm biến từ thiết bị. Từ danh sách menu bên trái, chọn vào **Entities** và chọn tiếp vào **Devices**.



Tiếp theo, từ kí hiệu dấu cộng (+) ở góc bên phải, nhấn vào đó và chọn tiếp vào Add new Device



Điền các thông tin tương ứng, như minh họa ở hình bên dưới:

Add new device

1 Device details

2 Credentials  
Optional

Name\*

YOLO UNO

Label

YOLO UNO

Device profile\*

default

X

Is gateway

Owner and groups

Owner\*

Next: Credentials

Cancel

Add

Nhấn tiếp vào nút **Next Credentials**, để đến giao diện sau đây:

Lưu ra file Notepad thông tin của token, bạn sẽ cần nó để gửi dữ liệu lên server Core IoT ở phần tiếp theo. Nhấn nút **Add** để kết thúc quá trình này.

Ở màn hình "Check connectivity", bạn có thể copy câu lệnh “curl -v -x POST <http://app.coreiot.io/api/v1...>” trong cửa sổ Windows command line (từ Windows 10 trở lên) để giả lập một thiết bị kết nối và gửi dữ liệu nhiệt độ lên Core IoT.

Sau khi chạy câu lệnh, trạng thái của thiết bị sẽ chuyển từ Inactive sang Active và thông tin nhiệt độ của được hiển thị dưới dạng một Telemetry có key là temperature như hình dưới đây.

Device created. Let's check connectivity! ✕

Windows
MacOS
Linux

**Install necessary client tools**

Starting Windows 10 b17063, cURL is available by default

**Execute the following command** HTTP HTTPs

```
curl -v -X POST http://app.coreiot.io/api/v1/898napce6v5pnchnvzu8/
```

**State** Active

Latest telemetry

| Time                | Key         | Value |
|---------------------|-------------|-------|
| 2025-02-01 21:11:10 | temperature | 25    |

☐ Do not show again
**Close**

Nhấn **Close** để kết thúc việc tạo một Device trên Core IoT.

## Màn hình điều hành (Dashboard)

Màn hình điều hành là một dịch vụ nổi bật trên Core IoT, với các giao diện rất phong phú và đa dạng. Trong bài hướng dẫn này, chúng ta sẽ tạo ra 2 ô hiển thị thông tin và một nút nhấn để điều khiển cho thiết bị phần cứng (bật/tắt bóng đèn). Các bước chi tiết để tạo ra một màn hình điều hành (Dashboard) được trình bày bên dưới.

Bước 1: Từ mục **Dashboards** ở menu bên trái, chọn tiếp vào biểu tượng thêm mới ở góc bên phải, như sau:

Bước 2: Đặt tên cho DashBoard (ví dụ như IOT UNO) và nhấn tiếp vào **Add**.

Add dashboard

Title\*

IOT UNO

Description

Mobile application settings

☐ Hide dashboard in mobile application

Dashboard order in mobile application

Dashboard image

No image selected

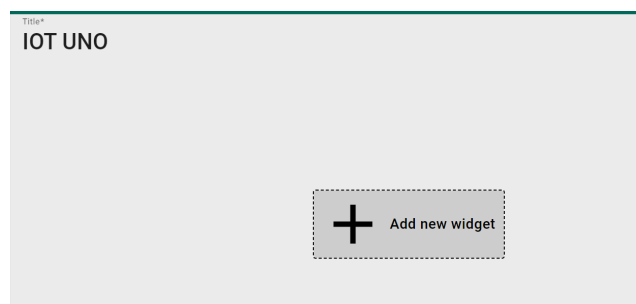
Browse from gallery

Set link

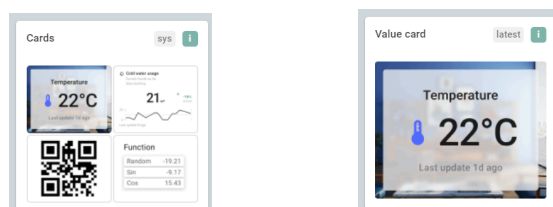
Cancel

Add

Bước 3: Giao diện sau đây sẽ hiện ra. Ở trạng thái mặc định, sẽ không có giao diện nào trên DashBoard. Chúng ta cần nhấn tiếp vào nút **Add new widget**



Bước 4: Chọn giao diện Cards để hiển thị thông tin, như minh họa ở hình bên dưới:



Bước 5: Liên kết giao diện với Device đã tạo ở bước trước, và liên kết tiếp với trường dữ liệu chứa thông tin nhiệt độ (còn gọi là telemetry data), ở đây là **temperature**

### Datasource

Device\*  
YOLO UNO

Data key\*  
temperature

Bước 6: Lặp lại các bước ở trên, để thiết kế thêm 1 giao diện cho việc hiển thị độ ẩm. Tuy nhiên lần này, khi cấu hình cho trường thông tin ở Bước 5, chúng ta phải chọn tạo mới một data key có tên là humidity, loại là Time series (biểu tượng chart trong thông báo cần tạo mới) và cấu hình cho phần hiển thị (Appearance) cho phù hợp với dữ liệu nhiệt độ, bao gồm chỉnh icon và đơn vị của nó, như minh họa sau đây:

Device\*  
YOLO UNO

Data key\*  
humidity

'humidi...' not found. Create a new one!   

Appearance

☒ Auto scale

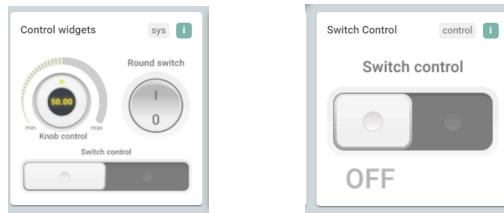
☒ Label  
Humidity

☒ Icon  
40 px

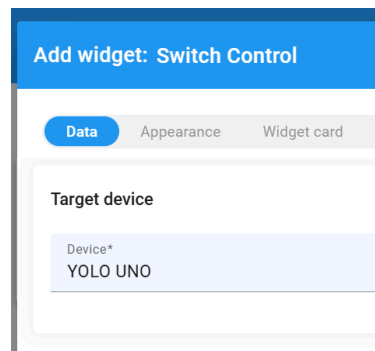
Value  
% 0 decimals

☒ Date  
Last update N ago

Bước 7: Thêm nút nhấn cho việc điều khiển thiết bị: Vẫn tương tự như việc thêm một giao diện hiển thị, nhưng lần này chúng ta sẽ chọn vào Control widgets rồi chọn tiếp vào Switch control, như sau:

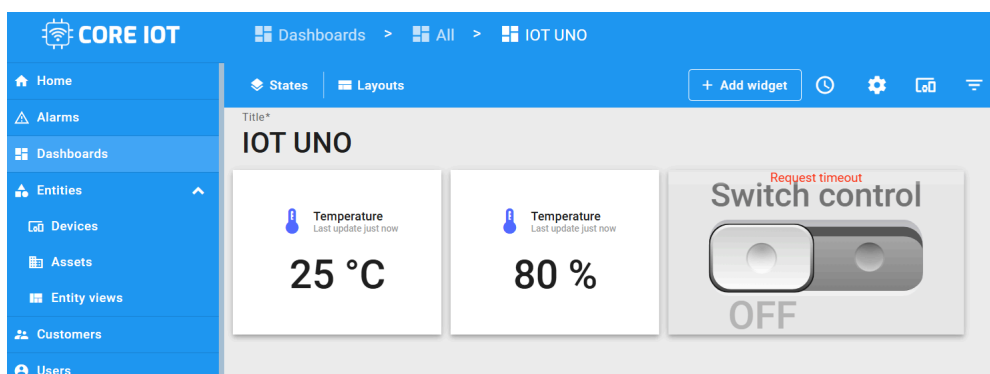


Bước 8: Với nút nhấn chúng ta chỉ cần chọn Device cho nó, chứ không cần chọn trường thông tin, như sau:

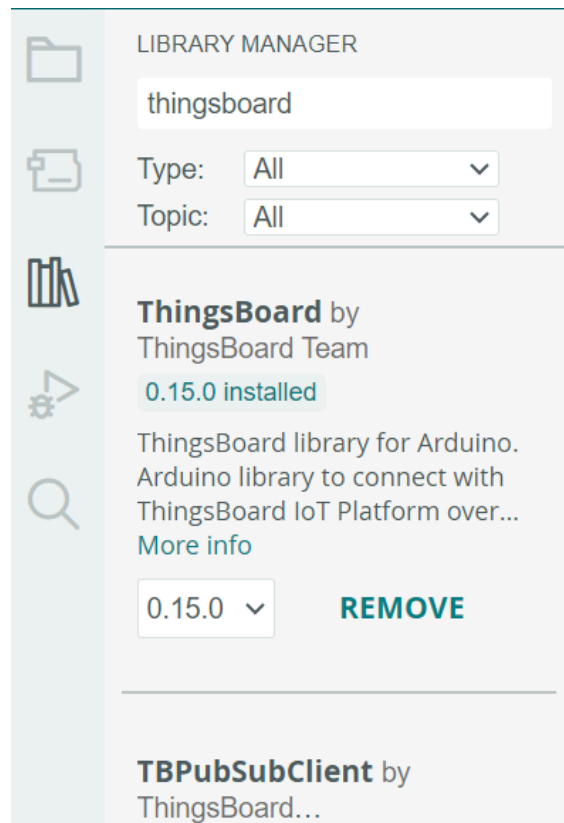


Trên dashboard sẽ báo lỗi Request timeout cho Switch control, tạm thời bạn cứ bỏ qua không cần quan tâm. Do hệ thống gửi yêu cầu cập nhật trạng thái hiện tại đến thiết bị nhưng thiết bị không hồi đáp (do chúng ta chưa lập trình cho thiết bị) nên Core IoT báo lỗi.

Cuối cùng, chúng ta có giao diện điều hành để có thể giám sát thông tin và điều khiển thiết bị như minh họa ở hình bên dưới.



Để chạy được chương trình cho thiết bị kết nối với Core IoT, bạn cần tham khảo hướng dẫn thêm thư viện cho Arduino IDE trong phần Phụ lục 1 để thêm thư viện ThingsBoard. Do Core IoT được phát triển dựa trên nền tảng ThingsBoard nên chúng có thể dùng chung thư viện lập trình qua lại.



Chương trình Arduino gửi dữ liệu lên Core IoT. Vì Core IoT được phát triển dựa trên Thingsboard nên chương trình dùng thư viện của Thingsboard để kết nối và gửi nhận dữ liệu.

```
#include <WiFi.h>
#include <WiFiClientSecure.h>

#include <Arduino_MQTT_Client.h>
#include <Server_Side_RPC.h>
#include <ThingsBoard.h>

#include "DHT20.h"
#include "Wire.h"

#define LED_PIN 48

// Enables the ThingsBoard class to be fully dynamic instead of requiring
// template arguments to statically allocate memory.
// If enabled the program might be slightly slower and all the memory will be
// placed onto the heap instead of the stack.
// #define THINGSBOARD_ENABLE_DYNAMIC 1

constexpr char WIFI_SSID[] = "YOUR_WIFI_SSID";
constexpr char WIFI_PASSWORD[] = "YOUR_WIFI_PASSWORD";
constexpr char TOKEN[] = "YOUR_DEVICE_ACCESS_TOKEN";
```

```

// Thingsboard we want to establish a connection too
constexpr char THINGSBOARD_SERVER[] = "app.coreiot.io";
constexpr uint16_t THINGSBOARD_PORT = 1883U;

// Maximum size packets will ever be sent or received by the underlying MQTT
// client,
// if the size is too small messages might not be sent or received messages will
// be discarded
constexpr uint16_t MAX_MESSAGE_SEND_SIZE = 128U;
constexpr uint16_t MAX_MESSAGE_RECEIVE_SIZE = 128U;

constexpr char CONNECTING_MSG[] = "Connecting to: (%s) with token (%s)\n";
constexpr char TEMPERATURE_KEY[] = "temperature";
constexpr char HUMIDITY_KEY[] = "humidity";

constexpr const char RPC_SET_SWITCH_METHOD[] = "setValue";
constexpr const char RPC_GET_SWITCH_METHOD[] = "getValue";
constexpr uint8_t MAX_RPC_SUBSCRIPTIONS = 3U;
constexpr uint8_t MAX_RPC_RESPONSE = 5U;

WiFiClient espClient;

// Initialize the Mqtt client instance
Arduino_MQTT_Client mqttClient(espClient);
// Initialize used apis
Server_Side_RPC<MAX_RPC_SUBSCRIPTIONS, MAX_RPC_RESPONSE> rpc;
const std::array<IAPI_Implementation*, 1U> apis = {
    &rpc
};
//const std::array<IAPI_Implementation*, 0U> apis = {};
// Initialize ThingsBoard instance with the maximum needed buffer size
ThingsBoard tb(mqttClient, MAX_MESSAGE_RECEIVE_SIZE, MAX_MESSAGE_SEND_SIZE,
Default_Max_Stack_Size, apis);

// Statuses for subscribing to rpc
bool subscribed = false;
bool switchState = false; // set switch to off when boot
constexpr int16_t telemetrySendInterval = 2000U;
uint32_t lastSendTime;

/// @brief Initializes WiFi connection,
// will endlessly delay until a connection has been successfully established
void InitWiFi() {
    Serial.println("Connecting to WiFi AP ...");
    // Attempting to establish a connection to the given WiFi network
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("Connected to AP");
}

```

```

}

/// @brief Reconnects the WiFi uses InitWiFi if the connection has been removed
/// @return Returns true as soon as a connection has been established again
bool reconnect() {
    // Check to ensure we aren't connected yet
    const wl_status_t status = WiFi.status();
    if (status == WL_CONNECTED) {
        return true;
    }

    // If we aren't establish a new connection to the given WiFi network
    InitWiFi();
    return true;
}

/// @brief Processes function for RPC call "setValue"
/// JsonVariantConst is a JSON variant, that can be queried using operator[]
/// See https://arduinojson.org/v5/api/jsonvariant/subscript/ for more details
/// @param data Data containing the rpc data that was called and its current
value
/// @param response Data containing the response value, any number, string or
json, that should be sent to the cloud. Useful for getMethods
void setSwitchValue(const JsonVariantConst &data, JsonDocument &response) {
    Serial.println("Received the set switch method");

    // Process data
    switchState = data;

    Serial.print("Example switch state: ");
    Serial.println(switchState);

    if (switchState == 1) {
        digitalWrite(LED_BUILTIN, HIGH);
    } else {
        digitalWrite(LED_BUILTIN, LOW);
    }

    response.set(switchState);
}

void getSwitchValue(const JsonVariantConst &data, JsonDocument &response) {
    Serial.println("Received the get switch method");
    response.set(switchState);
}

void setup() {
    // Initialize serial connection for debugging
    Serial.begin(115200);
    pinMode(LED_BUILTIN, OUTPUT);
    digitalWrite(LED_BUILTIN, LOW);
}

```

```

Wire.begin(SDA, SCL);
dht20.begin();

delay(1000);
InitWiFi();
}

void loop() {
    delay(10);

    if (!reconnect()) {
        return;
    }

    if (!tb.connected()) {
        // Reconnect to the ThingsBoard server,
        // if a connection was disrupted or has not yet been established
        Serial.printf(CONNECTING_MSG, THINGSBOARD_SERVER, TOKEN);
        if (!tb.connect(THINGSBOARD_SERVER, TOKEN, THINGSBOARD_PORT)) {
            Serial.println("Failed to connect");
            return;
        }
    }

    if (!subscribed) {
        Serial.println("Subscribing for RPC...");
        const std::array<RPC_Callback, MAX_RPC_SUBSCRIPTIONS> callbacks = {
            RPC_Callback{ RPC_SET_SWITCH_METHOD, setSwitchValue },
            RPC_Callback{ RPC_GET_SWITCH_METHOD, getSwitchValue }
        };

        if (!rpc.RPC_Subscribe(callbacks.cbegin(), callbacks.cend())) {
            Serial.println("Failed to subscribe for RPC");
            return;
        }

        Serial.println("Subscribe done");
        subscribed = true;
    }

    if (millis() - lastSendTime > telemetrySendInterval) {
        lastSendTime = millis();

        /*
        // Random data simulation
        Serial.println("Sending temperature data...");
        tb.sendTelemetryData(TEMPERATURE_KEY, random(10, 31));

        Serial.println("Sending humidity data...");
        tb.sendTelemetryData(HUMIDITY_KEY, random(40, 90));

```

```

*/

dht20.read();

float temperature = dht20.getTemperature();
float humidity = dht20.getHumidity();

if (isnan(temperature) || isnan(humidity)) {
    Serial.println("Failed to read from DHT20 sensor!");
} else {
    Serial.print("Temperature: ");
    Serial.print(temperature);
    Serial.print(" °C, Humidity: ");
    Serial.print(humidity);
    Serial.println(" %");

    Serial.println("Sending temperature data...");
    tb.sendTelemetryData("temperature", temperature);
    Serial.println("Sending humidity data...");
    tb.sendTelemetryData("humidity", humidity);
}
}

tb.loop();
}

```

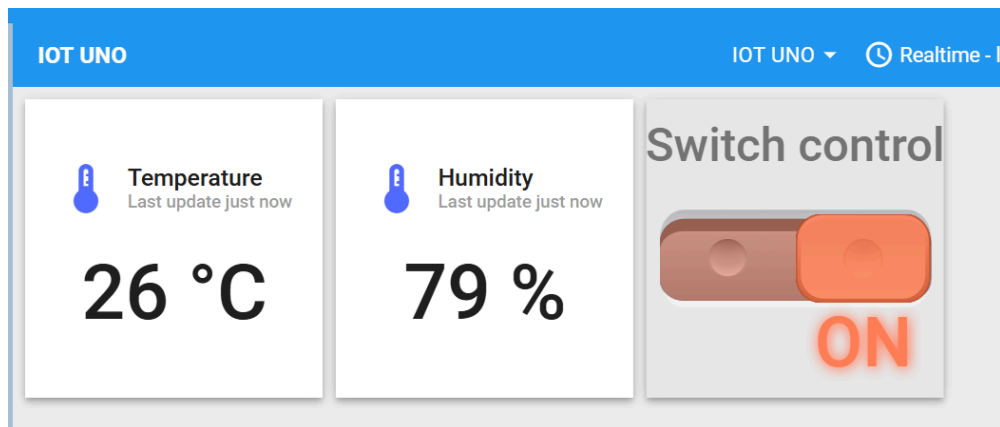
Bạn cần sửa các thông tin bao gồm thông tin mạng WiFi và quan trọng là access token của thiết bị đã tạo ở các bước trên cho đúng.

```

constexpr char WIFI_SSID[] = "YOUR_WIFI_SSID";
constexpr char WIFI_PASSWORD[] = "YOUR_WIFI_PASSWORD";
constexpr char TOKEN[] = "YOUR_DEVICE_ACCESS_TOKEN";

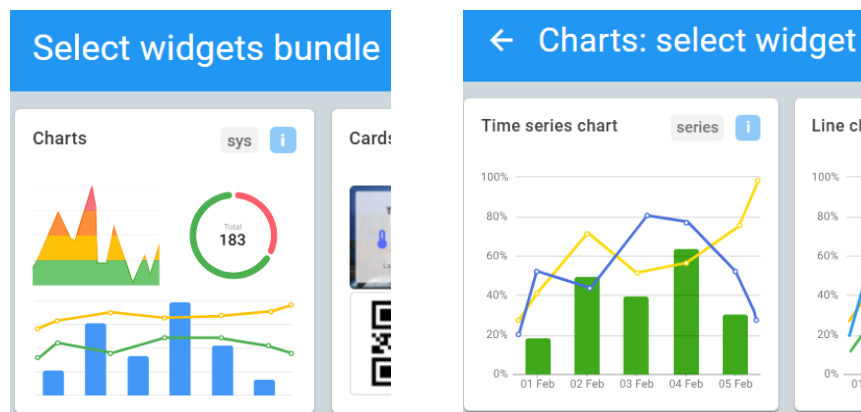
```

Sau khi compile và upload code vào mạch Yolo UNO, các thông tin sẽ được gửi lên Dashboard và bạn cũng có thể bật tắt đèn LED tích hợp trên mạch Yolo UNO.



Bạn cũng có thể thêm một biểu đồ cho thông tin nhiệt độ để Dashboard trực quan hơn. Biểu đồ cho phép bạn hiển thị các dữ liệu theo thời gian, với nhiều loại khác nhau.

- Vào chế độ chỉnh sửa Dashboard và nhấp vào "Add widget";
- Tìm loại Charts widget >> Time series chart và nhấn vào đó;



- Chọn thiết bị đã tạo trước đó "YOLO UNO" trong phần "Device". Trong phần "Series", chọn dữ liệu "temperature" để hiển thị giá trị nhiệt độ của thiết bị gửi lên. Sau đó, nhấp vào "Add";

**Add widget: Time series chart** Basic Advanced X

**Datasource** Device Entity alias

Device\*  
YOLO UNO X

**Series** Series Comparison

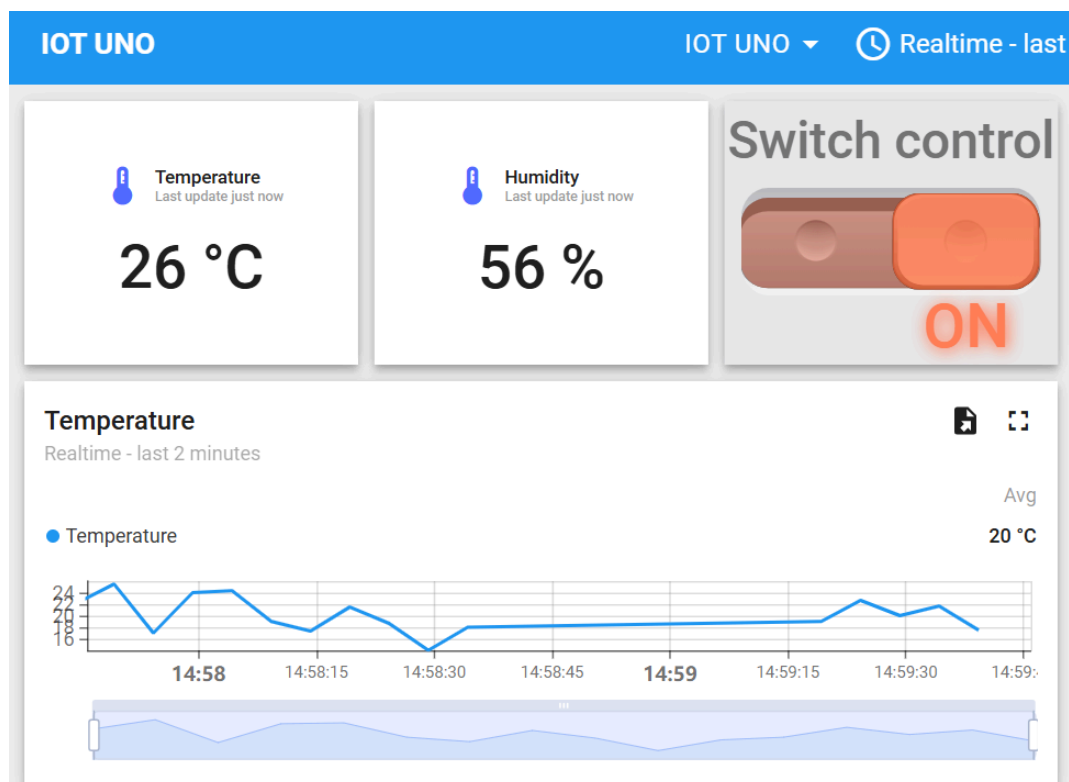
| Key     | Label       | Type | Y axis  | Color | Units | Decimals |
|---------|-------------|------|---------|-------|-------|----------|
| temp... | Temperature |      | default |       | °C    | 0        |

Add series

Y axes

Cancel Preview Add

- Bạn có thể thay đổi kích thước và chọn vị trí đặt trên Dashboard cho phù hợp. Mặc định thì biểu đồ sẽ hiển thị một phút dữ liệu theo mặc định. Nhấp vào "Add" để áp dụng các thay đổi;



- Bạn cũng có thể thay đổi khung thời gian (time interval) hiển thị dữ liệu cho biểu đồ, thay đổi hàm tính toán (aggregation function, ví dụ Min, Max, Average...) và các khoảng thời gian nhóm dữ liệu (grouping interval, ví dụ 5 giây, 10 giây...). Để thay đổi, bạn mở cửa sổ Time window và thay đổi các tùy chọn tương ứng và nhấn vào nút "Update" để cập nhật.

IOT UNO ▾ ⌚ Realtime - last 1 minute 🖨️ Edit mode ⬇️

Switch

Realtime History

Time window

Last Relative UTC+07:00

1 minute ▾

Aggregation Average ▾

Grouping interval 1 second ▾

Cancel Update

# Phụ lục 1: Hướng dẫn lập trình Yolo UNO bằng Arduino

## Giới thiệu về Arduino

Arduino là một nền tảng mã nguồn mở bao gồm cả phần mềm và phần cứng được thiết kế giúp làm việc với các bo mạch điện tử trở nên dễ dàng hơn bao giờ hết. Arduino xóa đi sự phức tạp của điện tử bằng cách đơn giản hóa các khái niệm, che đi sự phức tạp của phần mềm.

Bởi vì tính phổ biến và dễ dùng cùng với các vô vàn thư viện được chia sẻ sẵn để làm việc với rất nhiều module phần cứng khác nhau, bạn chỉ cần quan tâm đến tính năng sản phẩm mà bỏ qua các khái niệm chuyên sâu phức tạp (protocol, datasheet ...) từ đó dễ dàng tiếp cận và làm ra các sản phẩm tuyệt vời mà không cần phải biết nhiều về điện tử.

Ban đầu phần lớn các board mạch Arduino này đều dựa trên các chip họ AVR của Atmel sản xuất, nhưng sau này có rất nhiều nhà sản xuất sử dụng các chip khác nhau như ARM, PIC, STM32 và phổ biến sau này là các dòng ESP8266, ESP32 của hãng Espressif...

Tham khảo thêm về dự án Arduino

- [Arduino chính thức\(IDE & AVR/ARM/x86 Board\) www.arduino.cc](http://www.arduino.cc)
- [Arduino cho ESP8266 github.com/esp8266/Arduino](https://github.com/esp8266/Arduino)
- [Arduino cho ESP32 github.com/espressif/arduino-esp32](https://github.com/espressif/arduino-esp32)
- [Arduino cho PIC32 chipkit.net/](http://chipkit.net/)
- [Arduino cho STM32 www.stm32duino.com/](http://www.stm32duino.com/)
- [Các dự án Arduino www.hackster.io/arduino](http://www.hackster.io/arduino)

## Tải và cài đặt phần mềm Arduino IDE

Bước 1: Vào trang chủ của Arduino để download Arduino IDE theo link này <https://www.arduino.cc/en/software/>. Trong mục Download options như hình dưới, bạn chọn **Win 10 and newer** để download.

# Downloads



## Arduino IDE 2.3.2

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

SOURCE CODE

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

### DOWNLOAD OPTIONS

**Windows** Win 10 and newer, 64 bits  
**Windows** MSI installer  
**Windows** ZIP file

**Linux** AppImage 64 bits (X86-64)  
**Linux** ZIP file 64 bits (X86-64)

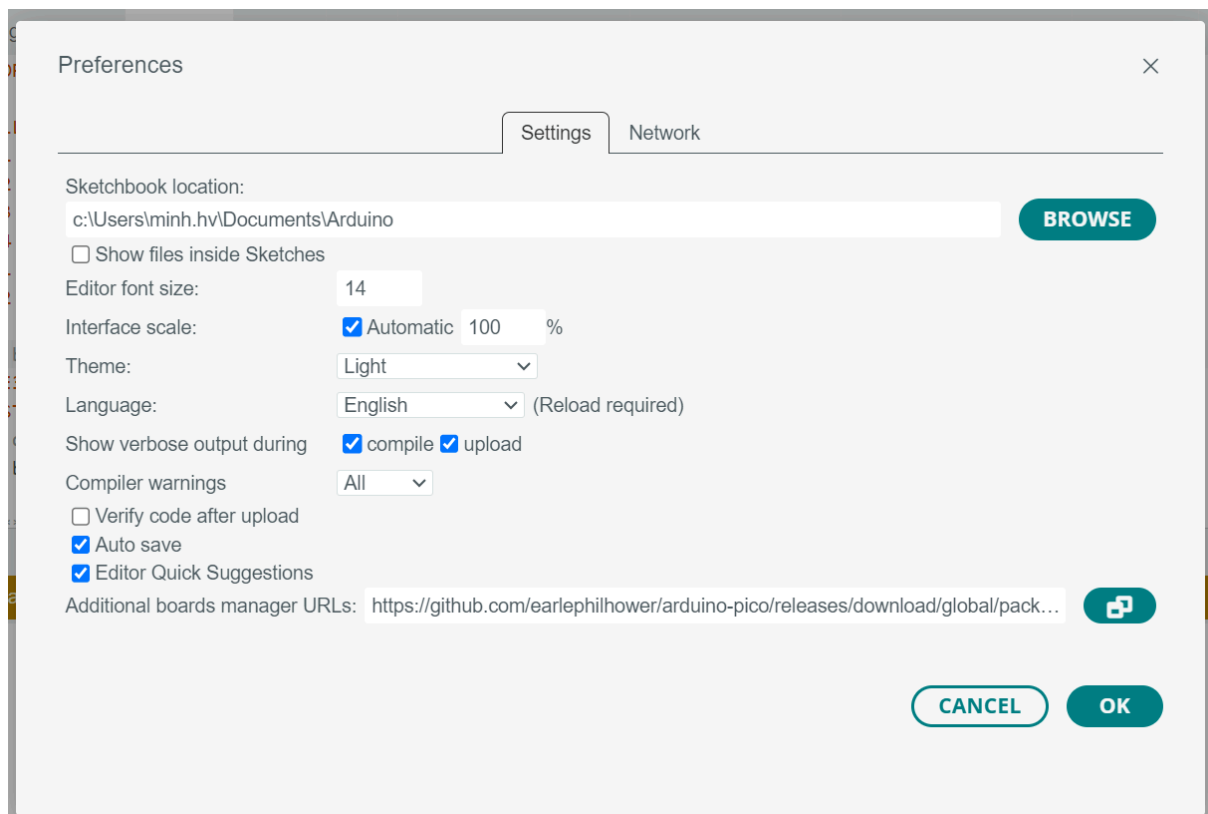
**macOS** Intel, 10.15: "Catalina" or newer, 64 bits  
**macOS** Apple Silicon, 11: "Big Sur" or newer, 64 bits

[Release Notes](#)

## Giao diện download Arduino IDE

Bước 2: Cài đặt hỗ trợ các board OhStem để Arduino hỗ trợ Yolo UNO

Mở phần mềm Arduino IDE đã tải về, vào **File > Preferences**. Trong tab Settings, chọn các tùy chọn như hình dưới:

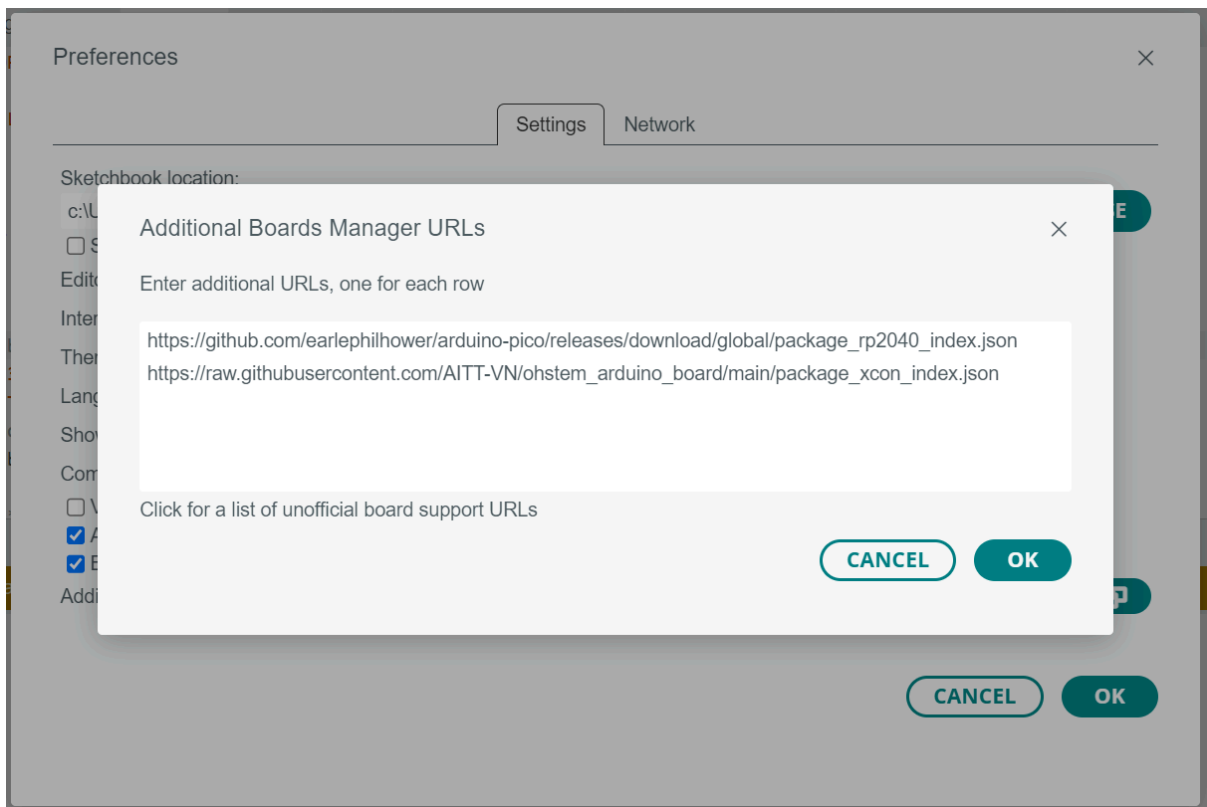


**Sketchbook location** là đường dẫn mà bạn muốn lưu các file chương trình. Trong Windows đường dẫn mặc định là **C:\Users\YourUserName\Documents\Arduino** còn trên các hệ điều hành Linux hoặc MacOS thì đường dẫn mặc định là **/home/name\_your\_computer/Arduino**. Đây cũng sẽ là nơi lưu những thư viện cho Arduino mà chúng ta sẽ thêm vào sau này.

Trong mục **Additional Board Manager URLs**, nhấn vào nút  bên cạnh để mở ra cửa sổ mới và nhập vào đường dẫn đến thư viện dành riêng cho các mạch của OhStem là:

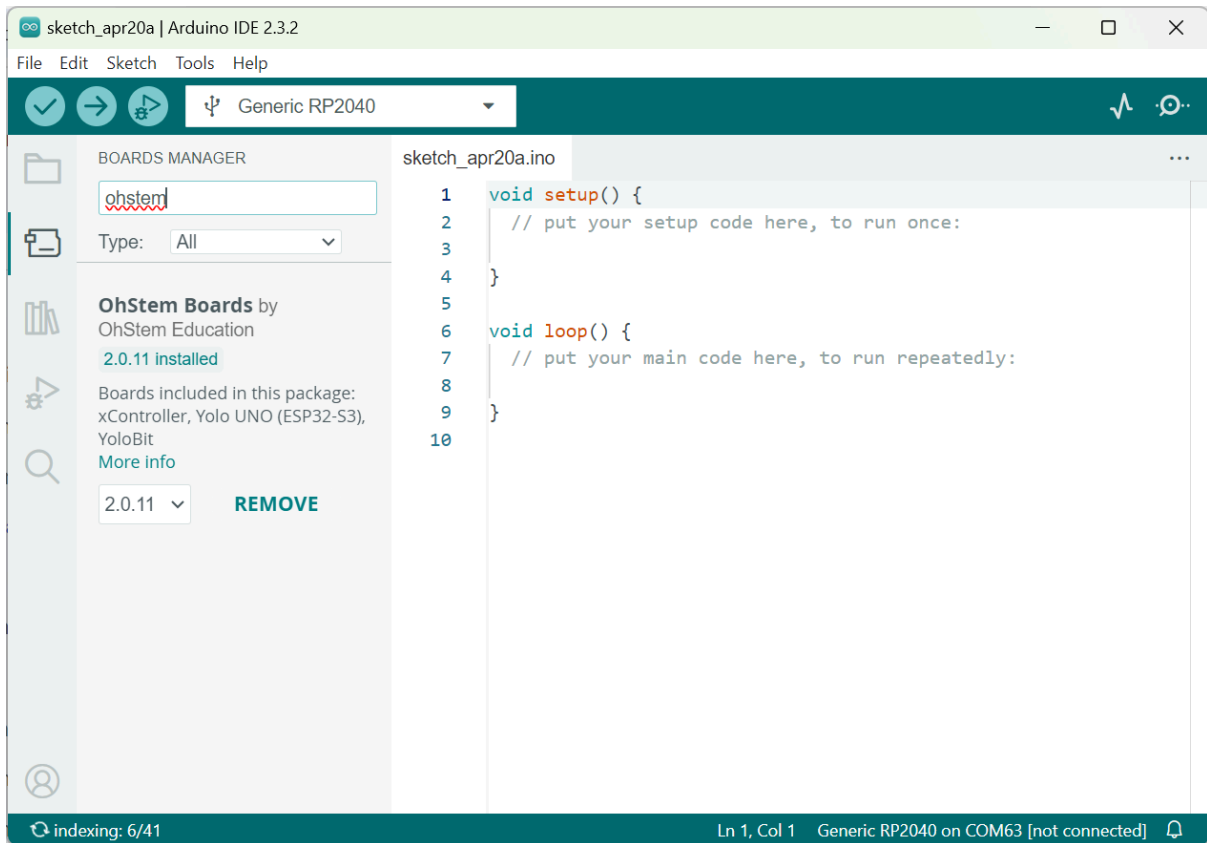
[https://raw.githubusercontent.com/AITT-VN/ohstem\\_arduino\\_board/main/package\\_xcon\\_index.json](https://raw.githubusercontent.com/AITT-VN/ohstem_arduino_board/main/package_xcon_index.json).

Thư viện cho mỗi loại mạch sẽ được đặt trên 1 hàng riêng biệt như hình dưới đây:

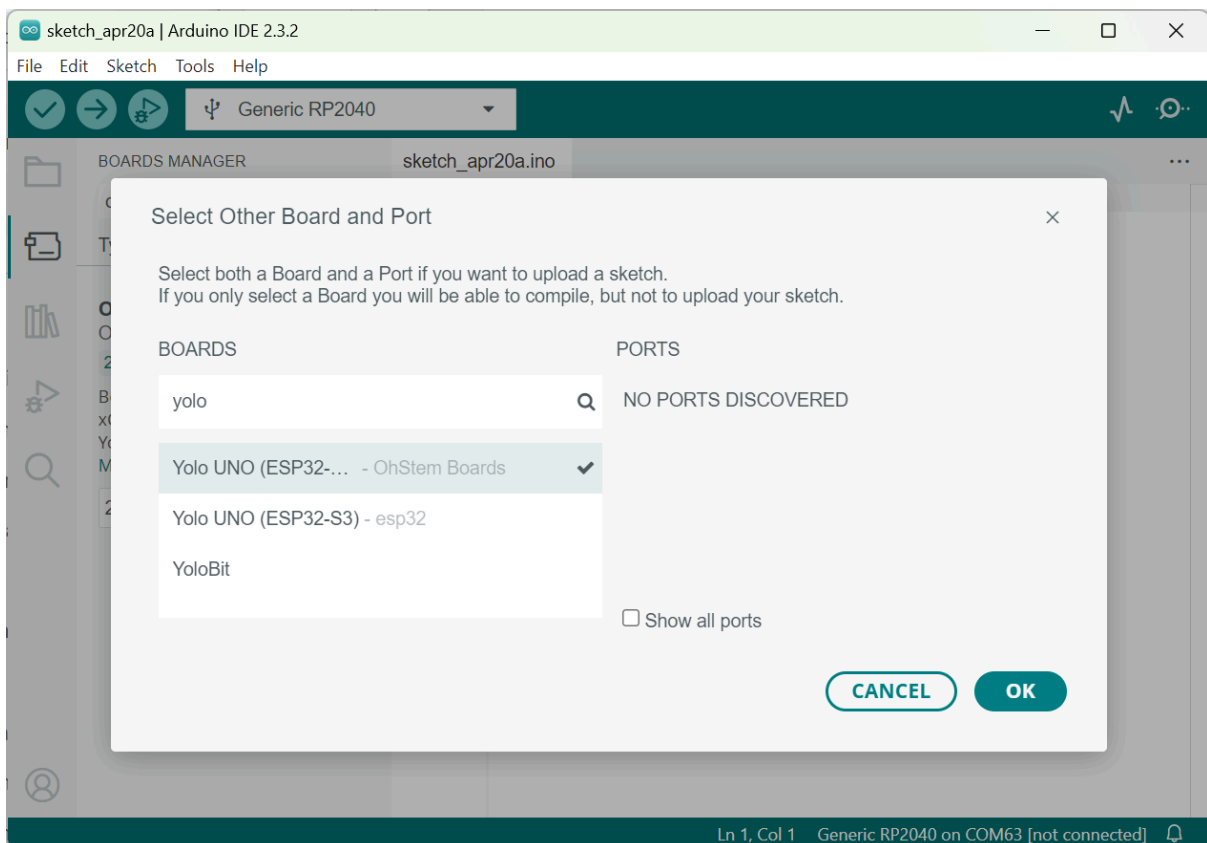


Để hỗ trợ nhiều loại board khác nhau, bạn có thể nhập thành nhiều dòng bằng cách Enter xuống dòng cho từng URL.

Bước 3: Mở menu **Tools > Board > Boards Manager...** hoặc nhấn vào biểu tượng bo mạch trên menu dọc trái, gõ ohstem vào thanh tìm kiếm, chọn board **OhStem Boards by OhStem Education** như hình dưới và nhấn vào install.



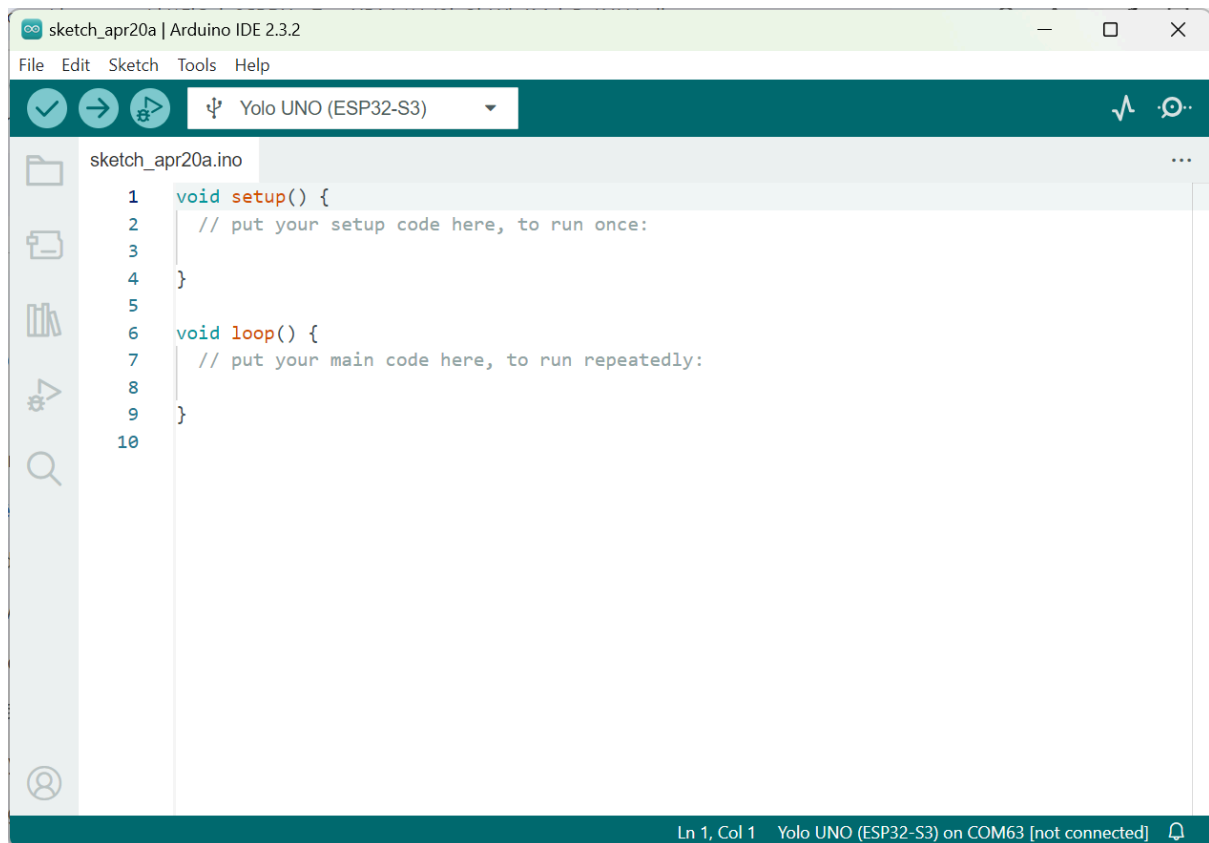
Sau khi cài đặt, kết nối mạch Yolo UNO với máy tính qua dây cáp USB, chọn mạch Yolo UNO và kết nối.



Nếu Yolo UNO đang được nối với máy tính thì phần Tools \ Port sẽ xuất hiện 1 COM port (không phải COM1, COM2), ví dụ COM3, COM4 hay COM5... tức là PC đã nhận được board và việc cài đặt phần mềm đã thành công.

## Làm quen với phần mềm Arduino IDE

Sau khi cài đặt thành công, chúng ta sẽ cùng tìm hiểu cách sử dụng phần mềm Arduino IDE và cấu trúc một chương trình của Arduino.



*Giao diện phần mềm Arduino IDE*

Về cơ bản, Arduino IDE có giao diện khá đơn giản và quen thuộc. Giao diện này nhìn rất giống với các phần mềm text editor phổ biến như Notepad. Ngoài thanh menu thì giao diện phần mềm Arduino IDE còn có thêm một số công cụ và khu vực hiển thị giúp bạn làm việc và quản lý code dễ dàng.

### Các nút chức năng trên giao diện

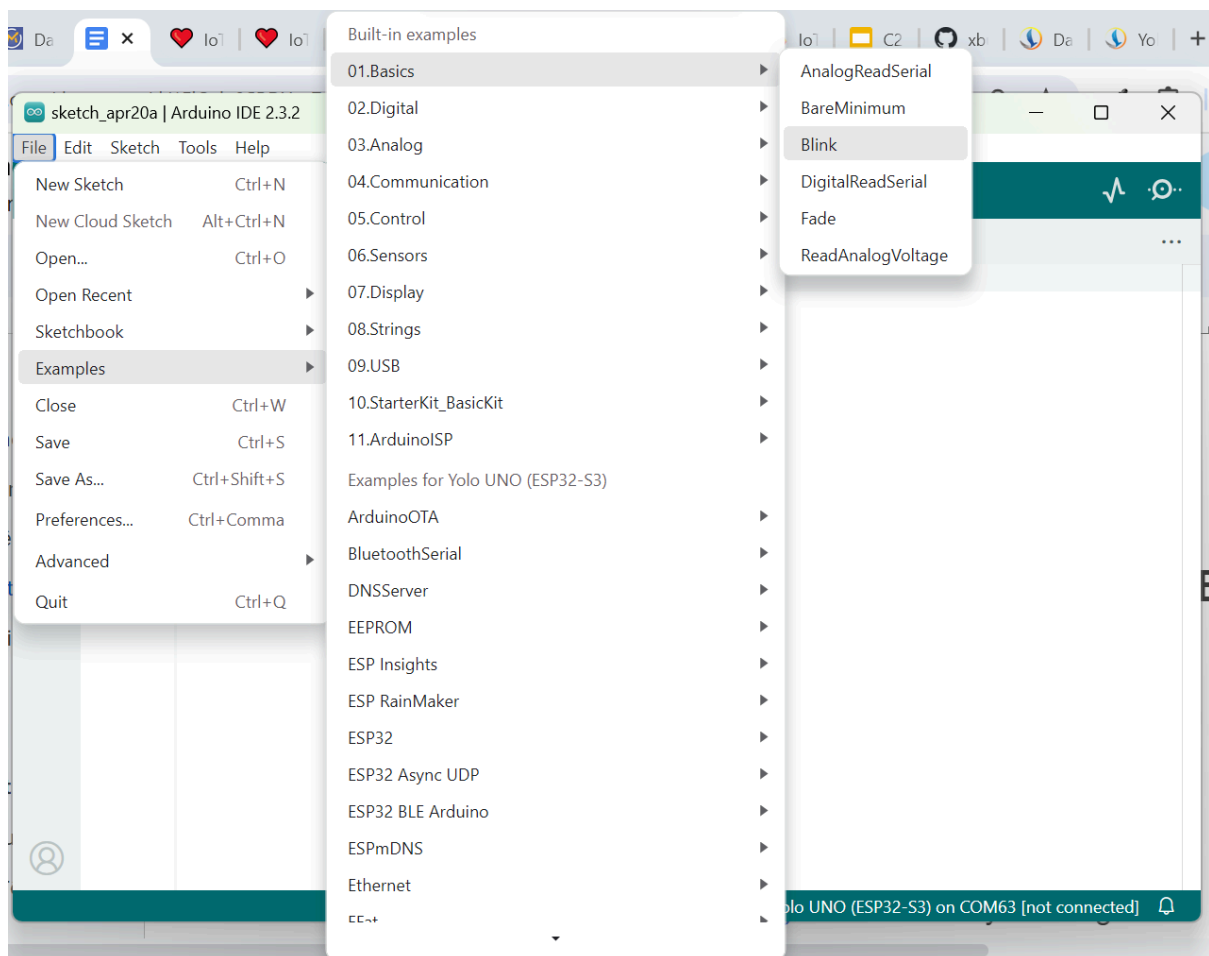
- Nút trông giống như dấu check là “Verify”. Bạn có thể dùng nút này để biên dịch thử code xem có bị lỗi gì không. Sau khi biên dịch, thông tin về kết quả biên dịch được hiển thị ở panel bên dưới. Đồng thời, nó cũng cho bạn

biết kích thước của đoạn code chiếm bao nhiêu bộ nhớ của board mạch mà bạn đang làm việc. Bạn có thể nhấn đồng thời 2 nút Control + R thay vì click vào “Verify”.

- Nút bên cạnh nút Verify, có hình mũi tên trở sang bên phải là nút để Upload code. Sau khi viết code xong, bạn có thể dùng nút này để biên dịch và nạp code vào board Arduino mà bạn đã chọn. Bạn cũng có thể dùng shortcut thay thế là Control + U.
- Nút có hình kính lúp ở góc trên bên phải là nút để mở cửa sổ Serial monitor. Bạn sẽ dùng chức năng Serial monitor để gửi và nhận thông tin qua lại giữa PC và board Arduino trong lúc board đang chạy. Nó sẽ giúp bạn dễ dàng debug đoạn code. Chúng ta sẽ tìm hiểu sâu hơn chức năng này ở các bài sau. Shortcut cho nút này là Shift + Control + M.

## Cấu trúc một chương trình của Arduino

Mỗi một chương trình trong Arduino được gọi là 1 sketch. Phần mềm Arduino IDE có kèm theo rất nhiều sketch mẫu (example) giúp bạn dễ dàng làm quen với các khái niệm cơ bản. Bạn hãy thử mở 1 sketch mẫu đầu tiên là Blink bằng cách vào menu **File > Examples > 01.Basics > Blink**.



Blink led được coi là bài thực hành cơ bản đầu tiên trong lập trình nhúng, tương tự như chương trình Hello World trong bất cứ ngôn ngữ lập trình nào.

```
/*
  Blink
  Turns an LED on for one second, then off for one second, repeatedly.
  Most Arduinos have an on-board LED you can control. On the UNO, MEGA
  and ZERO
  it is attached to digital pin 13, on MKR1000 on pin 6. LED_BUILTIN is
  set to
  the correct LED pin independent of which board is used.
  If you want to know what pin the on-board LED is connected to on your
  Arduino
  model, check the Technical Specs of your board at:
  https://www.arduino.cc/en/Main/Products
  modified 8 May 2014
  by Scott Fitzgerald
  modified 2 Sep 2016
  by Arturo Guadalupi
  modified 8 Sep 2016
  by Colby Newman
  This example code is in the public domain.
  http://www.arduino.cc/en/Tutorial/Blink
*/
// the setup function runs once when you press reset or power the board
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}
// the loop function runs over and over again forever
void loop() {
  digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the
  voltage level)
  delay(1000); // wait for a second
  digitalWrite(LED_BUILTIN, LOW); // turn the LED off by making the
  voltage LOW
  delay(1000); // wait for a second
}
```

## Comment

Trong đoạn code example Blink ở trên, bạn có thể thấy 23 dòng đầu tiên được hiển thị mờ hơn so với các dòng khác. Tất cả các đoạn text nằm giữa 2 dấu `/* */` được gọi là chú thích cho code (comment). Đây gọi là multi-line comment.

Ở các dòng như 25 hay dòng 27 cũng là các đoạn text comment nhưng bắt đầu bằng dấu // và không có dấu kết thúc bao lại như multi-line comment, được gọi là one line comment.

Comment được sử dụng để giúp cho code trở nên dễ đọc hơn. Trong các đoạn code ngắn như Blink thì comment có thể không cần thiết lắm nhưng sẽ rất hữu dụng trong các chương trình lớn với rất nhiều dòng code. Ngoài ra bạn cũng có thể dùng comment để tắt tạm thời 1 vài đoạn code trong quá trình viết code mà không cần xóa đi. Khi biên dịch và upload vào board, Arduino sẽ bỏ qua các đoạn code này.

[>>> Tham khảo Arduino comments](#)

## Hàm setup()

Đoạn code tiếp theo là hàm setup **`void setup()`**. Các dòng code trong hàm setup sẽ được chạy một lần duy nhất mỗi khi board Arduino được cấp nguồn và chạy, dùng cho các tác vụ khởi tạo và setup các tài nguyên cần thiết trước khi chương trình chính được chạy. “void” là một loại dữ liệu, với ý nghĩa là hàm setup này không trả về thông tin gì sau khi chạy.

Sau tên hàm là 2 dấu ngoặc ( ), có thể dùng để khai báo các dữ liệu đầu vào cho hàm này. Vì hàm setup không cần dữ liệu đầu vào nên ta để trống giữa 2 dấu ngoặc ( ) này. Bạn sẽ được tìm hiểu thêm vấn đề này qua các ví dụ ở các bài học sau.

Code của một hàm sẽ luôn được đặt trong 2 dấu ngoặc nhọn { }. Khi code, bạn cần để ý đóng ngoặc cho đúng để tránh các lỗi khi biên dịch. Phần mềm Arduino IDE sẽ giúp bạn bằng cách highlight các dấu ngoặc này khi con trỏ bạn đang được đặt ở dấu ngoặc còn lại.

- [Tham khảo void trong Arduino](#)
- [Tham khảo Arduino setup](#)
- [Tham khảo Arduino curly braces](#)

Dòng code thực sự đầu tiên là **`pinMode(LED_BUILTIN, OUTPUT);`** ở dòng 28. **pinMode** là câu lệnh của Arduino dùng để thiết lập chế độ hoạt động của một chân IO của vi điều khiển (gọi là IO pin). **LED\_BUILTIN** là một giá trị đã được khai báo trước trong Arduino và sẽ khác nhau tùy theo loại board Arduino mà bạn đang chọn.

Tham số thứ 2 khi gọi hàm này là chế độ hoạt động mà ta muốn thiết lập, **OUTPUT** là chế độ cho phép chúng ta bật hay tắt pin để điều khiển các module gắn vào pin này. Ngược lại với **OUTPUT** là **INPUT** trong trường hợp ta cần đọc trạng thái từ module bên ngoài gắn với pin này.

- [Arduino reference: pinMode](#)

- [Arduino tutorials: Digital Pins](#)

Trong phần mềm Arduino IDE và cũng như ngôn ngữ lập trình C, các câu lệnh được kết thúc bằng dấu chấm phẩy ; như bạn thấy trong đoạn code mẫu Blink ở trên. Một trong những lỗi thường gặp khi các bạn mới viết code cho Arduino là thiếu dấu chấm phẩy ; cuối mỗi lệnh và bạn sẽ thấy các thông báo như thế này: Blink:16: error: expected ';' or ',' before... Khi đó bạn cần kiểm tra xem có thiếu dấu ; ở cuối câu hay không.

- [Arduino reference: ; semicolon](#)

## Hàm loop()

Hàm loop() là chương trình chính của Arduino. Đoạn code trong loop() sẽ được Arduino chạy vô hạn. Trong chương trình này, có hàm **digitalWrite()** để đặt các chân (pin) ở mức điện áp cao (**HIGH**) hay thấp (**LOW**). Hàm tiếp theo là **delay()**, nhận đối số là một số nguyên, thể hiện số mili giây ta muốn chương trình tạm ngưng. 1000 millisecond tương đương với 1 giây.

- [Arduino reference: loop](#)
- [Arduino reference: digitalWrite](#)
- [Arduino reference: delay](#)

Sau khi câu lệnh dòng 36 chạy xong thì Arduino sẽ chạy lặp lại từ đầu hàm main(), tức là quay lại câu lệnh dòng 33 và cứ như vậy cho đến khi bị tắt nguồn.

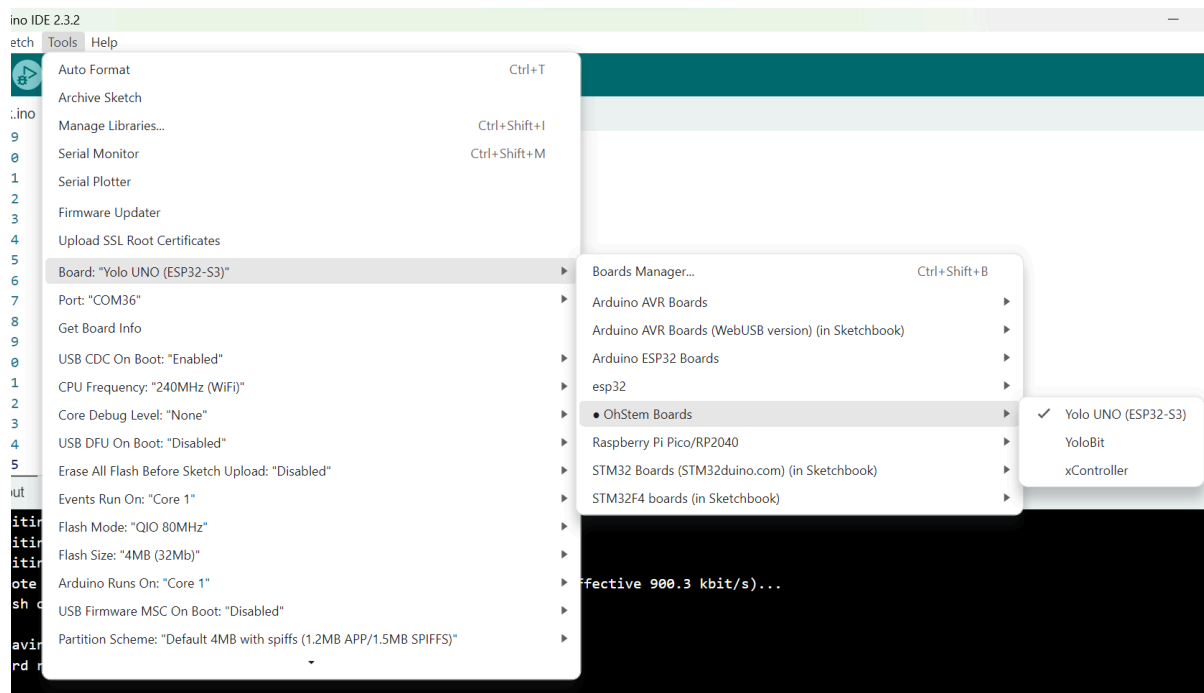
## Upload code trong Arduino IDE

Để có thể upload code cho ESP32-S3 lần đầu trên Arduino, bạn cần đưa vào chế độ bootloader. Chế độ này là một phần mềm nhỏ được lưu trữ trong bộ nhớ flash, chịu trách nhiệm khởi động chip và tải ứng dụng người dùng từ bộ nhớ flash hoặc qua OTA (cập nhật qua mạng).

Bạn thao tác bằng cách sau để vào mode bootloader:

1. Nhấn giữ nút Boot có trên mạch Yolo UNO
2. Đồng thời nhấn nút Reset và thả ra sau 1 giây để reset lại mạch
3. Sau bước trên khoảng 2 giây thì thả nút Boot ra

Trên Arduino IDE, bạn chọn đúng loại board là Yolo UNO và cổng COM tương ứng với port mà Arduino IDE nhận được và nhấn nút Upload.



Để biên dịch và upload code, Arduino IDE có thể cần khoảng 20-30 giây cho lần đầu tiên và sẽ nhanh hơn cho những lần biên dịch sau.

Sau khi upload thành công, bạn sẽ thấy đèn led onboard của Yolo UNO chớp tắt mỗi giây. Bạn có thể sửa lại thời gian delay từ 1000 thành 500 hoặc số tùy ý và upload lại và quan sát sự thay đổi.

Ở các lần upload sau đó thì bạn không cần phải làm thao tác đưa mạch Yolo UNO vào bootloader mode nữa.

## Cách thêm thư viện cho Arduino IDE

### Tại sao chúng ta cần thêm thư viện cho Arduino?

Có lẽ bạn đã biết, thế mạnh của Arduino nằm ở sự hỗ trợ mạnh mẽ của cộng đồng. Minh chứng dễ thấy nhất là vô số các thư viện đã được viết ra để chạy trên Arduino. Đôi lúc, bạn phải kết nối Arduino với nhiều phần cứng mà chưa được hỗ trợ sẵn bởi các thư viện mặc định có trong Arduino IDE. Lúc này, bạn sẽ phải cài đặt thêm một số loại thư viện khác.

## Cấu trúc của thư viện Arduino

Thư viện cho Arduino thường chứa file nguồn là .cpp và file .h, và thư mục examples để chứa các đoạn code ví dụ mô tả cách sử dụng thư viện đó, thường có đuôi .pde hay .ino. Trong một số trường hợp có thể có các file khác.

### Cách 1: Cách thêm thư viện cho Arduino IDE dùng Libraries Manager

Đây là cách đơn giản và tiện dụng nhất. Bạn vào **Sketch > Include Libraries > Manage Libraries...** để mở cửa sổ Library Manager. Bạn gõ tên thư viện muốn cài đặt và chọn Install.



Sau khi thư viện được cài đặt, bạn có thể thấy thư viện được xuất hiện trong danh sách các thư viện của Arduino.

Bạn có thể mở file code mẫu của thư viện để tham khảo cách sử dụng từ menu **File > Examples > Tên thư viện**.

## Cách 2: Cách thêm thư viện cho Arduino IDE từ file .zip

Nếu bạn thấy cách trên quá rắc rối, thì bạn có thể áp dụng cách thêm thư viện cho Arduino từ file .zip. Rất nhiều thư viện cho Arduino được chia sẻ ở trang web [github.com](https://github.com). Đây là trang web chứa các dự án mã nguồn mở cực kỳ phổ biến và bạn dễ dàng download toàn bộ file về dưới dạng .zip.

Ví dụ thư viện làm việc với led đèn đa màu RGB được chia sẻ ở link [https://github.com/adafruit/Adafruit\\_NeoPixel](https://github.com/adafruit/Adafruit_NeoPixel). Khi bạn vào trang này, bạn có thể click vào nút Clone or download và chọn Download ZIP.

Sau khi tải file zip của thư viện về, bạn vào **Sketch > Include Library > Add .ZIP Library...** sau đó chọn file vừa tải về để cài đặt.

## Giao tiếp dữ liệu với máy tính

Mỗi board Arduino sẽ kết nối với máy tính thông qua một cáp USB để nạp code. Tuy nhiên chúng ta cũng có thể dùng nó để giao tiếp trao đổi dữ liệu giữa board Arduino và máy tính. Board Yolo UNO sử dụng vi điều khiển có sẵn chức năng giao tiếp USB này. Phần lớn các board Arduino khác trên thị trường phải sử dụng thêm 1 chip riêng cho chức năng này, phổ biến là loại CH340C và sẽ cần cài đặt thêm driver.

Giao tiếp này thông qua giao thức gọi là kết nối serial, hay còn gọi là UART, sử dụng 2 dây là Transmit (TX) và Receive (RX).

## Gửi data từ board Arduino đến máy tính

Bạn copy đoạn code sau vào Arduino IDE:

```
int number = 0;
void setup() {
  //Initialize serial and wait for port to open:
  Serial.begin(115200);
  Serial.println("Program started");
}
void loop() {
  Serial.println(number);
  number = number + 1;
  delay(1000);
}
```

Để làm việc với giao tiếp serial, chúng ta cần sử dụng thư viện Serial. Trong hàm setup(), câu lệnh Serial.begin() giúp khởi tạo thư viện Serial. 115200 là tốc độ truyền tải dữ liệu qua lại giữa board Yolo UNO và máy tính. Bạn cần chọn đúng tốc độ này trong màn hình Serial monitor để theo dõi dữ liệu serial nhận được.

Câu lệnh Serial.println() là để in ra một dòng text trong màn hình Serial monitor. Bạn cũng có thể dùng Serial.print() để in ra nhưng không xuống hàng và các text in ra sẽ nối nhau cho đến khi có câu lệnh Serial.println().

Trong hàm loop, chúng ta sẽ in ra giá trị của biến (variable) number, sau đó tăng giá trị biến này lên 1 và tạm dừng 1000ms và lặp lại. Delay giúp cho các giá trị in ra không chạy qua nhanh khiến chúng ta rất khó nhìn.

Trước khi bạn sử dụng 1 biến trong Arduino, bạn cần khai báo nó ở đầu chương trình. Biến number được khai báo ở dòng 1 với kiểu là int (viết tắt của Integer), là một kiểu số nguyên. Ngoài ra, biến còn có thể mang giá trị thuộc nhiều kiểu dữ liệu khác nhau như char hay String (kiểu text), float (số thập phân có lẽ) hay bool (Boolean, true hay false)...

Trước khi upload chương trình, bạn cần kiểm tra và đảm bảo lựa chọn USB CDC On Boot được bật như dưới đây để có thể xem được các giá trị in ra cửa sổ Serial:



Bạn chọn đúng loại board và upload. Sau khi upload thành công, bạn mở chức năng Serial monitor và quan sát trong cửa sổ này các giá trị được in ra và tăng dần.

👤 **Biến là gì?**

*Trong lập trình, biến đóng vai trò rất quan trọng. Bạn có thể hiểu biến như là những tờ giấy nháp mà bạn dùng để ghi các giá trị tính toán trong quá trình giải 1 bài toán. Mỗi biến này sẽ có một tên không được trùng nhau và một kiểu dữ liệu của giá trị mà nó sẽ lưu trữ. Bạn nên đặt tên các biến một cách dễ hiểu, điều này sẽ giúp cho chương trình của bạn rõ ràng hơn.*

Serial sẽ là một thư viện mà bạn sẽ sử dụng rất nhiều trong các chương trình Arduino, nó giúp bạn in các giá trị hiện tại của các biến hay trạng thái của các chân của vi điều khiển... trong chương trình nhằm mục đích debug.

- [Arduino reference: Serial](#)

## Gửi data từ máy tính đến Arduino

Có một số trường hợp chúng ta cần gửi dữ liệu đến board Arduino với mục đích điều khiển. Chúng ta sẽ thử viết một chương trình cho phép nhập 1 giá trị vào cửa sổ Serial và board Yolo UNO sẽ bật tắt đèn led onboard tùy vào giá trị nhập vào là 0 hay 1.

```
void setup() {
  Serial.begin(9600);
  pinMode(D13, OUTPUT);
}
void loop() {
  // if there's any serial available, read it:
  while (Serial.available() > 0) {
    // look for the next valid integer in the incoming serial stream:
    int command = Serial.parseInt();
    // look for the newline. That's the end of your sentence:
    if (Serial.read() == '\n') {
      if (command == 1) {
        Serial.println("ON");
        digitalWrite(D13, HIGH);
      } else {
        Serial.println("OFF");
        digitalWrite(D13, LOW);
      }
    }
  }
}
```

- Trong hàm setup ta khởi tạo thư viện Serial và set mode cho chân D13 (là chân nối với led onboard) như bạn đã tìm hiểu trong các phần trước.
- Trong hàm loop(), chúng ta sử dụng 1 vòng lặp while { } để liên tục kiểm tra xem có dữ liệu nào được gửi vào Serial hay không bằng câu lệnh Serial.available(). Nếu phát hiện có dữ liệu, thì câu lệnh Serial.parseInt() sẽ giúp lấy và chuyển đổi thành kiểu int (số), đồng thời lưu vào biến command.
- Bên cạnh đó, ta cũng kiểm tra tiếp trong Serial có kí tự '\n' hay không. '\n' là kí tự xuống hàng được gửi khi ta nhấn phím Enter trên máy tính. Chúng ta dùng kí tự này để kết thúc 1 câu lệnh do user gửi đến cho Arduino.
- Sau đó ta dùng if else để kiểm tra user đang muốn bật (giá trị 1) hay tắt (giá trị 0) đèn led và dùng câu lệnh digitalWrite() để bật tắt tương ứng.

Bạn hãy thử upload đoạn code trên. Sau khi bật cửa sổ Serial monitor, bạn thử nhập vào số 0 và 1 thì sẽ thấy đèn led onboard của Yolo UNO được bật tắt tương ứng.

- [Arduino reference: While](#)
- [Arduino reference: If \(and comparison operators\)](#)
- [Arduino reference: Else](#)

## Làm việc với cảm biến ánh sáng

---

### Cảm biến ánh sáng

Cảm biến ánh sáng loại analog là loại cảm biến có thể đo cường độ ánh sáng môi trường xung quanh và chuyển đổi thành tín hiệu điện áp analog. Tín hiệu điện áp này tỷ lệ thuận với cường độ ánh sáng, nghĩa là ánh sáng càng mạnh, điện áp càng cao.

Cảm biến ánh sáng analog có nhiều loại khác nhau, sử dụng các nguyên lý hoạt động khác nhau, phổ biến nhất là:

- Cảm biến quang trở (LDR): Thay đổi điện trở khi tiếp xúc với ánh sáng.
- Photodiode: Tạo ra dòng điện khi tiếp xúc với ánh sáng.
- Phototransistor: Khuếch đại dòng điện do photodiode tạo ra.

Module cảm biến ánh sáng đi kèm trong bộ Yolo UNO Starter Kit là loại cảm biến quang trở.



## Kết nối

Kết nối cảm biến ánh sáng vào cổng A0 của Yolo UNO.

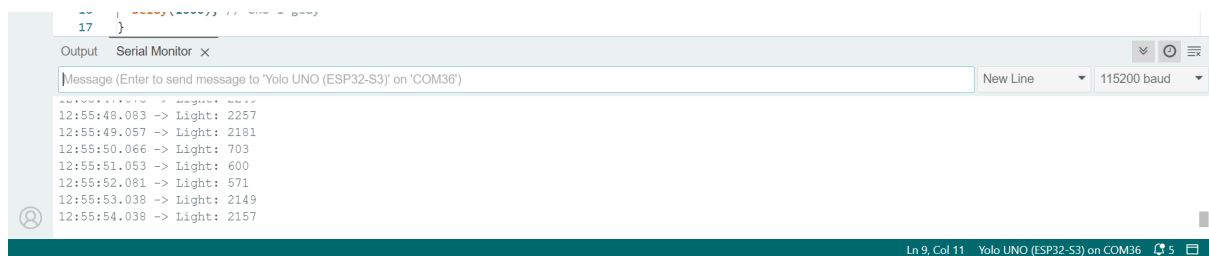
## Chương trình Arduino

```
void setup() {  
    Serial.begin(115200); // Khởi tạo Serial  
  
    // Cấu hình chân analog  
    pinMode(A0, INPUT);  
}  
  
void loop() {  
    // Đọc giá trị cảm biến  
    int sensorValue = analogRead(A0);  
  
    // In ra giá trị ánh sáng  
    Serial.print("Light: ");  
    Serial.println(sensorValue); // In với 2 chữ số thập phân  
  
    delay(1000); // Chờ 1 giây  
}
```

Trước khi upload chương trình, bạn cần kiểm tra và đảm bảo lựa chọn USB CDC On Boot được bật như dưới đây để có thể xem được các giá trị in ra cửa sổ Serial:



Sau khi upload, bạn quan sát giá trị cường độ ánh sáng được in ra cửa sổ Serial và sự thay đổi khi bạn che cảm biến lại.

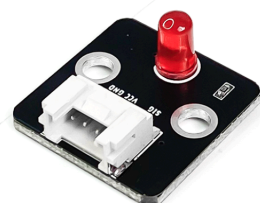


## Đèn LED tự động

Trong hướng dẫn này, bạn sẽ lập trình cho Yolo UNO để bật tắt đèn LED theo kết quả đo được từ cảm biến ánh sáng, bật khi trời tối và tắt khi trời sáng.

### Kết nối

Kết nối cảm biến ánh sáng vào cổng A0 và module đèn LED đơn sắc vào cổng D3-D4 của Yolo UNO.



## Chương trình Arduino

```
void setup()
{
    Serial.begin(115200); // Khởi tạo Serial

    // Cấu hình chân GPIO
    pinMode(D3, OUTPUT); // Chân GPIO điều khiển đèn LED
    pinMode(A0, INPUT); // Chân analog đọc cảm biến ánh sáng
}

void loop()
{
    // Đọc giá trị cảm biến
    int sensorValue = analogRead(A0);

    // Điều khiển đèn LED
    if (sensorValue < 1000) { // Bật đèn LED khi ánh sáng thấp
        digitalWrite(D3, HIGH);
    } else {
        digitalWrite(D3, LOW); // Tắt đèn LED khi ánh sáng cao
    }

    // In ra giá trị ánh sáng
    Serial.print("Light: ");
    Serial.println(sensorValue);

    delay(1000); // Chờ 1 giây
}
```

Sau khi upload chương trình, bạn quan sát giá trị cường độ ánh sáng được in ra cửa sổ Serial và trạng thái của đèn LED khi bạn che cảm biến lại.

## Lập trình PWM với module quạt mini

Trong bài này, chúng ta sẽ lập trình thay đổi tốc độ quay của module quạt mini sử dụng kỹ thuật PWM đã nhắc đến trong chương 3 ở phần bộ truyền động.



## Kết nối

Kết nối module quạt mini vào cổng D5 của Yolo UNO

## Chương trình Arduino

Để sử dụng PWM trong Arduino, ta dùng hàm ***analogWrite()***. Bạn không cần gọi hàm ***pinMode()*** để đặt chế độ OUTPUT cho chân sẽ dùng để phát xung PWM trên mạch Arduino.

Cú pháp: ***analogWrite***([chân phát xung PWM], [giá trị xung PWM]);

Giá trị mức xung PWM nằm trong khoảng từ 0 đến 255, tương ứng với mức duty cycle từ 0% đến 100%.

```
void setup() {
    // Cấu hình chân PWM
    pinMode(D5, OUTPUT);
}

void loop() {
    // Thay đổi chu kỳ duty cycle PWM
    for (int dutyCycle = 0; dutyCycle <= 255; dutyCycle++) {
        analogWrite(D5, dutyCycle); // Điều chỉnh tốc độ quay của quạt
        delay(10); // Chờ 10 mili giây
    }

    // Thay đổi chu kỳ duty cycle PWM theo hướng ngược lại
    for (int dutyCycle = 255; dutyCycle >= 0; dutyCycle--) {
        analogWrite(D5, dutyCycle); // Điều chỉnh tốc độ quay của quạt
        delay(10); // Chờ 10 mili giây
    }
}
```

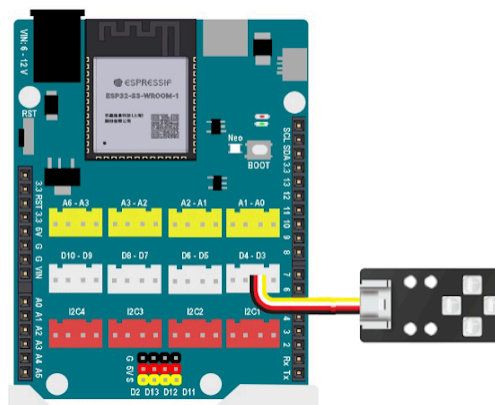
Sau khi upload, bạn sẽ thấy tốc độ quay của module quạt mini sẽ thay đổi từ 0 đến tối đa và ngược lại liên tục.

## Đổi màu đèn led RGB

Viết chương trình điều khiển module LED RGB thay đổi thành các màu khác nhau.

### Kết nối

Nối module đèn LED RGB vào cổng D3-D4 của Yolo UNO.



Bạn cũng cần cài đặt thêm thư viện **Adafruit\_NeoPixel** trong Arduino.

### Chương trình Arduino

```
#include <Adafruit_NeoPixel.h>

// khởi tạo LED màu chân D3
Adafruit_NeoPixel rgb(4, D3, NEO_GRB + NEO_KHZ800);

void setup() {
  rgb.begin();
}

void loop() {
  rgb.setPixelColor(0, rgb.Color(255,102,0));
  rgb.setPixelColor(1, rgb.Color(255,102,0));
  rgb.setPixelColor(2, rgb.Color(255,102,0));
  rgb.setPixelColor(3, rgb.Color(255,102,0));
  rgb.show();
  delay(1000);
}
```

```

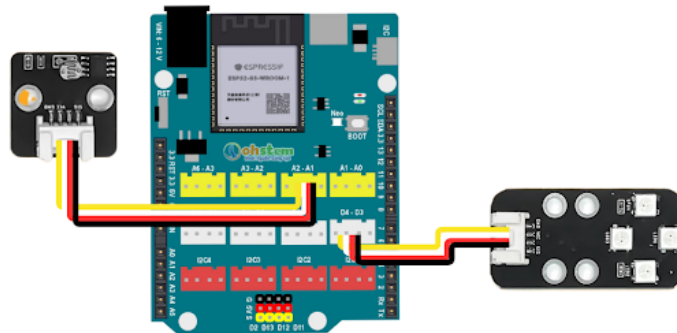
    rgb.setPixelColor(0, rgb.Color(0,0,0));
    rgb.setPixelColor(1, rgb.Color(0,0,0));
    rgb.setPixelColor(2, rgb.Color(0,0,0));
    rgb.setPixelColor(3, rgb.Color(0,0,0));
    rgb.show();
    delay(1000);
}

```

## Đèn thông minh dùng LED RGB

Sử dụng cảm biến ánh sáng (cổng A1) và LED màu (cổng D3) để điều khiển đèn LED. Khi mức độ ánh sáng thấp, đèn sẽ sáng, ngược lại khi mức độ sáng cao, đèn sẽ tắt.

### Kết nối



### Chương trình Arduino

```

#include <Arduino.h>
#include <Adafruit_NeoPixel.h>

Adafruit_NeoPixel rgb(4, D3, NEO_GRB + NEO_KHZ800);

void setup() {
    rgb.begin();
}

void loop() {
    // cảm biến ánh sáng gắn vào cổng A1

```

```

if ((analogRead(A1)/10.24 < 30)) {
  rgb.fill(rgb.Color(255,255,255));
  rgb.show();
} else {
  rgb.fill(rgb.Color(0,0,0));
  rgb.show();
}
}

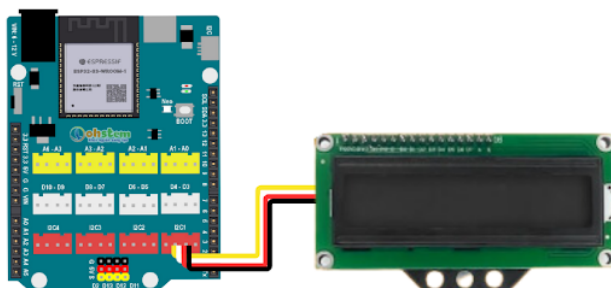
```

## Hiển thị lên màn hình LCD1602

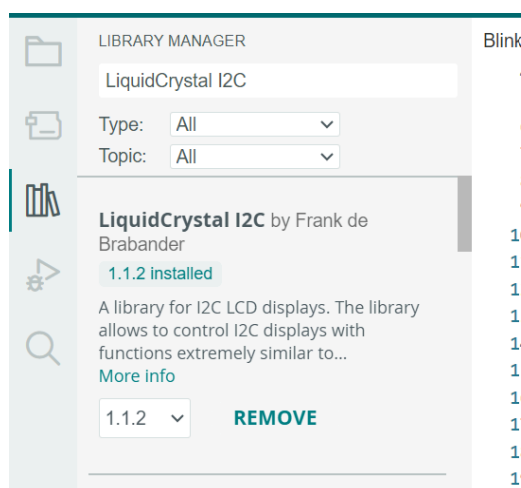
Màn hình LCD 1602 là một loại màn hình hiển thị ký tự dạng điểm ảnh, được sử dụng phổ biến trong các dự án điện tử và tự động hóa. Màn hình có kích thước 16 ký tự x 2 dòng, có thể hiển thị chữ, số và các ký tự đặc biệt. Màn hình LCD 1602 hoạt động theo nguyên tắc phản xạ ánh sáng, do đó không cần đèn nền và tiêu thụ ít điện năng.

### Kết nối

Kết nối màn hình LCD1602 với cổng I2C màu đỏ bất kỳ trên mạch YOLO UNO.



Bạn cần cài đặt thêm thư viện **LiquidCrystal\_I2C by Frank de Brabander** cho Arduino.



## Chương trình Arduino

```
#include "LiquidCrystal_I2C.h";
#include <Arduino.h>

LiquidCrystal_I2C lcd(0x21,16,2); // địa chỉ màn hình là 0x21 hexa

void setup() {
    //Initialize the LCD
    lcd.init();
    // Turn on the backlight
    lcd.backlight();
}

void loop() {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("OhStem");
    lcd.setCursor(0, 1);
    lcd.print(123);
    delay(5000);
}
```

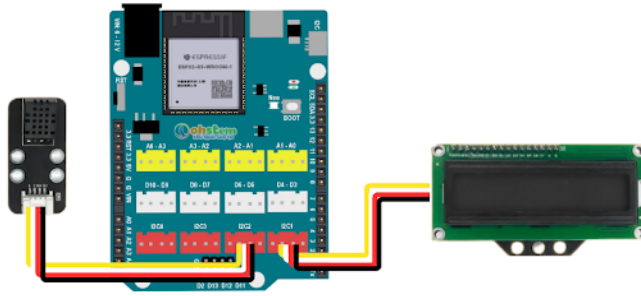
## Giám sát và hiển thị nhiệt độ

Cảm biến DHT20 là một cảm biến kỹ thuật số đo lường nhiệt độ và độ ẩm môi trường. Nó được sử dụng rộng rãi trong các ứng dụng như hệ thống điều hòa không khí, trạm thời tiết, thiết bị nhà thông minh, v.v.

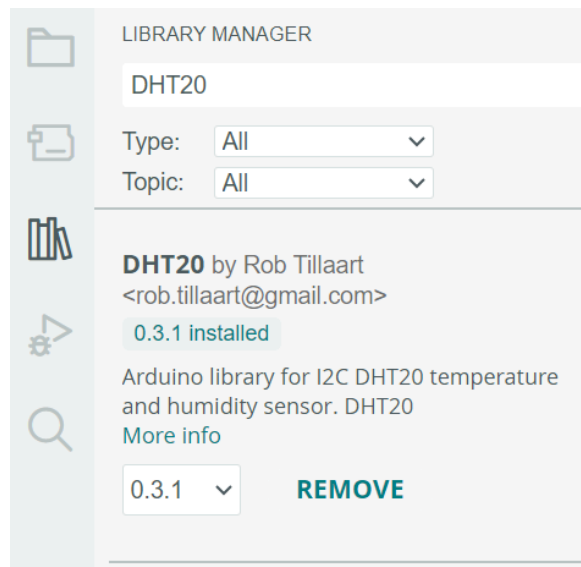
Cảm biến này có độ chính xác tốt hơn nhiều các loại DHT11 thường thấy và sử dụng kết nối I2C.

Chúng ta sẽ sử dụng DHT20 để đo và hiển thị lên trên màn hình LCD1602.

### Kết nối



Bạn cần cài đặt thư viện DHT20 cho Arduino.



## Chương trình Arduino

```
#include "LiquidCrystal_I2C.h";
#include "DHT20.h"

#include <Arduino.h>

LiquidCrystal_I2C lcd(0x21, 16, 2);
DHT20 DHT;

void setup() {
    //Initialize the LCD
    lcd.init();
    // Turn on the backlight
    lcd.backlight();
}

void loop() {
```

```

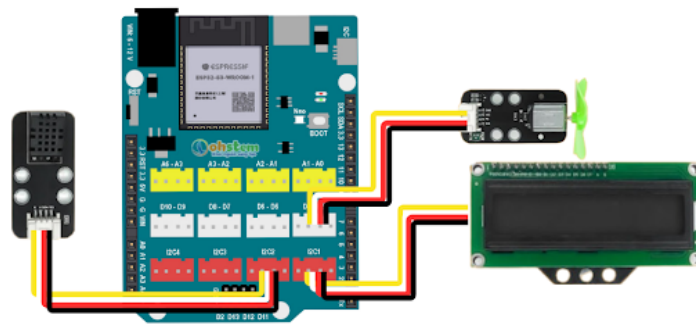
DHT.read();
lcd.clear();
lcd.setCursor(0, 0);
lcd.print((DHT.getTemperature()));
lcd.setCursor(0, 1);
lcd.print((DHT.getHumidity()));
delay(5000);
}

```

## Quạt tự động

Trong chương trình này, chúng ta sử dụng cảm biến DHT20 để đo nhiệt độ, khi nhiệt độ lớn hơn 28 sẽ bật quạt chân D7, ngược lại, nhiệt độ bé hơn 28 sẽ tắt quạt.

### Kết nối



### Chương trình Arduino

```

#include "LiquidCrystal_I2C.h";
#include "DHT20.h"

#include <Arduino.h>

LiquidCrystal_I2C lcd(0x21, 16, 2);
DHT20 DHT;

void setup() {
    //Initialize the LCD
    lcd.init();
    // Turn on the backlight
    lcd.backlight();
}

```

```

void loop() {
    lcd.clear();
    DHT.read();
    lcd.setCursor(0, 0);
    lcd.print((DHT.getTemperature()));
    lcd.setCursor(0, 1);
    lcd.print((DHT.getHumidity()));
    if ((DHT.getTemperature() > 28)) { // nhiệt độ lớn hơn 28
        analogWrite(D7, 250); // quay quạt 100%
    } else {
        analogWrite(D7, 0); // tắt quạt
    }
    delay(5000);
}

```

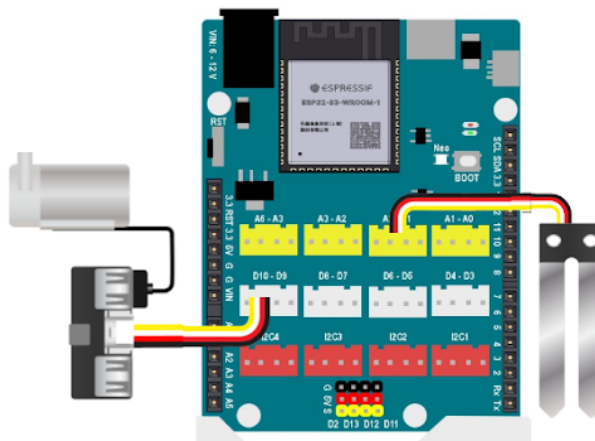
## Vườn thông minh

Trong bài này, chúng ta làm một ứng dụng nhỏ đó là giám sát độ ẩm đất và bật tắt máy bơm tự động.

### Kết nối

Để sử dụng máy bơm mini bạn cần module 2 kênh USB để kích điện áp lên 5V và kết nối vào cổng D9-D10 của Yolo UNO và máy bơm mini vào cổng USB trên module này.

Kết nối cảm biến độ ẩm đất vào cổng A1:



### Chương trình Arduino

```

void setup() {
}

void loop() {
    if ((analogRead(A1)/10.24 > 60)) { // độ ẩm đất lớn hơn ngưỡng 60%
        analogWrite(D9, 0); // tắt máy bơm
    }

    if ((analogRead(A1)/10.24 < 20)) {
        analogWrite(D9, 250); // bật máy bơm
    }
}

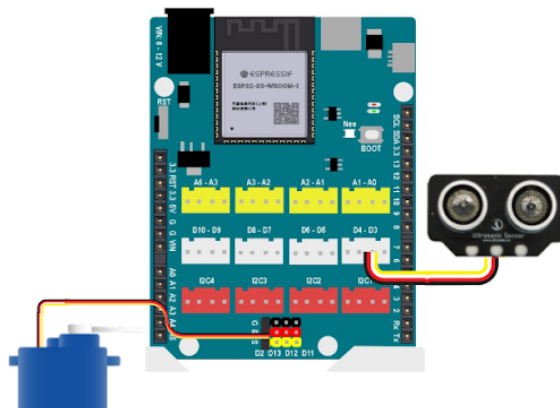
```

## Thùng rác thông minh

Chương trình này sẽ đọc khoảng cách bằng cảm biến siêu âm để phát hiện có người đứng trước thùng rác hay không. Nếu có thì sẽ quay servo để mở nắp thùng rác và đóng lại sau 3 giây.

### Kết nối

Kết nối cảm biến siêu âm ở cổng D3-D4, động cơ servo kết nối cổng D2.



Bạn cần cài đặt thư viện **HCSR04** và **SoftServo**:

**HCSR04** by Martin Susic  
<susic.martin@gmail.com>

2.0.0 installed

Library for HC-SR04 ultrasonic distance sensor. You can measure distance in centimeters.

[More info](#)

2.0.0 ▾

**REMOVE**

**SoftServo** by AlexGyver  
<alex@alexgyver.ru>

1.2 installed

Library for software servo motor control (by system timer) Library for software servo motor control (by system timer)

[More info](#)

## Chương trình Arduino

```
#include <Arduino.h>
#include "HCSR04.h";
#include "SoftServo.h";

UltrasonicDistanceSensor ultrasonic(D3, D4); //cảm biến nối với D3-D4
SoftServo myservo;

void setup() {
    myservo.attach(D2); // servo nối với cổng D2
    myservo.asyncMode();
    myservo.delayMode();

    myservo.tick();
    myservo.write(90);
}

void loop() {
    if ((ultrasonic.measureDistanceCm() < 10)) {
        myservo.tick();
        myservo.write(180);
        delay(3000);
        myservo.tick();
        myservo.write(90);
    } else {
        myservo.tick();
        myservo.write(90);
    }
}
```

## Tài liệu tham khảo

- <https://github.com/microsoft/loT-For-Beginners>
- <https://thingsboard.io/docs/>
- <https://e-ra-iot-wiki.gitbook.io/documentation>
- [https://docs.ohstem.vn/en/latest/yolo\\_uno/lam\\_quen/gioi\\_thieu\\_yolo\\_uno.html](https://docs.ohstem.vn/en/latest/yolo_uno/lam_quen/gioi_thieu_yolo_uno.html)